

Engenharia de Prompt para Devs

Um guia para aprender a usar a IA antes que a IA aprenda a usar você



Sumário

- ISBN
- Sobre o livro
- Sobre o autor
- Agradecimentos
- 1. Modelos de linguagem
- 2. Engenharia de prompt
- 3. Elaboração de prompts
- 4. Testes de prompts
- 5. Organização de prompts
- 6. IA e o programador moderno
- 7. Prompts de apoio à modelagem
- 8. Prompts de apoio à codificação
- 9. Prompts de apoio a testes e revisão de código
- 10. Prompts de apoio à documentação
- 11. Modelos de Linguagem de Código (Code Language Models)
- 12. Desafios e tendências futuras na Engenharia de Prompt
- 13. Referências

ISBN

Impresso: 978-85-5519-370-5

Digital: 978-85-5519-371-2

Dados Internacionais de Catalogação na Publicação (CIP)
(Câmara Brasileira do Livro, SP, Brasil)

Larguesa, Ricardo Pupo

Engenharia de prompt para DEVS : um guia para aprender a usar a IA antes que IA aprenda a usar você / Ricardo Pupo Larguesa ; Vivian Matsui. -- 1. ed. -- São Paulo : AOV5 Sistemas de Informática, 2024.

Bibliografia.

ISBN 978-85-5519-370-5

1. ChatGPT 2. Engenharia 3. Inteligência artificial - Inovações tecnológicas 4. Tecnologia I. Matsui, Vivian. II. Título.

24-196406

CDD-621.399

Índices para catálogo sistemático:

1. Inteligência artificial : Engenharia 621.399

Aline Grazielle Benitez - Bibliotecária - CRB-1/3129

Caso você deseje submeter alguma errata ou sugestão, acesse
<http://erratas.casadocodigo.com.br>.

Sobre o livro

Como engenheiro de computação e empreendedor, sempre estive ciente do imenso potencial da tecnologia, especialmente da Inteligência Artificial (IA). Minha experiência com IA revelou tanto suas capacidades extraordinárias quanto suas limitações inerentes. O desenvolvimento de grandes modelos de linguagem, uma façanha principalmente de grandes corporações, sempre me empolgou.

A surpresa veio quando percebi que a IA estava revolucionando não só tarefas laborais simples, mas também atividades intelectuais complexas. Isso me inspirou a escrever este livro: para compartilhar como a IA aprimorou significativamente minha produtividade e a escala do meu trabalho, não só como engenheiro de software, mas também como líder de projetos e empreendedor. Este livro é um reflexo da minha jornada e insights sobre como a IA pode ser uma ferramenta transformadora na análise e desenvolvimento de sistemas, um campo crucial em tempos de escassez profissional.

Para quem é este livro

Este livro é ideal para uma variedade de pessoas, desde profissionais da área de tecnologia até entusiastas do aprendizado de máquina e Inteligência Artificial. É particularmente valioso para programadores(as), analistas de sistemas e engenheiros(as) de software que desejam entender e aplicar modelos de linguagem e engenharia de prompt em seus projetos. Além disso, educadores e estudantes em cursos de computação ou áreas relacionadas encontrarão neste material uma fonte rica para ampliar seu conhecimento e suas habilidades práticas. Gestores de projetos de TI e líderes de equipes de desenvolvimento também se beneficiarão, ao obterem insights sobre como a IA pode aprimorar processos e produtos.

Para aproveitar ao máximo este livro, é recomendado ter conhecimentos básicos em programação e uma compreensão geral sobre o uso de ferramentas de Inteligência Artificial como ChatGPT, Bard etc. Não é necessário ser um expert, mas um entendimento fundamental da tecnologia de software e interesse em IA facilitará a compreensão e aplicação dos conceitos abordados.

Como este livro está organizado

Este livro oferece um mergulho no fascinante mundo da Inteligência Artificial, com ênfase em modelos de linguagem e engenharia de prompt nas atividades de análise e desenvolvimento de sistemas. Ao explorar os primeiros capítulos, você descobrirá os fundamentos da IA, acompanhados de uma visão histórica que contextualiza sua evolução e a emergência dos modelos de linguagem. O capítulo sobre engenharia de prompt desvenda os segredos por trás da criação de prompts eficazes, revelando como essas ferramentas influenciam o desempenho dos sistemas de IA.

À medida que avança, você será guiado pela arte da elaboração de prompts, entendendo o papel da instrução, da resposta esperada e do contexto conversacional. Os capítulos sobre testes de prompts e sua organização ampliam essa compreensão, destacando metodologias para aprimorar e sistematizar o uso de prompts em diferentes cenários.

O livro também aborda o papel vital da IA para a pessoa programadora moderna, discutindo não apenas o que a IA pode fazer por você, mas também seus limites. Este é um convite para refletir sobre o perfil emergente de analista de sistemas na era da IA.

Nos capítulos finais, você encontrará insights valiosos sobre a utilização de prompts para apoiar a modelagem, codificação, testes,

revisão de código e documentação, culminando em uma seção dedicada aos Modelos de Linguagem de Código e um olhar preditivo sobre os desafios e tendências futuras na Engenharia de Prompt.

Prepare-se para uma jornada que não apenas enriquece o conhecimento técnico, mas também instiga a reflexão crítica sobre o futuro da tecnologia e seu impacto na sociedade e no desenvolvimento profissional.

Sobre o autor

Ricardo Pupo Larguesa é um apaixonado não só pela tecnologia em si, mas também pelo que ela pode proporcionar. Pai, marido, filho e nerd, é engenheiro de computação formado pela Unisanta, com pós-graduação pela UFU e MBA pela FGV. Atua como professor desde 2002, tendo lecionado em instituições como Senac, IFS e FATEC. Seu envolvimento no Centro Paula Souza (CPS), incluindo a participação na criação da Agência de Inovação do CPS, destaca sua contribuição significativa para a educação e inovação tecnológica. Ricardo é também fundador da T2S, uma empresa brasileira que atua desde 2003 no desenvolvimento de sistemas de logística portuária no Brasil e no exterior, que reflete seu compromisso com a criação de soluções tecnológicas de alto desempenho.

Links das redes sociais

Perfis profissionais

- <http://lattes.cnpq.br/7815220582960179>
- <https://www.linkedin.com/in/rlarguesa>

Canal Reprogramando

- <https://www.youtube.com/@reprogramando>
- <https://www.facebook.com/canal.reprogramando>
- <https://www.instagram.com/canal.reprogramando>
- <https://www.tiktok.com/@canal.reprogramando>

Agradecimentos

Agradeço, antes de tudo, à minha esposa, minha filha e meu filho pela paciência, união e felicidade que me proporcionam no dia a dia.

Agradeço ao meu falecido pai, à minha mãe, aos meus irmãos, parentes, amigos e colegas pela influência tão positiva na minha formação como pessoa e profissional. Um agradecimento especial também aos professores que fizeram parte da minha jornada e sem os quais eu jamais conseguiria acumular o conhecimento que adquiri.

Não poderia deixar de agradecer também à Vivian Matsui, minha editora, que teve tanta paciência comigo e que tanto colaborou nesta minha primeira experiência como autor.

A todos vocês, meu muito obrigado.

CAPÍTULO 1

Modelos de linguagem

"A inteligência artificial pode descobrir soluções para problemas do passado, que mudarão a nossa forma de resolver os mesmos problemas no futuro" — Dawiny Bastos

Imagine uma biblioteca abstrata, repleta de livros que acumulam o conhecimento e a sabedoria da humanidade, juntamente de suas idiossincrasias, piadas e até erros gramaticais. E você nem sequer precisa ver os livros, pois toda essa informação pode ser acessada simplesmente por meio de uma... conversa. Essa não é a trama de um filme de ficção científica, mas sim uma analogia bastante próxima do que os modelos de linguagem modernos oferecem atualmente.

No final dos anos 1950, Alan Turing, o pai da Ciência da Computação, lançou um desafio ousado: ele imaginou um futuro em que as máquinas poderiam simular inteligência humana tão bem que um juiz humano não conseguiria distinguir entre as respostas de uma máquina e de uma pessoa. Esse foi o pontapé inicial para uma corrida repleta de obstáculos, sucessos e fracassos notáveis. Hoje, estamos muito mais próximos de realizar essa visão do que jamais estivemos, graças aos modelos de linguagem.

Mas o que é exatamente um modelo de linguagem? E por que essa tecnologia é tão relevante para campos tão diversos quanto Medicina, Direito, Jornalismo e até mesmo para o cidadão comum?

Neste capítulo, vou oferecer um panorama conciso da **Inteligência Artificial** e dos **modelos de linguagem**, ressaltando as tecnologias contemporâneas e emergentes que desencadearam a inovação revolucionária com a qual convivemos hoje. O objetivo aqui não é mergulhar na criação ou otimização de um modelo específico,

mas fornecer uma visão ampla que permita a um usuário experiente compreender, ainda que de maneira abstrata, a mecânica subjacente a um sofisticado modelo de linguagem.

1.1 Conceitos e história da Inteligência Artificial

A **Inteligência Artificial** (IA) transcende a simples programação de computadores para abranger a capacidade de máquinas e software em simular funções cognitivas semelhantes às humanas. Esse ramo da Ciência da Computação não se dedica apenas à criação desses sistemas autônomos, mas também a entender os princípios subjacentes que regem a inteligência, seja ela humana, animal ou mecânica.

O estudo formal do raciocínio iniciou com filósofos e matemáticos na antiguidade, evoluindo até a teoria da computação de Alan Turing. No entanto, a IA como disciplina acadêmica nasceu em 1956 em um workshop em Dartmouth College. Nos anos subsequentes, houve períodos alternados de otimismo e desapontamento, muitas vezes seguidos de cortes de financiamento, mais notavelmente durante os chamados "invernos da IA". O renascimento da IA ocorreu nos anos 80 com o sucesso comercial dos sistemas especialistas e ganhou tração renovada em 2012 com o avanço do **aprendizado profundo** (*Deep Learning*). Essa técnica, apoiada por melhorias de hardware e acesso a grandes volumes de dados, tem dominado o campo desde então.

A disciplina da IA é multifacetada e abrange diversas subáreas:

- **Raciocínio:** algoritmos que mimetizam o pensamento lógico humano, até mesmo quando confrontados com incertezas ou dados incompletos;

- **Representação de conhecimento:** estratégias para modelar o entendimento de fatos e relações no mundo, permitindo que as IAs respondam de forma perspicaz a perguntas complexas;
- **Planejamento e tomada de decisão:** mecanismos para definir e alcançar metas com base em uma análise profunda de variáveis e cenários;
- **Aprendizado automático:** estudo de como máquinas podem aprimorar suas habilidades ou conhecimento de forma autônoma;
- **Processamento de linguagem natural:** técnicas que possibilitam a comunicação em linguagem humana, abrangendo desde a leitura e escrita até a interpretação contextual;
- **Percepção:** algoritmos que captam e processam dados sensoriais, como visão, som e toque, para interpretar e interagir com o ambiente.

Aplicações de IA permeiam várias esferas da vida moderna. Motores de busca como o Google Search vão além da mera indexação de informações, usando algoritmos avançados para entender a intenção do usuário. Plataformas como YouTube, Amazon e Netflix utilizam sistemas de recomendação extremamente refinados para personalizar a experiência do usuário. Assistentes de voz, como Siri e Alexa, não apenas reconhecem palavras, mas também entendem contexto e intenção. Carros autônomos como os da Waymo são capazes de navegar no mundo real, adaptando-se a imprevisíveis situações de tráfego.

Mais recentemente, uma das inovações mais impactantes na IA são os *modelos de transformadores*, que têm mostrado desempenho notável em diversas modalidades como texto, imagem e áudio. Esses modelos são a base de sistemas como o GPT e o CLIP, da OpenAI, e têm aplicações que vão desde a geração de texto até a análise de imagens.

No entanto, o desafio mais significativo na integração dessas diferentes modalidades não é apenas técnico, mas também de experiência do usuário (UX). Determinar a interface de usuário mais

eficaz para apresentar essas diferentes formas de saída — seja áudio, texto, imagens ou código — é uma questão em aberto. Isso inclui entender como os usuários podem iterar nesses modelos e fornecer feedback, o que é especialmente relevante quando se trata de ajustar um modelo multimodal. O UX adequado pode fazer a diferença entre uma tecnologia que é simplesmente poderosa e uma que é também acessível e intuitiva.

Hoje, há diferentes modalidades de interface para os modelos transformadores:

- Text-to-Text (OpenAI ChatGPT);
- Text-to-Image (Stable Diffusion);
- Image-to-Text (Open AI CLIP);
- Speech-to-Text (OpenAI Whisper);
- Text-to-Speech (Meta's Massively Multilingual Speech);
- Image-to-Image (img2img or pix2pix);
- Text-to-Audio (Meta MusicGen);
- Text-to-Code (OpenAI Codex / GitHub Copilot);
- Code-to-Text (ChatGPT etc.).

Exemplares na área de geração criativa, como ChatGPT e Midjourney, estão até mesmo desafiando nossas noções tradicionais sobre criatividade. Inclusive, graças à popularidade dessas novas aplicações, a sociedade está sendo inundada com as mais diversas ferramentas de Inteligência Artificial. Dentre inúmeros repositórios, há um portal que promete ser o maior diretório de ferramentas de Inteligência Artificial, atualizado diariamente: o Futurepedia (<https://www.futurepedia.io/>).

Mas ainda há desafios. Em minha opinião, o maior deles é o aprendizado contínuo, mas ainda há dificuldade em implantar conhecimento comum, a imprevisibilidade inerente ao mundo real, e a enorme complexidade que envolve o planejamento e a tomada de decisão em ambientes não estruturados.

1.2 O que são modelos de linguagem

Modelos de linguagem são ferramentas probabilísticas que analisam e geram sequências de palavras com base em dados textuais. Originalmente, os modelos mais simples eram baseados em estatísticas puras, como o modelo de n -gramas, onde a probabilidade de cada palavra subsequente era calculada com base em uma janela de palavras anteriores.

A era das redes neurais trouxe inovações significativas nesse campo. Modelos de redes neurais recorrentes (RNN) superaram modelos estatísticos, oferecendo uma maneira mais eficaz de capturar relações temporais em sequências de texto. No entanto, os modelos de linguagem de grande porte, que utilizam arquiteturas como os transformadores, representam o estado atual da arte, com desempenho excepcional em tarefas diversas.

Esses modelos avançados são particularmente úteis em uma série de aplicações tecnológicas, tais como:

- **Reconhecimento de fala:** auxiliam na previsão de sequências de palavras mais prováveis, minimizando erros;
- **Tradução automática:** aumentam a precisão e a naturalidade do texto traduzido;
- **Geração de texto natural:** criam textos que se assemelham mais à linguagem humana, sendo úteis em *chatbots*, por exemplo;
- **Reconhecimento óptico de caracteres e de escrita à mão:** melhoram a precisão na conversão de texto escaneado ou escrito à mão para texto digital.

Apesar do progresso impressionante, também há desafios específicos a serem enfrentados pelos modelos de linguagem, incluindo a necessidade de métodos de avaliação robustos e a minimização de preconceitos inerentes nos dados de treinamento.

Modelos de linguagem de grande porte

Modelos de linguagem de grande porte (LLMs - *Large Language Models*) são modelos de linguagem que se caracterizam por seu tamanho. Seu tamanho é possibilitado por aceleradores de IA, que são capazes de processar grandes quantidades de dados textuais, a maioria dos quais raspada da internet. Eles são redes neurais artificiais que podem conter dezenas de milhões e até bilhões de pesos, e são (pré-)treinados usando aprendizagem autossupervisionada e semissupervisionada. A arquitetura do transformador contribuiu para um treinamento mais rápido.

Como modelos de linguagem, eles trabalham tomando um texto de entrada e repetidamente prevendo o próximo *token* ou palavra. Até 2020, o ajuste fino era a única maneira de um modelo ser adaptado para ser capaz de realizar tarefas específicas. No entanto, modelos de tamanho maior, como o GPT-3, podem ser otimizados com engenharia de prompt para obter resultados semelhantes. Esses modelos são projetados para aprender as regras da gramática, o significado das palavras e o conhecimento geral da forma como usamos a linguagem. No entanto, eles também podem captar erros e preconceitos presentes nos textos que processam.

Exemplos notáveis incluem os modelos GPT da OpenAI (por exemplo, GPT-3.5 e GPT-4, usados no ChatGPT), o PaLM do Google (usado no Bard) e o LLaMa da Meta, bem como BLOOM, Ernie 3.0 Titan e Claude.

Os dados usados para treinar LLMs são normalmente coletados da internet e incluem uma ampla variedade de fontes, como livros, artigos, páginas da web, redes sociais e fóruns. O pré-processamento dos dados é uma etapa importante para garantir que os modelos sejam treinados em dados consistentes e de alta qualidade.

O pré-processamento de dados para LLMs geralmente inclui as seguintes etapas:

- **Tokenização:** os dados são divididos em unidades menores, chamadas de tokens. Os tokens podem ser palavras, números, símbolos ou outras sequências de caracteres;
- **Limpeza:** os dados são analisados para remover conteúdo indesejado, como erros de digitação, conteúdo ofensivo ou spam;
- **Normalização:** os dados são normalizados para que todos os tokens tenham o mesmo formato, por exemplo, convertendo todos os caracteres maiúsculos para minúsculos;
- **Codificação:** os tokens são codificados em números ou outros valores numéricos para que possam ser processados pelos modelos.

Os LLMs são treinados com um método segundo o qual eles aprendem com exemplos rotulados. Esses exemplos têm uma 'entrada' e uma 'saída' certa. Por exemplo, para treinar um modelo de tradução, usamos frases em português e suas traduções em inglês. Durante o treinamento, o modelo ajusta suas configurações para que suas respostas fiquem o mais próximo possível das respostas corretas. Esse ajuste é feito usando diversos algoritmos específicos. Treinar LLMs pode levar tempo e ser caro. Isso acontece porque eles precisam aprender com uma grande quantidade de informações complicadas. Além disso, às vezes os modelos aprendem demais com os dados de treino e têm dificuldade para aplicar esse conhecimento em situações novas.

LLMs normalmente usam algo chamado *arquitetura do transformador*. Essa estrutura é uma forma mais eficaz de rede neural, que ajuda os modelos a entenderem as conexões entre as palavras ou símbolos em um texto. Essa eficiência torna possível treiná-los com mais dados e informações mais complicadas do que antes.

LLMs possuem uma vasta variedade de usos, abrangendo desde a criação de textos inéditos e traduções até a composição de sínteses, elaboração de resumos, formulação de perguntas e respostas, categorização de textos e simulação de comportamentos.

Modelos de linguagem populares e suas características

Vários modelos de linguagem têm sido desenvolvidos e aprimorados ao longo dos anos, cada um com suas próprias características e capacidades. Alguns dos modelos de linguagem mais populares e suas características distintas são:

- **BERT (*Bidirectional Encoder Representations from Transformers*)**: desenvolvido pelo Google AI e apresentado em 2018, o BERT é um modelo de linguagem pré-treinado baseado em *transformers* que é projetado para lidar com tarefas de Processamento de Linguagem Natural (PLN), como análise de sentimento, tradução automática e preenchimento de lacunas. Uma das características únicas do BERT é o seu treinamento bidirecional, o que permite que ele entenda melhor o contexto de palavras dentro de uma frase.

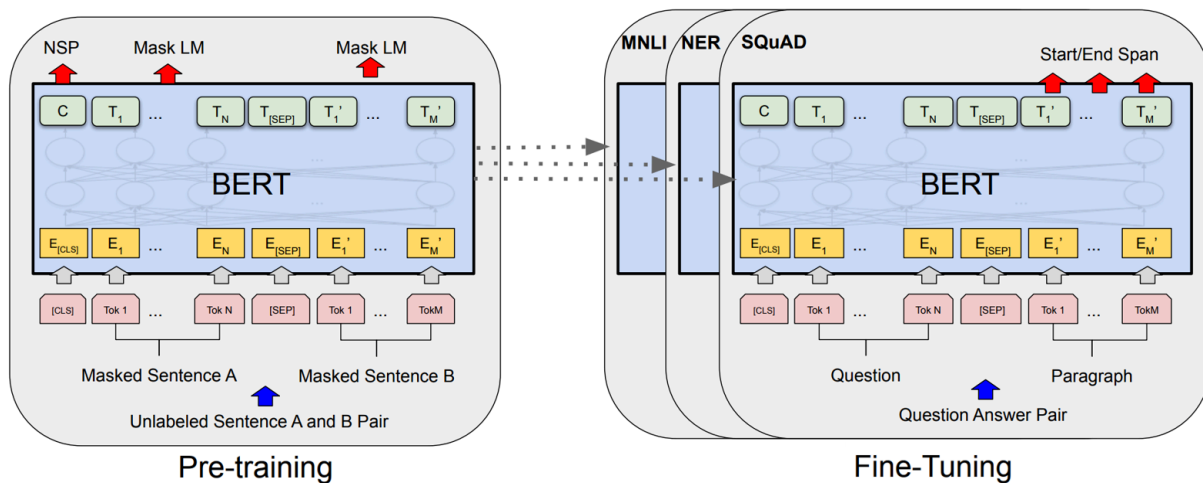


Figura 1.1: Processo geral de pré-treinamento e ajuste fino para o modelo BERT

Extraído do artigo *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* (DEVLIN *et al.*, 2018), disponível em <https://arxiv.org/abs/1810.04805>.

- **GPT (*Generative Pre-trained Transformer*)**: lançado em 2020, o GPT e suas versões subsequentes, como GPT-2, GPT-3

e GPT-4, são desenvolvidos pela OpenAI. Esses modelos são baseados na arquitetura Transformer e são pré-treinados em grandes conjuntos de dados de texto. É treinado para prever qual será o próximo token com base nos tokens anteriores. E token é uma unidade mínima de informação textual, podendo ser uma palavra, subpalavra ou até mesmo um único caractere, dependendo da forma como o modelo foi treinado e como os dados são divididos. O GPT é conhecido por sua capacidade de gerar texto coerente e relevante, e tem sido amplamente utilizado em uma variedade de aplicações, incluindo geração de texto, respostas a perguntas e tradução automática.

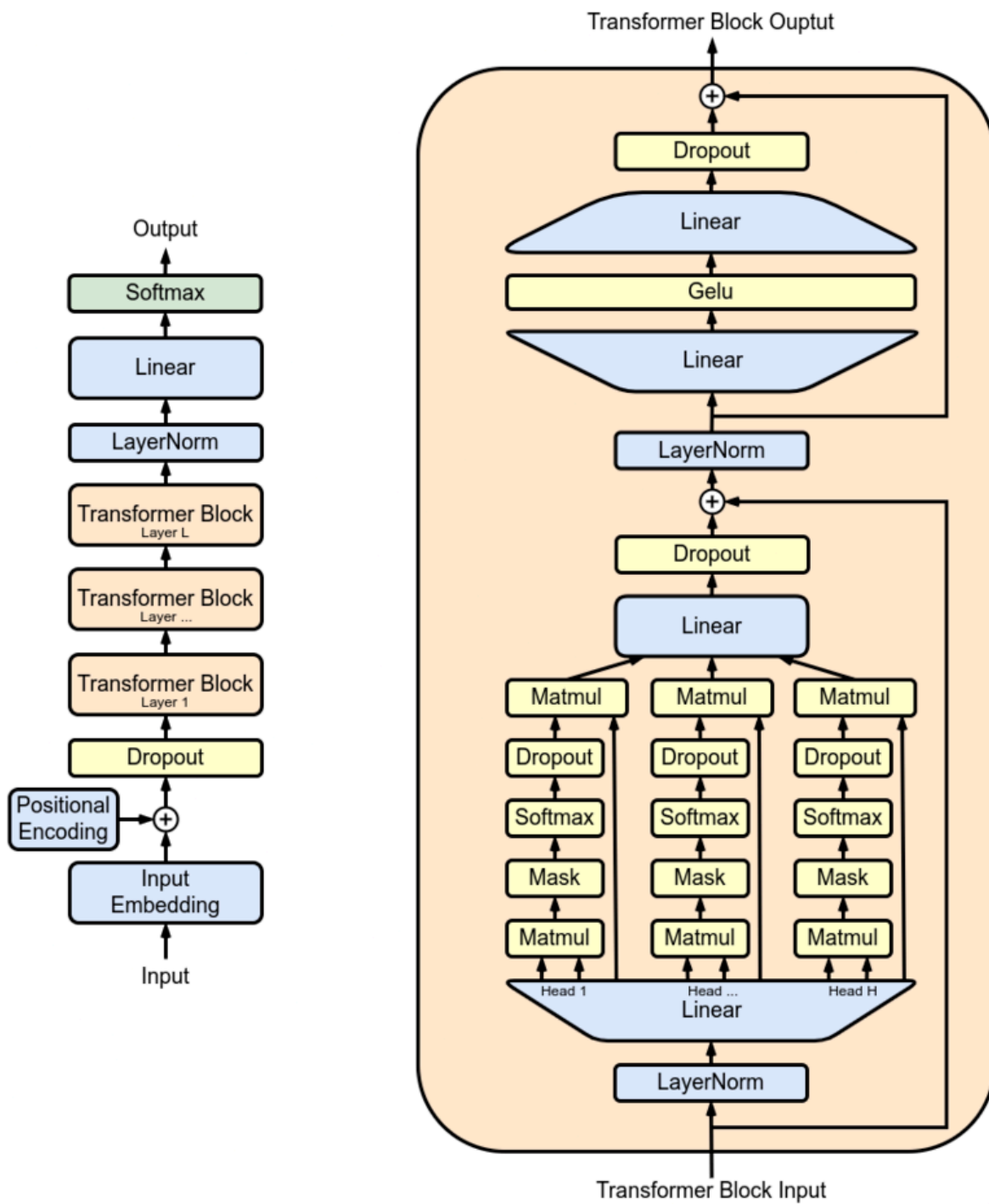


Figura 1.2: Modelo original do GPT

Extraído do artigo *Generative pre-trained transformer*, da Wikipédia, disponível em:

https://en.wikipedia.org/wiki/Generative_pre-trained_transformer.

Outros modelos que valem a pena ser mencionados são:

- **RoBERTa (*Robustly Optimized BERT Pretraining Approach*)**: o RoBERTa é uma variante do BERT que foi otimizado para melhorar o desempenho em tarefas de PLN. Ele utiliza um processo de treinamento diferente e maior quantidade de dados em comparação com o BERT original. A otimização do RoBERTa resulta em melhor desempenho em várias tarefas de PLN, como análise de sentimento, respostas a perguntas e classificação de texto;
- **T5 (*Text-to-Text Transfer Transformer*)**: desenvolvido pelo Google AI, o T5 é um modelo de linguagem baseado em *transformers* que aborda todas as tarefas de PLN como problemas de tradução texto para texto. Isso permite que o T5 seja treinado e ajustado para uma ampla gama de tarefas com uma abordagem unificada. O T5 é conhecido por seu desempenho de ponta em várias tarefas, incluindo respostas a perguntas, geração de resumo e tradução automática;
- **XLNet (*Generalized Autoregressive Pretraining for Language Understanding*)**: o XLNet é um modelo de linguagem pré-treinado que combina os benefícios do treinamento autorregressivo e bidirecional. Ao contrário do BERT, o XLNet é capaz de capturar informações contextuais de maneira mais eficiente, o que resulta em melhor desempenho em tarefas como análise de sentimento, preenchimento de lacunas e respostas a perguntas.

Esses são apenas alguns exemplos dos muitos modelos de linguagem disponíveis atualmente. Ao escolher um modelo de linguagem para suas necessidades, é importante considerar suas características e capacidades, bem como a aplicação específica em que você planeja usá-lo. Algumas aplicações podem se beneficiar mais de um modelo específico, enquanto outras podem exigir a combinação de vários modelos para obter os melhores resultados.

Adaptação e personalização de modelos de linguagem

"Para não ser substituído por um robô, não seja um robô" —
Martha Gabriel

Embora a implementação de um modelo não seja o propósito deste livro, se este assunto lhe interessa e você deseja implementar seu próprio modelo de linguagem, é possível que você queira adaptá-los e personalizá-los para atender a requisitos específicos. Essa personalização pode melhorar a precisão e a eficiência do modelo em suas tarefas de processamento de linguagem natural.

Aqui estão algumas estratégias para adaptar e personalizar modelos de linguagem para suas necessidades:

- ***Fine-tuning* (ajuste fino):** método comum para adaptar um modelo de linguagem pré-treinado a uma tarefa específica ou domínio. O ajuste fino envolve o treinamento do modelo em um conjunto de dados relacionado à sua aplicação, permitindo que o modelo aprenda as nuances e características específicas desse domínio. Ao ajustar o modelo, você pode melhorar sua precisão e desempenho em tarefas específicas, como classificação de texto, geração de texto ou análise de sentimento;
- **Transferência de aprendizado:** é uma técnica que permite que um modelo treinado em uma tarefa seja adaptado para outra tarefa relacionada. Isso é feito compartilhando informações e conhecimentos entre as duas tarefas. A

transferência de aprendizado é particularmente útil quando você possui poucos dados rotulados para sua tarefa específica, pois o modelo pode aproveitar o conhecimento aprendido em outra tarefa para melhorar seu desempenho;

- **Modelos específicos de domínio:** para algumas aplicações, pode ser útil treinar um modelo de linguagem específico do domínio do zero. Isso geralmente é feito quando a aplicação requer um vocabulário ou uma estrutura de conhecimento muito específicos, que não podem ser facilmente adaptados a partir de modelos de linguagem genéricos. Modelos específicos de domínio podem ser mais precisos e eficientes em suas tarefas, mas também podem exigir mais recursos e tempo de treinamento;
- **Ajuste de hiperparâmetros:** os hiperparâmetros do modelo, como a taxa de aprendizado, o tamanho do lote e o número de camadas, podem ser ajustados para melhorar o desempenho do modelo em sua aplicação específica. Ajustar os hiperparâmetros corretamente pode resultar em melhor desempenho e menor tempo de treinamento, mas também pode ser um processo demorado e exigir experimentação;
- **Incorporação de conhecimento externo:** em alguns casos, pode ser útil incorporar conhecimento externo, como ontologias, bases de conhecimento ou recursos lexicais, ao seu modelo de linguagem. Isso pode melhorar a precisão e a capacidade do modelo de lidar com informações específicas do domínio e fornecer respostas mais ricas e contextuais.

A combinação dessas estratégias pode ajudar você a criar soluções de processamento de linguagem natural altamente eficientes e personalizadas para suas aplicações específicas. Ao adaptar e personalizar modelos de linguagem para suas necessidades, é importante estar consciente de que se trata de um empreendimento muito complexo, que exigirá um alto investimento de tempo e dinheiro.

Há uma lista enorme de modelos de linguagem abertos e gratuitos, inclusive para uso comercial. Dentre inúmeros repositórios, há um índice mantido por Eugene Yan em um repositório centralizado do GitHub, que pode ser acessado por este link:

<https://github.com/eugeneyan/open-llms>.

Este outro repositório é ainda mais completo e contém o Vicuna, que eu, particularmente, testei e gostei muito:

<https://github.com/Hannibal046/Awesome-LLM>.

Neste primeiro capítulo, abordamos uma introdução abrangente à Inteligência Artificial e aos modelos de linguagem, incluindo alguns exemplos como BERT e GPT. Discutimos suas aplicações e possibilidade de personalização e adaptação. Com esse conhecimento em mãos, você está agora preparado(a) para o próximo tópico de estudo: a *engenharia de prompt*. No capítulo subsequente, focaremos em técnicas e métodos para obter o máximo proveito de um modelo de linguagem.

CAPÍTULO 2

Engenharia de prompt

"Diz-se da melhor companhia: a sua conversa é instrutiva, o seu silêncio, formativo" — Johann Goethe

2.1 Definição e conceito de engenharia de prompt

Prompt é uma expressão de origem inglesa que tem vários significados. No campo da Tecnologia da Informação, geralmente se refere ao "prompt de comando" ou "linha de comando". Nesse contexto, trata-se de uma interface de texto onde os usuários inserem comandos para interagir com um sistema operacional ou programa.

A origem do termo "prompt" na computação é difusa, mas sabe-se que seu uso começou com as primeiras interfaces de linha de comando, quando os computadores iniciaram a interação com os usuários por meio de texto, substituindo os antigos cartões perfurados. A identificação precisa de quando e por quem o termo foi usado pela primeira vez é desafiadora, devido à evolução natural da indústria de computação e à ausência de registros consistentes durante seus estágios iniciais. Ainda assim, o termo é amplamente reconhecido e usado no campo da TI.

Quando utilizada como verbo, a palavra "prompt" pode ser traduzida para o português como "incitar", "motivar" ou "estimular". Na esfera da Tecnologia da Informação, esse significado se mantém relevante, pois o "prompt" é, em sua essência, um estímulo ou convite que solicita uma interação do usuário. Quando um sistema apresenta um

"prompt", está na realidade incitando o usuário a fornecer informações, responder a uma pergunta ou executar uma ação, facilitando a interação entre o humano e a máquina.

No prompt de comando, os usuários podem digitar comandos que acionam tarefas específicas, tais como criar diretórios, remover arquivos, mover arquivos ou executar programas. Para os administradores de sistemas e desenvolvedores, o prompt de comando é uma ferramenta útil, pois permite um controle mais direto sobre o computador e a execução de tarefas avançadas.

Assim, o termo "prompt" indica o ato de induzir ou incentivar alguém a fazer algo ou responder a uma pergunta. Nesse sentido, pode ser aplicado tanto em diálogos de pessoas quanto em programação. Por exemplo, um programa pode "incitar" (ou "prompt") o usuário a fornecer mais informações ou uma resposta específica para completar uma tarefa. Da mesma forma, o usuário pode fornecer um prompt ao programa na forma de comandos ou respostas. E assim se inicia uma interação recorrente entre humano e computador. Em ambos os casos, a função do prompt é facilitar a interação entre o usuário e o programa, ajudando na comunicação e na execução das tarefas.

É importante ressaltar que a natureza específica de um prompt pode variar dependendo do contexto. Em diferentes sistemas operacionais, linguagens de programação ou aplicativos, os prompts podem assumir formas diversas. Apesar dessas variações, todos os prompts compartilham o objetivo comum de facilitar a comunicação entre o usuário e o programa. Eles são projetados para serem intuitivos e de fácil compreensão, permitindo que os usuários, independentemente de seu nível de conhecimento técnico, possam interagir efetivamente com o sistema.

Nos dias de hoje, o conceito de "prompt" mantém sua relevância, sendo empregado especialmente no contexto de Interfaces de Linha de Comando (CLI). CLI é um método que permite aos usuários interagir com um software ou sistema operacional por meio de linhas

de texto, em vez de interfaces gráficas. Em um CLI, o "prompt" é a solicitação que o sistema faz para que o usuário digite um comando. Ele é um elemento crucial que permite aos usuários controlar softwares e sistemas com precisão, podendo ser encontrado em diversas aplicações, desde sistemas operacionais até ferramentas de desenvolvimento modernas.

Em um contexto mais atual e com foco na Inteligência Artificial (IA), o conceito de "prompt" é ainda mais essencial. Na IA, um "prompt" funciona como um gatilho ou diretriz para sistemas baseados em aprendizado de máquina. Em essência, um prompt orienta a IA sobre que tipo de informação ou resposta ela deve produzir. Por exemplo, se você tem um modelo de linguagem, um prompt pode ser uma frase ou pergunta que você fornece ao modelo para gerar uma resposta. A habilidade de criar prompts eficazes é uma parte crucial do uso de tecnologias de IA, pois facilita a comunicação e colaboração entre humanos e sistemas de IA, já que a interação com esse tipo de tecnologia é muito mais abstrata do que com programas convencionais.

Podemos considerar que uma combinação de razões práticas, teóricas e históricas tenha contribuído para a adoção do prompt como interface padrão em IA. Dentre as razões, podemos citar:

- **Semelhança com a interação humana:** a interação via chat é uma forma intuitiva e natural de comunicação para humanos. Ao projetar sistemas de IA para interagir por meio de prompts textuais, as pessoas desenvolvedoras podem aproveitar essa familiaridade e criar interfaces de usuário que são acessíveis e fáceis de usar;
- **Versatilidade:** o texto é uma forma de dados que pode representar uma ampla variedade de instruções, perguntas e comandos. Isso torna os prompts de texto uma escolha versátil para interfaces de IA;

- **Flexibilidade:** os prompts permitem uma interação bidirecional flexível entre o usuário e a IA. O usuário pode emitir uma variedade de comandos e perguntas, e a IA pode responder de maneira correspondente. Isso permite um alto grau de personalização e adaptabilidade;
- **Eficiência em termos de dados:** trabalhar com texto pode ser mais eficiente em termos de dados do que outras formas de interação, como voz ou vídeo, que podem exigir mais largura de banda e capacidade de processamento;
- **Tradição:** o uso de prompts textuais tem raízes profundas na história da computação e remonta ao uso de linhas de comando em sistemas operacionais. Essa tradição pode ter influenciado a escolha do prompt como interface padrão em IA.

Veja a seguir alguns exemplos de prompts utilizados em plataformas de Inteligência Artificial:

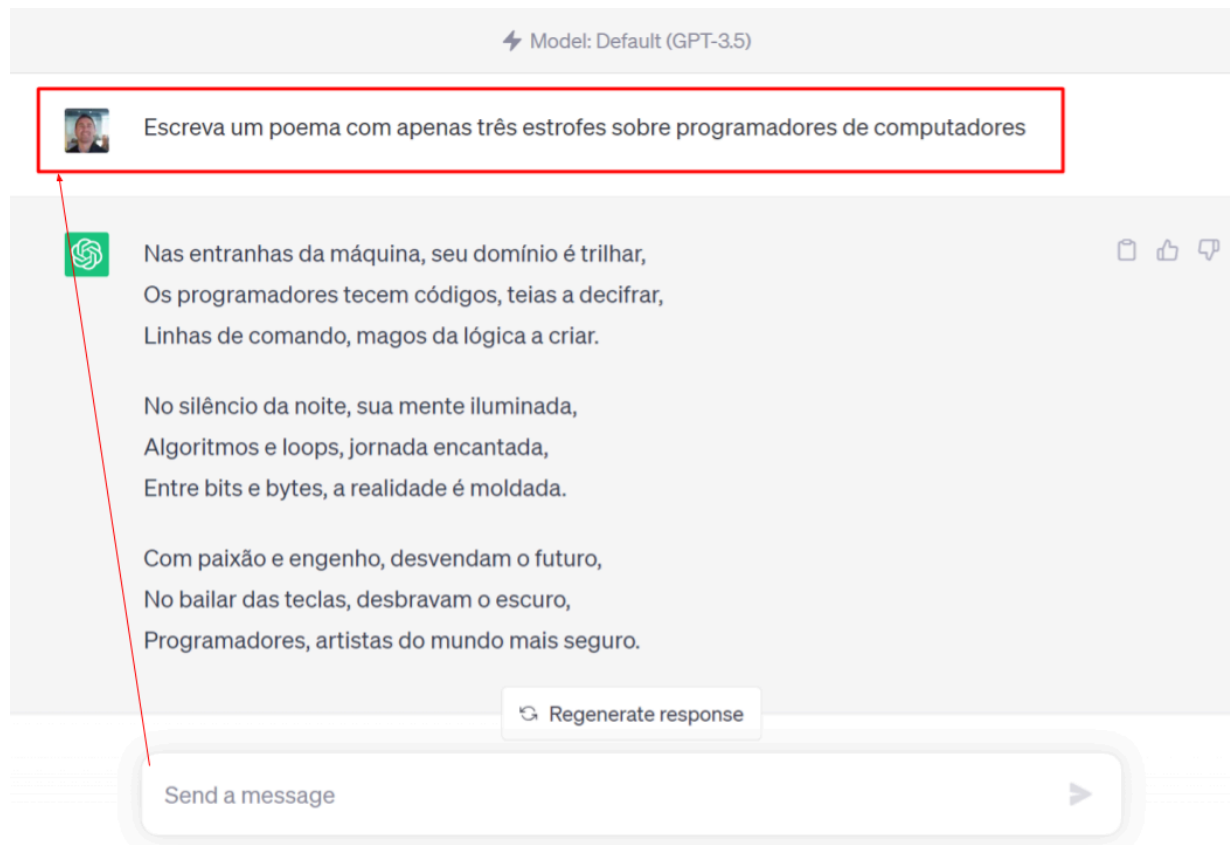


Figura 2.1: Exemplo de um prompt do ChatGPT

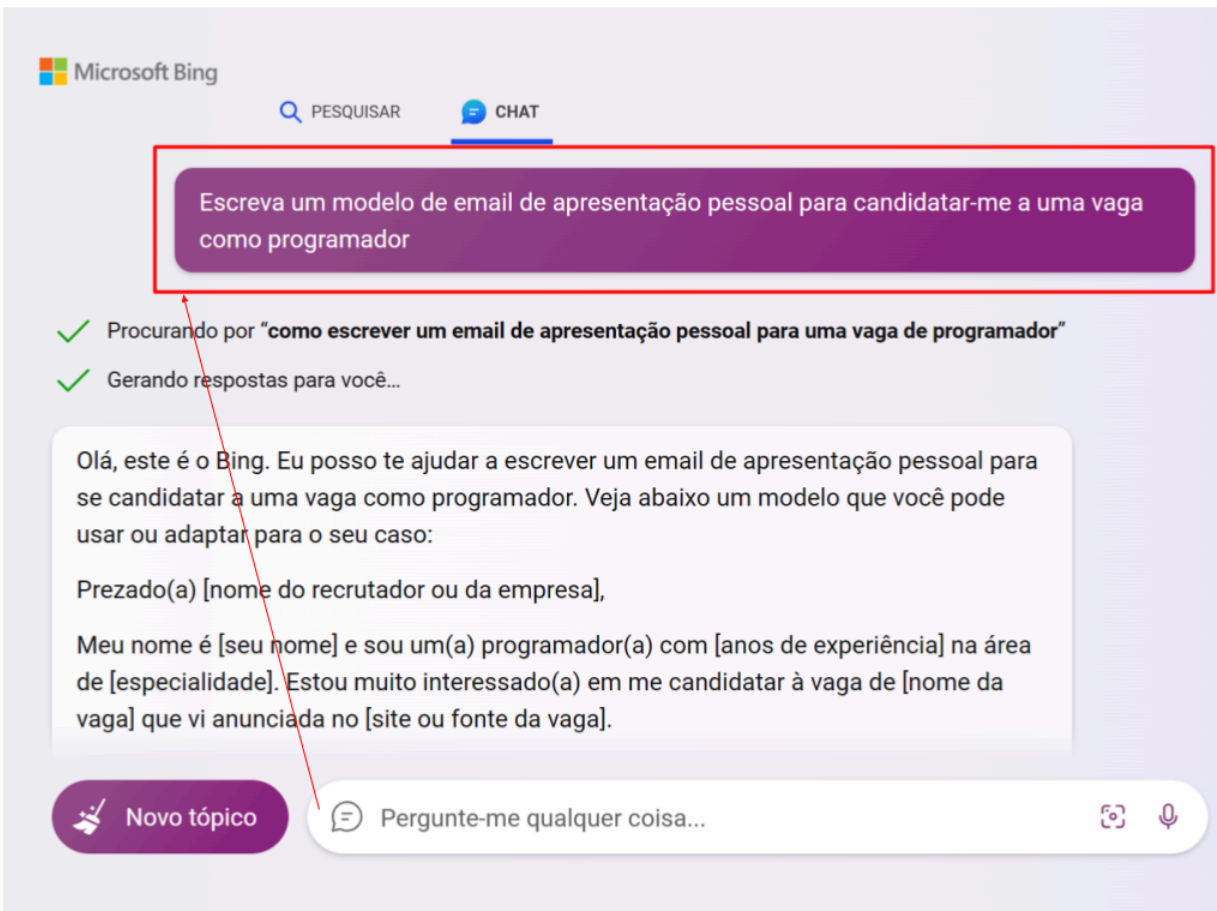


Figura 2.2: Exemplo de um prompt do buscador Bing, da Microsoft

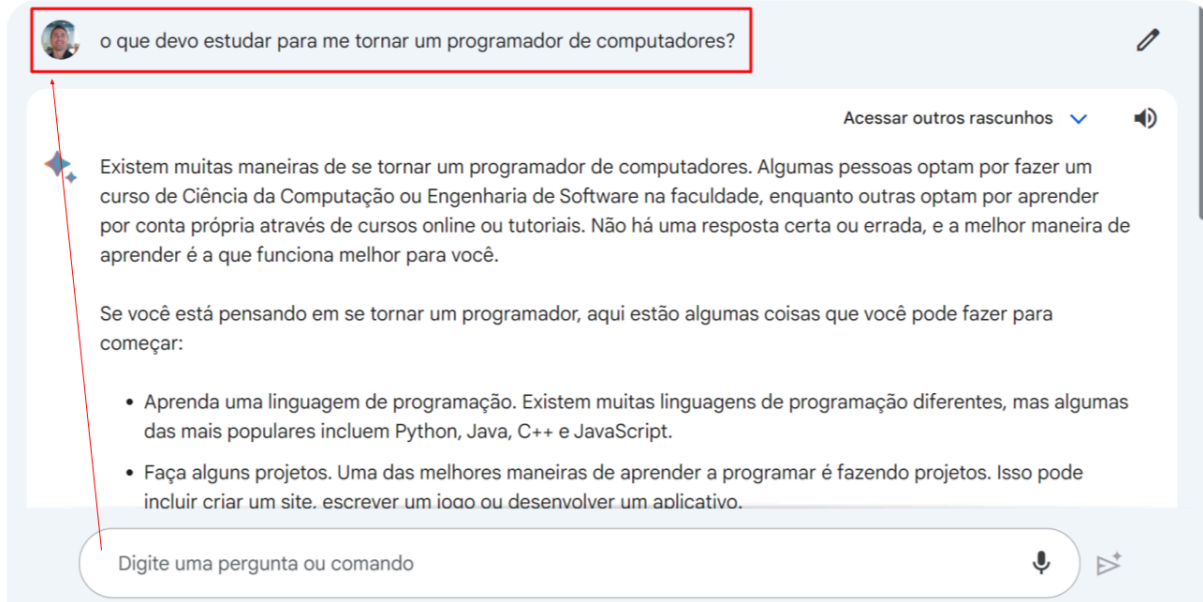


Figura 2.3: Exemplo de um prompt do Bard, do Google

```
1 #!/usr/bin/env ts-node
2
3 import { fetch } from "fetch-h2";
4
5 // Determine whether the sentiment of text is positive
6 // Use a web service
7 async function isPositive(text: string): Promise<boolean> {
8   const response = await fetch(`http://text-processing.com/api/sentiment/`, {
9     method: "POST",
10    body: `text=${text}`,
11    headers: {
12      "Content-Type": "application/x-www-form-urlencoded",
13    },
14  });
15  const json = await response.json();
16  return json.label === "pos";
17 }
```

Figura 2.4: Exemplo de um prompt do GitHub Copilot

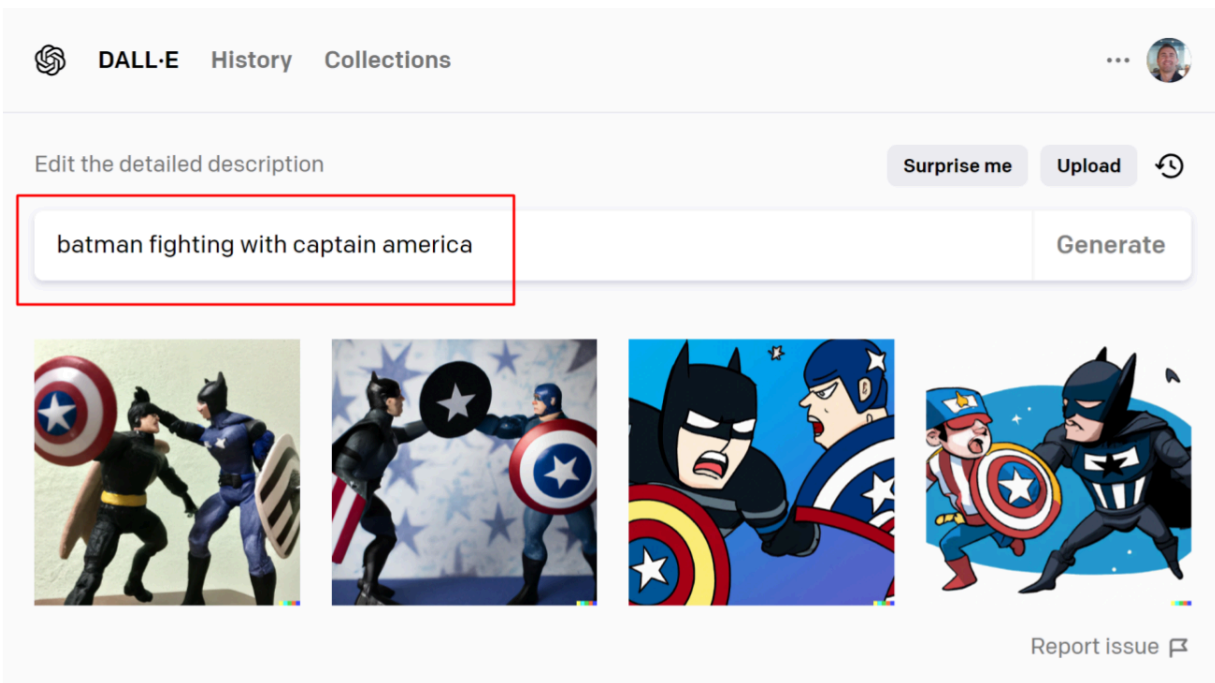


Figura 2.5: Exemplo de um prompt do DALL-E

Analisando os exemplos de prompt apresentados anteriormente, percebemos a sutileza e complexidade que envolvem os prompts em IA, quando comparados a sistemas convencionais. Nota-se que o uso do prompt na IA é conceitualmente mais abstrato. Aqui, ele não é simplesmente um comando direto inserido por um usuário, mas um elemento de comunicação complexo, multifacetado e dinâmico, projetado para orientar a IA em sua interação com o usuário.

Com a maior complexidade dessa interação humano-máquina na IA, os prompts precisam ser projetados e otimizados de maneira mais sofisticada para garantir uma comunicação eficaz. Esse desafio remete ao campo do Processamento de Linguagem Natural (PLN), uma subárea da Inteligência Artificial que se concentra na capacidade dos computadores de compreender e responder à linguagem humana de maneira significativa e contextualmente relevante.

Essa reflexão nos conduz diretamente ao campo da Engenharia de Prompt. Nesse domínio, a atenção é voltada para a criação

cuidadosa de prompts que guiam sistemas de IA para produzir respostas não apenas corretas, mas também contextuais, coerentes e úteis para o usuário. A engenharia de prompt, então, pode ser vista como um processo que envolve a elaboração, otimização e ajuste de instruções, perguntas e solicitações — os "prompts" — com o intuito de extrair respostas e ações específicas de sistemas embasados em Inteligência Artificial (IA) e Processamento de Linguagem Natural (PLN). Tal processo é fundamental para aprimorar a comunicação entre humanos e sistemas baseados em IA, assegurando que as respostas e ações geradas pelos sistemas sejam úteis, pertinentes e precisas.

No contexto da engenharia de prompt, o foco principal é entender e satisfazer as necessidades do usuário. Esse objetivo requer uma análise cautelosa das metas e expectativas dos usuários, além de um profundo entendimento das capacidades e limitações dos modelos de linguagem e sistemas de IA em uso.

Na criação de um prompt eficaz para IA, formula-se instruções que direcionem precisamente o sistema para a geração da resposta ou ação desejada. Esse resultado é geralmente obtido por meio de diretrizes claras e precisas, complementadas por exemplos e contextos adequados. Além disso, é essencial eliminar quaisquer ambiguidades nas instruções, uma vez que elas podem levar a respostas imprecisas ou errôneas.

Ao longo das próximas seções, exploraremos a importância da engenharia de prompt na IA e PLN, discutindo algumas aplicações comuns nas quais a engenharia de prompt desempenha um papel crítico para o sucesso de um projeto.

2.2 A importância da engenharia de prompt na IA e PLN

A engenharia de prompt é um componente essencial na interação entre humanos e sistemas baseados em Inteligência Artificial (IA) e Processamento de Linguagem Natural (PLN). À medida que os modelos de linguagem e as aplicações de IA se tornam mais sofisticados e prevalentes, a capacidade de criar prompts eficazes e precisos é cada vez mais importante para garantir a utilidade e relevância desses sistemas em diversos contextos.

Neste tópico, apresentamos algumas razões pelas quais a engenharia de prompt é de vital importância na IA e PLN.

Comunicação eficiente

"A finalidade da comunicação é fazer-se entender. Mas há quem prefira se desentender" — Augusto Branco

A engenharia de prompt, quando bem executada, se assemelha a uma coreografia cuidadosamente planejada, onde a criação de prompts claros e concisos conduz a dança entre humanos e Inteligência Artificial (IA). Assim como um coreógrafo habilidoso orienta os dançarinos a se moverem no ritmo e na direção corretos, a engenharia de prompt direciona a IA para produzir respostas pertinentes e úteis.

Essa comunicação eficiente, orientada por prompts precisos, serve como um intérprete em uma conversa entre duas pessoas que falam idiomas diferentes. A IA, que por natureza não possui compreensão humana ou intuição, se baseia nesses prompts para interpretar as necessidades dos usuários e responder de maneira adequada. Da mesma forma que um intérprete eficaz traduz não apenas palavras, mas o tom, a intenção e o contexto, um bom prompt ajuda a IA a "entender" o que é esperado dela.

Por exemplo, imagine um sistema de IA usado para agendamentos médicos. Um prompt eficaz poderia ser: "Por favor, agende uma consulta com o Dr. Silva para a próxima terça-feira". Isso fornece à

IA todas as informações de que ela precisa: o tipo de ação (agendar uma consulta), com quem (Dr. Silva), e quando (próxima terça-feira). O sistema pode então fornecer uma resposta útil, como confirmar o agendamento ou indicar que o Dr. Silva não tem horários disponíveis na terça-feira.

Assim, a engenharia de prompt melhora a comunicação entre os usuários e os sistemas. Isso garante que as informações e ações geradas sejam não apenas tecnicamente corretas, mas também úteis e relevantes para o usuário, atendendo e, em muitos casos, superando suas expectativas e necessidades. A finalidade é que cada interação deixe o usuário com a sensação de que sua solicitação foi compreendida e adequadamente atendida, assim como uma dança bem coreografada deixa a plateia com uma sensação de satisfação e apreciação.

Redução de ambiguidade

Na comunicação entre duas pessoas, a ambiguidade geralmente resulta em confusão e mal-entendidos. O mesmo princípio se aplica na interação entre humanos e sistemas de IA. Instruções claras e precisas são necessárias para uma comunicação eficaz, caso contrário, a IA pode interpretar erroneamente a solicitação do usuário e fornecer respostas indesejáveis.

Por exemplo, se um usuário pede à IA *"um livro sobre estrelas"*, a solicitação é ambígua. A IA pode interpretar "estrelas" como referência a celebridades, estrelas do mar ou corpos celestes. No entanto, se o usuário der um prompt mais específico, como *"Por favor, encontre um livro sobre a formação de estrelas no universo"*, a IA terá uma compreensão clara do que o usuário está procurando, eliminando a ambiguidade e levando a uma resposta mais precisa.

Em ambientes de alto risco, como hospitais, a precisão do prompt é ainda mais crítica. Imagine um sistema de IA que administra medicamentos aos pacientes. Um prompt como *"Dê ao paciente o medicamento"* é muito vago e pode levar a erros. No entanto, um

prompt mais preciso, como *"Administre 50mg de medicamento X ao paciente Y às 8h da manhã"*, fornece instruções claras e diretas, reduzindo a probabilidade de erros.

Outra estratégia importante para reduzir a ambiguidade na engenharia de prompt, como veremos adiante em uma abordagem mais prática, é através da criação de contexto por meio de interações conversacionais prévias. Os sistemas de IA modernos são capazes de manter o estado da conversação, levando em conta as entradas anteriores para entender melhor as solicitações subsequentes. Isso se torna especialmente útil em situações onde a instrução final por si só pode não fornecer detalhes suficientes.

Por exemplo, em uma sequência de interações onde um usuário primeiro menciona *"Estou fazendo um trabalho de pesquisa sobre corpos celestes"*, e depois pede *"Agora me mostre um livro sobre estrelas"*, a IA pode usar o contexto fornecido na primeira interação para deduzir que "estrelas" se refere a corpos celestes, e não a celebridades ou estrelas do mar. Essa técnica de fornecer contexto por meio de interações anteriores ajuda a criar prompts mais eficazes e a orientar a IA para uma resposta mais precisa e relevante.

Personalização e adaptação

Na engenharia de prompt, a personalização e adaptação otimizam a interação entre os usuários e os sistemas de IA. A eficácia de um prompt depende de sua relevância para o contexto e as necessidades do usuário, de forma semelhante à importância da especificidade na comunicação entre duas pessoas. Isso significa que a engenharia de prompt precisa ser sensível ao contexto de uso da IA e adaptável a uma ampla gama de cenários.

Vamos considerar o exemplo de um assistente virtual de IA que opera em diferentes ambientes de trabalho. Em uma empresa de design de interiores, o assistente pode ser chamado para auxiliar na escolha de paletas de cores ou na visualização de diferentes layouts

de sala. Nesse contexto, prompts eficazes poderiam envolver perguntas claras sobre o tipo de espaço a ser projetado, o estilo desejado e as preferências de cor do usuário.

Por outro lado, se o mesmo assistente virtual fosse implementado em uma fábrica de manufatura, os prompts teriam que ser ajustados para se adequar a esse contexto muito diferente. As interações poderiam envolver consultas sobre a manutenção de máquinas, acompanhamento de ordens de produção ou gerenciamento de inventário.

Esses exemplos ilustram a importância da personalização na engenharia de prompt. Ajustar os prompts para que se alinhem às necessidades e circunstâncias específicas do usuário pode aumentar significativamente a utilidade e eficácia de um sistema de IA. Isso garante que o sistema possa atender a uma variedade de demandas, operando como um assistente pessoal adaptável que pode ser personalizado para atender a necessidades individuais específicas. Portanto, a engenharia de prompt não só aprimora a precisão das respostas da IA, mas também contribui para a flexibilidade e personalização desses sistemas.

A capacidade de personalização e adaptação tem um impacto direto na utilidade dos sistemas de IA. Ao ajustar a interação com base no contexto e nas necessidades do usuário, a engenharia de prompt permite que a IA forneça um serviço mais personalizado e relevante. Esse processo pode ser comparado a criar um guia personalizado para cada usuário, que leva em conta suas necessidades específicas. Na mesma linha do exemplo anterior, ao desenvolver um prompt, uma pessoa engenheira pode considerar o domínio de aplicação (como design de interiores ou manufatura), o nível de conhecimento do usuário (especialista ou novato), as preferências do usuário (como brevidade versus detalhamento), e muitos outros fatores. Esse cuidado na criação de prompts resulta em interações mais eficazes e satisfatórias com o sistema de IA.

Melhoria contínua

A próxima camada de valor na engenharia de prompt que vamos desvendar é a **melhoria contínua**. Esse processo não é um destino final, mas uma viagem constante de teste, avaliação e ajuste dos prompts — semelhante a um escultor refinando continuamente sua obra de arte até alcançar a visão desejada.

Em um mundo perfeito, nossos sistemas de IA entenderiam todas as nuances do linguajar humano logo de cara. No entanto, a realidade é que a nossa linguagem é complexa, cheia de sutilezas, ambiguidades e regionalismos. Isso exige um processo de aperfeiçoamento e afinação, semelhante a sintonizar um instrumento musical para produzir o tom certo.

Nesse contexto, cada teste de um prompt é uma oportunidade para aprender mais sobre como os sistemas de IA respondem a diferentes estímulos. As respostas geradas pelos sistemas oferecem informações valiosas para identificar problemas e oportunidades de melhoria. Por exemplo, um prompt pode resultar em respostas longas demais para serem práticas ou muito curtas para serem úteis. Identificar essas questões permite aos usuários refinar os prompts, buscando o ponto ideal para maximizar a eficácia da comunicação.

Da mesma forma que um corredor de maratona continua a treinar e a ajustar sua técnica para melhorar seu tempo e eficiência, a engenharia de prompt é um exercício de aprimoramento contínuo. A ideia aqui é buscar a excelência iterativamente, sempre com o objetivo de melhorar a qualidade e a eficácia dos sistemas de IA e PLN, tornando-os cada vez mais capazes de entender e responder de maneira eficaz e precisa às demandas dos usuários.

Convergência entre humanos e IA

Por fim, um aspecto a destacar é a convergência entre humanos e IA. A engenharia de prompt busca integrar a interação entre esses dois agentes, assegurando uma coordenação eficaz entre todas as partes do sistema de IA para maximizar a satisfação do usuário.

O componente crítico dessa integração é a criação de prompts bem estruturados que orientem os sistemas de IA de maneira eficaz, assegurando uma interação produtiva entre humanos e IA. Por exemplo, ao usar um sistema de navegação por GPS, se as instruções dadas ao sistema forem claras e precisas, o usuário chegará ao destino sem problemas. No entanto, instruções vagas ou confusas podem levar a desvios ou perda de tempo. Da mesma forma, a engenharia de prompt tem como objetivo assegurar que os prompts fornecidos aos sistemas de IA sejam claros, concisos e úteis, facilitando a obtenção do resultado desejado.

Prompts bem elaborados aumentam a confiança dos usuários na capacidade da IA de compreender suas necessidades e fornecer respostas relevantes e úteis. Isso é evidente no uso de assistentes virtuais, que podem realizar tarefas como marcar compromissos, responder a perguntas complexas ou até mesmo contar uma piada para iluminar o dia do usuário. Com prompts eficazes, a interação com o assistente virtual pode se tornar tão intuitiva que a IA é percebida não apenas como uma ferramenta, mas como um verdadeiro assistente confiável para o dia a dia.

Portanto, a engenharia de prompt é essencial para criar sistemas de IA que não só executem tarefas eficientemente, mas que também proporcionem experiências de usuário gratificantes. Isso aproxima ainda mais os humanos e a IA, criando uma sinergia produtiva e satisfatória.

Em resumo, a engenharia de prompt é uma habilidade inestimável para os profissionais que trabalham com IA e PLN. Dominar essa técnica é como ter um guia confiável para a comunicação eficiente e precisa entre humanos e sistemas de IA, o que leva a soluções mais relevantes e úteis para os usuários finais. No próximo tópico, vamos explorar algumas aplicações comuns da engenharia de prompt e como essa prática pode ser aplicada em uma variedade de contextos.

2.3 Aplicações da engenharia de prompt

A engenharia de prompt desempenha um papel importante em uma ampla variedade de aplicações e setores. Dominar a arte de criar prompts eficazes é essencial para analistas e desenvolvedores de sistemas que desejam aproveitar ao máximo as capacidades dos sistemas baseados em IA e PLN na otimização do seu trabalho. Nesta seção, apresentaremos algumas aplicações comuns onde a engenharia de prompt é fundamental para o sucesso.

Assistência virtual e bots de conversação

A importância da engenharia de prompt em assistentes virtuais ou *bots* de conversação se dá na formação de interações fluidas e significativas para os usuários. Ao criar um diálogo natural e coerente, essas tecnologias se tornam mais do que simples ferramentas, emergindo como interlocutores confiáveis e capazes de fornecer informações precisas e pertinentes. Porém, é preciso contextualizar o assistente ou bot com informações sobre a empresa, produtos oferecidos, serviços prestados, detalhes de operação como dias e horários de atendimento, procedimentos etc. Também é preciso contextualizar a conversa da IA com o usuário em relação ao seu histórico de pedido e interações.

Como o treinamento de um modelo de linguagem é sazonal, ele não terá informações recentes. E como o usuário comum não tem conhecimento sobre engenharia de prompt, ele não saberá como escrever o prompt apropriadamente. A engenharia de prompt, neste caso, age nos bastidores. Ela intermedeia a comunicação para incluir contexto na conversa.

Por exemplo, se um usuário está interagindo com um bot de atendimento ao cliente de uma empresa de telefonia e menciona um problema com a conexão de internet, a engenharia de prompt pode contextualizar o bot com respostas a questões frequentes, procedimentos internos específicos etc. Isso fará o bot reconhecer o

contexto da reclamação e direcionar a conversa para soluções possíveis, como reiniciar o roteador ou verificar a velocidade da conexão.

A personalização também é vital para tornar a interação mais relevante e satisfatória para o usuário. Ao lidar com um cliente, o bot pode ser alimentado com conversas passadas e pedidos abertos, o que permite ao bot adaptar suas respostas de acordo com as preferências ou o comportamento do usuário.

Outro aspecto importante é a restrição de informações que não sejam relevantes. A engenharia de prompt pode instruir previamente o bot a não responder determinados assuntos que não sejam relevantes para o negócio. Imagine um cliente de uma empresa de telefonia entrando em contato com o bot para fazer perguntas sobre saúde ou nutrição, demandando recursos do sistema sem atender seu propósito principal. A engenharia de prompt pode evitar isso.

Somado a isso, a engenharia de prompt pode ser usada para melhorar a interação ao longo do tempo por meio de aprendizado contínuo. Por exemplo, se um assistente virtual perceber que um usuário frequentemente pede o horário do nascer do sol, a IA, com a ajuda da engenharia de prompt, poderia começar a fornecer essa informação de forma proativa, mostrando sua capacidade de aprendizado e adaptação.

Portanto, em aplicações de assistência virtual e bots de conversação, a engenharia de prompt pode ser aplicada para orientar a IA a interagir de maneira eficaz e significativa com os usuários, proporcionando experiências mais satisfatórias e personalizadas.

Análise de sentimentos e classificação de texto

Cada texto, como uma avaliação de produto, um comentário nas redes sociais ou um trecho de notícia, possui aspectos detalhados e únicos. Esses detalhes podem incluir sentimentos, tópicos ou categorias específicas que, se identificados e extraídos

eficientemente, fornecem uma compreensão aprofundada do contexto e das intenções subjacentes.

Por exemplo, considere a administração de um serviço de *streaming* de filmes e séries que recebe milhares de avaliações e comentários dos usuários todos os dias. Analisar manualmente esses dados seria uma tarefa extremamente trabalhosa. A engenharia de prompt pode equipar a IA para processar esse grande volume de dados e identificar sentimentos, avaliações positivas ou negativas e até sugestões úteis presentes nos comentários. Com base nessas informações, o serviço de streaming pode aprimorar continuamente seus serviços para atender às preferências e necessidades de seus usuários.

Em uma situação diferente, suponha que você esteja gerenciando uma grande editora de notícias online. Seu site possui uma vasta quantidade de artigos cobrindo uma variedade de tópicos. A engenharia de prompt pode capacitar a IA para classificar esses artigos com base em tópicos ou categorias, facilitando para os leitores encontrar o conteúdo de seu interesse. Esse tipo de classificação não apenas torna a navegação mais conveniente, mas também ajuda a aumentar a retenção de usuários e a satisfação geral.

Geração automática de texto e resumos

A engenharia de prompt na geração automática de textos e resumos é uma aplicação prática que pode ser útil no processo de análise e desenvolvimento de sistemas. Vamos explorar como essa técnica pode ser empregada em cenários específicos relacionados ao campo de desenvolvimento de software e sistemas.

Por exemplo, na elaboração de documentação técnica, a engenharia de prompt pode ser usada para instruir um sistema de IA a produzir descrições detalhadas de módulos, classes ou funções. Essas

instruções podem especificar o tema e o formato desejados, permitindo que o sistema gere documentação consistente e de acordo com as diretrizes do projeto. Isso pode acelerar o processo de documentação, liberando os analistas de sistemas para se concentrarem em tarefas mais críticas.

No desenvolvimento de interfaces de usuário, a engenharia de prompt pode ser aplicada para definir literais para *labels* de sistemas, como botões e menus. Um sistema de IA pode ser instruído a gerar rótulos e descrições que sejam consistentes com as diretrizes de estilo e linguagem do projeto. Por exemplo, um desenvolvedor pode precisar de *labels* em diferentes idiomas para internacionalizar uma aplicação; a engenharia de prompt pode guiar a IA para produzir esses textos de acordo com as regras gramaticais e culturais de cada idioma.

A geração de textos para protótipos ou *landing pages* é outra área em que essa técnica pode ser aplicada. Suponha que uma equipe de desenvolvimento esteja trabalhando em um protótipo de um novo aplicativo. A engenharia de prompt pode ser usada para criar descrições convincentes e informativas das funcionalidades do aplicativo. Essas descrições podem ser incorporadas diretamente no protótipo, ajudando a comunicar a visão e o propósito do aplicativo para *stakeholders* e usuários potenciais.

Processamento de Linguagem Natural em análise de dados

Na análise de dados, a extração de informações relevantes de grandes conjuntos de dados é uma tarefa complexa e demorada. Com a ajuda da engenharia de prompt, os analistas de sistemas podem criar instruções claras que guiam um sistema de IA na extração e análise de informações específicas. Isso pode ser especialmente útil na identificação de padrões, tendências, correlações e anomalias.

Por exemplo, se uma equipe está trabalhando em um sistema de comércio eletrônico e quer entender o comportamento do cliente, a

engenharia de prompt pode ser usada para guiar a IA na análise de dados de compra, navegando por padrões de compra e tendências sazonais. Essas informações podem então ser usadas para ajustar o design do sistema, como recomendar produtos ou otimizar a interface do usuário.

Além disso, a geração de visualizações e relatórios é uma tarefa comum na análise de dados que pode ser aprimorada através da engenharia de prompt. Instruções podem ser criadas para guiar a IA na geração de gráficos, tabelas e outras representações visuais que tornam os dados mais acessíveis e compreensíveis. Por exemplo, ao analisar o desempenho de um aplicativo, um analista pode utilizar a engenharia de prompt para criar um relatório visual mostrando a quantidade de usuários ativos, a duração das sessões, as páginas mais visitadas etc., facilitando a interpretação dos dados.

A engenharia de prompt também pode ser usada para destacar anomalias nos dados que podem indicar potenciais problemas. Suponha que um sistema de monitoramento esteja em vigor para rastrear o desempenho de um aplicativo. A engenharia de prompt pode ser empregada para instruir a IA a identificar e alertar sobre picos de uso incomuns ou falhas recorrentes, ajudando a equipe de desenvolvimento a agir proativamente.

Tradução automática

Na era da globalização, a necessidade de traduzir interfaces, documentação ou conteúdo de um sistema para vários idiomas é comum. Imagine uma equipe de desenvolvimento trabalhando em um software que será lançado internacionalmente. O conteúdo terá que ser traduzido para diversos idiomas, mantendo a precisão e considerando as diferenças culturais. A engenharia de prompt pode ser usada para orientar a IA na tradução, garantindo que as nuances linguísticas e culturais sejam levadas em conta.

Por exemplo, ao traduzir os rótulos de botões, menus e mensagens de erro dentro de um sistema, simplesmente traduzir palavra por

palavra pode não ser suficiente. Expressões e termos técnicos podem ter diferentes significados em diferentes culturas. A engenharia de prompt permite que instruções detalhadas sejam dadas à IA, incluindo o contexto em que a tradução será usada, o público-alvo e quaisquer especificações culturais relevantes. Isso ajuda a garantir que a tradução seja não apenas precisa, mas também apropriada para o contexto.

Outro cenário em que a engenharia de prompt pode ser útil é na tradução de documentação técnica. Documentos como manuais de usuário, guias de instalação e especificações de produto precisam ser cuidadosamente traduzidos para garantir que o significado original seja mantido. A engenharia de prompt pode fornecer diretrizes claras para a IA, enfocando a terminologia técnica e o estilo de escrita desejado.

A tradução automática também pode ser aplicada em tempo real em sistemas de suporte ao cliente, onde perguntas e respostas precisam ser traduzidas entre diferentes idiomas. A engenharia de prompt pode ser usada para fornecer informações contextuais, ajudando a IA a compreender e traduzir de forma mais eficiente as consultas dos clientes.

Moderação e filtragem de conteúdo

A moderação e filtragem de conteúdo são particularmente importantes para manter um ambiente seguro e construtivo. E sistemas de IA podem realizar essas tarefas de forma mais eficaz.

Em primeiro lugar, considere um sistema de comentários em um site de uma empresa. Os usuários podem deixar feedback, perguntas ou comentários, mas é essencial que o conteúdo impróprio seja identificado e removido. A engenharia de prompt pode ser utilizada para fornecer à IA diretrizes claras sobre o que constitui "conteúdo inadequado", como discurso de ódio, spam ou informações falsas. Por exemplo, a IA pode ser instruída a buscar por certas palavras-

chave, padrões de linguagem ou links suspeitos, permitindo a filtragem e a moderação automatizadas.

Além disso, a engenharia de prompt pode ser aplicada na personalização do processo de moderação, de acordo com as políticas e normas específicas de uma organização. Suponha que uma empresa de software queira moderar os fóruns da comunidade, onde os desenvolvedores discutem questões técnicas. A engenharia de prompt pode ser usada para instruir a IA sobre as regras da comunidade, garantindo que a moderação seja alinhada com a cultura e os objetivos da empresa.

A filtragem de conteúdo também pode ser aplicada no contexto de e-mails e mensagens. Uma empresa que recebe uma grande quantidade de comunicações pode usar a IA para filtrar mensagens irrelevantes ou maliciosas. A engenharia de prompt pode fornecer instruções detalhadas sobre como identificar essas mensagens, tornando o processo mais eficiente e permitindo que a equipe se concentre nas comunicações importantes.

Em alguns casos, a moderação pode requerer uma compreensão mais profunda do contexto — por exemplo, no gerenciamento de mídias sociais, onde a interpretação de sarcasmo ou humor pode ser complexa. A engenharia de prompt pode ser utilizada para fornecer orientações adicionais, ajudando a IA a compreender as nuances e a tomar decisões de moderação mais informadas.

Essas são apenas algumas das muitas aplicações em que a engenharia de prompt desempenha um papel crucial. À medida que a IA e o PLN continuam a evoluir e a se expandir para novos domínios e setores, isso só aumentará, tornando-se uma habilidade essencial para profissionais que desejam aproveitar o poder dessas tecnologias emergentes.

2.4 Aplicações da engenharia de prompt na análise e no desenvolvimento de sistemas

A capacidade de aplicar a Inteligência Artificial no suporte às nossas tarefas é limitada apenas pela nossa criatividade. Alguns exemplos teóricos que ilustram como a IA pode potencializar a eficiência do processo de desenvolvimento de sistemas incluem:

- **Geração automática de código:** com o uso de IA e modelos de linguagem avançados, pessoas desenvolvedoras podem criar prompts específicos para gerar trechos de código com base em requisitos fornecidos. Isso pode economizar tempo e esforço no desenvolvimento de sistemas, permitindo que se concentre em tarefas mais complexas e criativas;
- **Deteção e correção de bugs:** a IA pode ser usada para analisar o código-fonte e identificar possíveis erros ou falhas de segurança. Isso pode melhorar a qualidade do código e reduzir o tempo gasto na depuração e correção de problemas, resultando em sistemas mais confiáveis e seguros;
- **Otimização de desempenho:** a IA pode analisar o desempenho de um sistema e sugerir melhorias no código ou na arquitetura do sistema. Isso pode levar a sistemas mais rápidos e eficientes, melhorando a experiência do usuário e reduzindo os custos de infraestrutura;
- **Análise de requisitos e priorização de recursos:** a IA pode ser usada para analisar e entender os requisitos dos usuários, ajudando desenvolvedores a priorizar os recursos que terão maior impacto. Isso pode resultar em sistemas mais eficientes e focados nas necessidades reais dos usuários;
- **Assistência no desenvolvimento de interfaces de usuário:** a IA pode ser usada para gerar protótipos de interfaces de usuário com base nos requisitos e preferências dos usuários. Isso pode ajudar desenvolvedores a criar interfaces

mais intuitivas e agradáveis, aumentando a satisfação do usuário e a adoção do sistema;

- **Teste automatizado e validação:** a IA pode ser empregada para realizar testes automatizados e validar o funcionamento correto dos sistemas. Isso pode reduzir o tempo e o esforço necessários para garantir a qualidade e a confiabilidade do sistema;
- **Aprendizado e adaptação contínua:** a IA pode ser usada para analisar o comportamento do usuário e adaptar os sistemas com base nas necessidades e preferências dos usuários. Isso pode levar a sistemas mais personalizados e eficientes, melhorando a experiência do usuário e aumentando a eficiência geral do processo de desenvolvimento.

2.5 A profissão de Engenharia de Prompt

A profissão de Engenharia de Prompt é uma especialização emergente na interseção entre a Linguística, a Ciência da Computação e a Inteligência Artificial. Essa função envolve a criação de prompts ou instruções claras que guiam os modelos de IA em tarefas específicas, como análise de sentimentos, tradução automática, moderação de conteúdo, entre outras. A pessoa engenheira de prompt deve compreender profundamente tanto as necessidades do usuário quanto a maneira como os modelos de IA interpretam e respondem aos prompts, equilibrando fatores técnicos e humanos para obter os resultados desejados.

Dentro da Engenharia de Prompt, existem várias variantes e especializações, dependendo da aplicação e da indústria. Por exemplo, na área de Saúde, pode haver uma necessidade de prompts altamente especializados para interpretar dados médicos e literatura, requerendo uma compreensão tanto da linguagem médica quanto das normas éticas e regulatórias. No campo do Marketing, a

engenharia de prompt pode ser usada para gerar conteúdo criativo e persuasivo, exigindo habilidades em redação e estratégia de comunicação. Na Computação não é diferente, e a engenharia de prompt exige habilidades específicas da área.

A educação e formação para a Engenharia de Prompt podem ser diversas, refletindo a natureza interdisciplinar da profissão. Um(a) engenheiro(a) de prompt pode ter formação em Ciência da Computação, Linguística, Design de Interação ou campos relacionados. O conhecimento profundo em Processamento de Linguagem Natural (PLN) e a capacidade de trabalhar com modelos de aprendizado de máquina são frequentemente importantes, mas a sensibilidade à linguagem e ao contexto humano também é. Cursos, workshops e certificações específicos em Engenharia de Prompt estão começando a surgir para fornecer formação focada nessa área.

A carreira em Engenharia de Prompt oferece oportunidades em diversas indústrias, desde tecnologia e saúde até finanças e entretenimento. À medida que a IA continua a avançar e encontrar novas aplicações, a demanda por profissionais que possam traduzir necessidades complexas em instruções claras e eficazes para modelos de IA deve crescer. Mas acredito que a tendência é que seja uma formação complementar, não específica. Então, profissionais de todas as áreas em que a IA possa ser aplicada precisarão desenvolver essa nova habilidade.

CAPÍTULO 3

Elaboração de prompts

"A inquietude é seu futuro implorando aos seus pés por instruções" — Mike Murdock

Na prática, um prompt é uma caixa de texto onde um usuário faz uma pergunta ou solicitação. Mas há um conjunto de componentes implícitos, que são elementos que formam a estrutura e o conteúdo de uma solicitação feita a um modelo de linguagem. Eles trabalham em conjunto para transmitir claramente a questão ou tarefa, incluindo todas as informações, instruções e detalhes necessários para obter uma resposta relevante e precisa. Esses componentes implícitos são formados pela **instrução** (ou pergunta), pela **resposta** (ou ação esperada) e pelo **contexto**.

3.1 A instrução (ou pergunta)

Quando interagimos com modelos de linguagem avançados, a interface mais comum é uma simples caixa de texto. É aqui que o usuário informa ao modelo sua solicitação ou pergunta, buscando gerar uma saída pertinente. A riqueza da instrução influencia diretamente a qualidade e relevância da resposta. Há diversas abordagens que podemos adotar ao estruturar essa instrução, cujo limite é a criatividade do usuário.

Abordagem direta

Uma das abordagens que um usuário pode adotar é a direta, em que o modelo de linguagem responde de forma livre e sem nenhum contexto. Por um lado, o usuário pode obter uma resposta mais

criativa com essa abordagem. Porém, muitas vezes, as respostas sem contexto podem não atender ao propósito, principalmente se a instrução contiver redundâncias ou ambiguidades. É importante entender que, por padrão, um modelo de linguagem não questiona; então, ele não pedirá esclarecimentos da instrução, limitando-se apenas a responder conforme sua base de aprendizado.

Podemos conferir a seguir um exemplo de uma instrução com abordagem direta, bem como a resposta do modelo de linguagem:

Você: *Quais são os princípios básicos da Programação Orientada a Objetos?*

IA: A Programação Orientada a Objetos (POO) é um paradigma de programação baseado no conceito de "objetos", que podem conter dados e código para manipular esses dados. Este paradigma é fundamental em muitas linguagens de programação modernas, como Java, C++, Python, Ruby, entre outras.

Aqui estão os princípios básicos da Programação Orientada a Objetos:

1. **ENCAPSULAMENTO:** refere-se à restrição do acesso direto a alguns dos componentes do objeto e à prevenção de modificações não intencionais. Em linguagens de programação que suportam POO, isso é realizado usando modificadores de acesso, como `private`, `protected` e `public`.
2. **ABSTRAÇÃO:** é a ideia de esconder detalhes complexos e mostrar apenas o necessário. Em POO, as classes fornecem uma abstração dos dados e dos métodos que operam nesses dados. Por exemplo, um objeto "Carro" pode ter métodos como "ligar" e "acelerar" sem precisar saber como essas ações são implementadas internamente.

(...)

Abordagem contextualizada

Por outro lado, um usuário pode enriquecer o prompt com *contextos explícitos* que produzirão respostas mais aprimoradas. Por exemplo, em vez de apenas pedir "*Traduza o seguinte*", um usuário pode especificar "*Traduza o seguinte texto do português para o inglês*". Dependendo da natureza da tarefa e da informação que o usuário deseja extrair, diferentes abordagens podem ser usadas para contextualizar a instrução. O usuário pode criar, por exemplo, um cenário que fará com que o modelo de linguagem forneça uma resposta aderente ao contexto requisitado.

No exemplo seguinte, podemos observar como uma simples frase adicional na mesma instrução (no caso, "*Sou um programador novato tentando entender OOP.*") produz uma resposta mais detalhada e com linguagem simples, mais aderente à instrução agora contextualizada.

Você: *Sou um programador novato tentando entender OOP. Pode me explicar os princípios básicos?*

IA: Claro! A Programação Orientada a Objetos (OOP) é uma abordagem de programação que se baseia no conceito de "objetos". Estes objetos podem conter tanto dados, na forma de campos (muitas vezes conhecidos como atributos ou propriedades), quanto código, na forma de procedimentos (frequentemente conhecidos como métodos). A OOP é baseada em quatro princípios fundamentais:

1. **ENCAPSULAMENTO:**

- Encapsulamento refere-se à restrição de acesso direto a alguns dos componentes do objeto e à proteção dos mesmos.
- Em termos simples, isso significa que os detalhes internos do funcionamento de um objeto estão ocultos do mundo exterior.
- Isto é alcançado usando modificadores de acesso como `private`, `protected` e `public` (em linguagens como Java e C++).
- Isso permite que os detalhes internos do objeto sejam alterados sem afetar as partes do programa que usam o objeto.

2. **ABSTRAÇÃO:**

- A abstração permite criar um modelo simplificado de um objeto mais complexo.
- Permite focar nas características de alto nível de um objeto, ignorando os detalhes desnecessários.
- Na OOP, classes fornecem a abstração, e você pode criar uma instância dessa classe, o que é chamado de objeto.
- Por exemplo, se você tem uma classe `Carro`, a abstração permite a você entender que um carro tem métodos como `ligar()` e `acelerar()` sem precisar entender todos os detalhes de como isso é feito.

(...)

Repare como, neste último exemplo, o texto é mais completo e está em uma linguagem mais simples. E o contexto pode também pedir mais especificidade, como neste outro exemplo a seguir:

Você: *Me explique os princípios básicos de OOP exemplificados em javascript.*

IA: A Programação Orientada a Objetos (OOP - Object-Oriented Programming) é um paradigma de programação baseado no conceito de "objetos", que podem conter dados, na forma de campos (muitas vezes conhecidos como atributos ou propriedades) e código, na forma de procedimentos (frequentemente conhecidos como métodos). Quando aplicamos a OOP ao JavaScript, aproveitamos seus conceitos para estruturar e organizar o código de maneira mais modular e reutilizável.

Vamos ver os princípios fundamentais da OOP e como eles são aplicados ao JavaScript:

1. ENCAPSULAMENTO:

- Conceito: A ideia de agrupar os dados (atributos) e os códigos (métodos) que operam sobre os dados em uma única unidade, ou objeto.
- JavaScript: Podemos usar funções construtoras, classes (introduzidas no ES6) ou literais de objeto para encapsular dados e comportamento.

```
class Carro {  
  constructor(marca, modelo) {  
    this.marca = marca;  
    this.modelo = modelo;  
  }  
  
  exibirInfo() {  
    console.log(`Este é um ${this.marca}  
    ${this.modelo}`);  
  }  
}
```

2. ABSTRAÇÃO:

- Conceito: Isolar a complexidade de um objeto, expondo apenas o que é necessário.

- JavaScript: Usando classes e módulos, você pode esconder detalhes internos e mostrar apenas os métodos e propriedades relevantes.

(...)

No desenvolvimento web, o uso da OOP em JavaScript ajuda na criação de componentes reutilizáveis, manutenção mais fácil do código e uma estrutura mais organizada, especialmente em aplicativos maiores e mais complexos.

É possível observar que uma singela mudança na instrução produz um resultado totalmente diferente, com este último incluindo até exemplos práticos na linguagem mencionada. E, em outra abordagem, o usuário pode submeter mais informações antes ou após a instrução principal. Assim, você pode, por exemplo, pedir a tradução de um parágrafo, a revisão ortográfica de um texto ou mesmo uma síntese de um artigo que não seja muito extenso (falarei sobre os limites da instrução no tópico *O contexto conversacional*). No exemplo a seguir, podemos ver um pedido de revisão ortográfica:

Você: *Faça a revisão ortográfica do seguinte texto:*

É possível observar que uma sinjela mudança na instrução produz um resultado totalmente diferente este ultimo incluindo até inclusive mesmo exemplos práticos na linguagem mencionada. E, duma outra abordagem, o usuario pode submeter mais informações antes ou após as instrução principal

IA: Claro, aqui está a revisão do seu texto:

É possível observar que uma singela mudança na instrução produz um resultado totalmente diferente, este último incluindo até mesmo exemplos práticos na linguagem mencionada. E, de uma outra abordagem, o usuário pode submeter mais informações antes ou após a instrução principal.

As correções foram:

- "póssível" para "possível"
- "sinjela" para "singela"
- Retirado a repetição com "incluindo até inclusive mesmo" e substituído por "incluindo até mesmo"
- "duma" para "de uma"
- "usuario" para "usuário" (acrescentando o acento)

Agora, observe a seguir uma instrução que é explicitamente contextualizada com um texto público sobre mecânica quântica, extraído da Wikipédia (https://pt.wikipedia.org/wiki/Mec%C3%A2nica_qu%C3%A2ntica):

Você: *Considere o seguinte texto sobre mecânica quântica:*

"A mecânica quântica (também conhecida como física quântica, teoria quântica, modelo mecânico de ondas e mecânica de matriz) é a teoria física que obtém sucesso no estudo dos sistemas físicos cujas dimensões são próximas ou abaixo da escala atômica, tais como moléculas, átomos, elétrons, prótons e de outras partículas subatômicas, muito embora também possa descrever fenômenos macroscópicos em diversos casos.

A mecânica quântica é um ramo fundamental da física com vasta aplicação. A teoria quântica fornece descrições precisas para muitos fenômenos previamente inexplicados tais como a radiação de corpo negro e a estabilidade dos átomos. Apesar de, na maioria dos casos,

a mecânica quântica ser relevante para descrever sistemas microscópicos, os seus efeitos específicos não são somente perceptíveis em tal escala.

Por exemplo, a explicação de fenômenos macroscópicos como a super fluidez e a supercondutividade só é possível se considerarmos que o comportamento microscópico da matéria é quântico. A quantidade característica da teoria, que determina quando ela é necessária para a descrição de um fenômeno, é a chamada constante de Planck, que tem dimensão de momento angular ou, equivalentemente, de ação.

A mecânica quântica recebe esse nome por prever um fenômeno bastante conhecido dos físicos: a quantização. No caso dos estados ligados (por exemplo, um elétron orbitando em torno de um núcleo positivo) a Mecânica Quântica prevê que a energia (do elétron) deve ser quantizada. Este fenômeno é completamente alheio ao que prevê a teoria clássica."

...e faça uma síntese de um único parágrafo para que uma criança de 12 anos compreenda.

IA: A mecânica quântica é uma parte da física que estuda coisas muito, muito pequenas, como átomos e partículas menores ainda. Ela nos ajuda a entender por que certos materiais e objetos se comportam de maneiras especiais. Uma das coisas mais interessantes é que, em vez de ter qualquer quantidade de energia, como imaginamos normalmente, essas pequenas partículas só podem ter certas quantidades específicas de energia, como se fossem degraus em uma escada. Isso é diferente do que a física comum nos diz e é por isso que chamamos de "quântica".

Com esses exemplos, é possível perceber que o limite da contextualização de uma instrução é a própria criatividade do usuário.

Abordagem com anexos ou links contextuais

Alguns modelos de linguagem oferecem a possibilidade de incluir, como contexto da instrução, um link ou um anexo. Algumas IAs que geram imagens, por exemplo, permitem que seja mencionado um link público de uma imagem existente como referência na instrução. É o caso do Midjourney. Já no DALL-E, da OpenAI, é possível fazer o upload de uma imagem para gerar variações, como pode ser observado no exemplo a seguir, que apresenta variações da foto do meu perfil no LinkedIn:

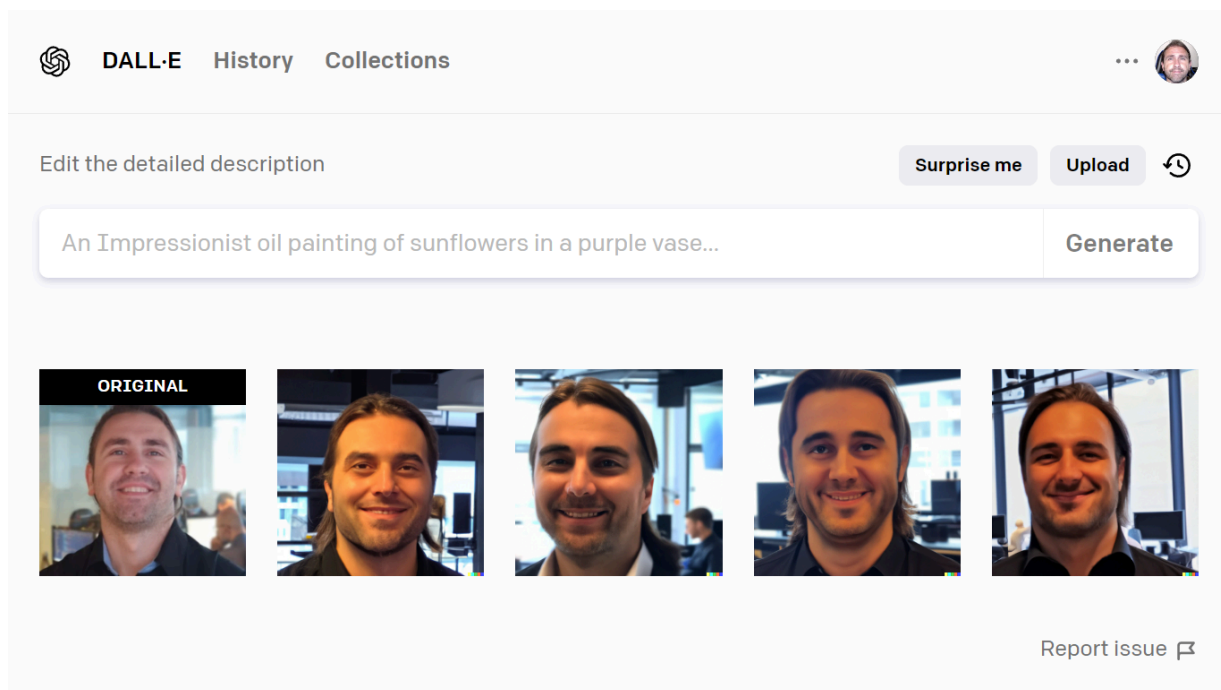


Figura 3.1: Exemplo de um prompt do DALL-E gerando variações de imagens a partir de uma foto minha como anexo

Abordagem com busca contextual automática

Alguns modelos de linguagem são configurados para realizar uma pesquisa automatizada em mecanismos de busca. É o caso do Bing, da Microsoft, e do Bard, do Google. A seguir, podemos observar como o Bard usa automaticamente o mecanismo de busca do

Google para fazer uma pesquisa em tempo real, citando, inclusive, a fonte.

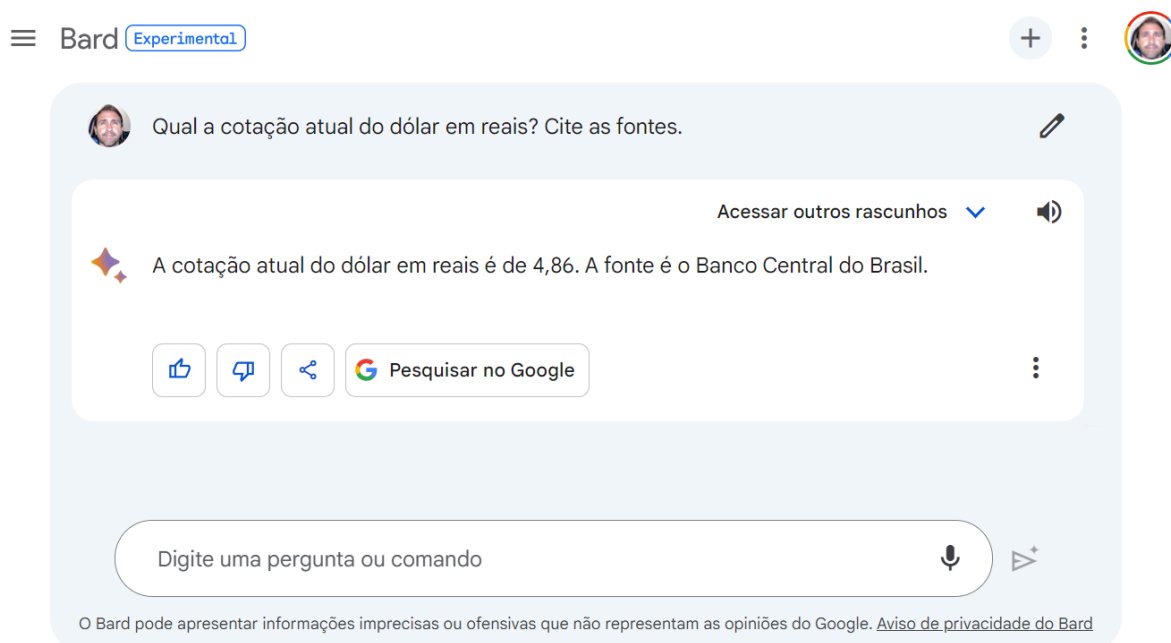


Figura 3.2: Exemplo de um prompt do Bard consultando a internet para dar a resposta

A título de comparação, o ChatGPT, que é um modelo de linguagem não configurado para usar mecanismos de busca, informa que não pode realizar a consulta, como mostra o exemplo a seguir:

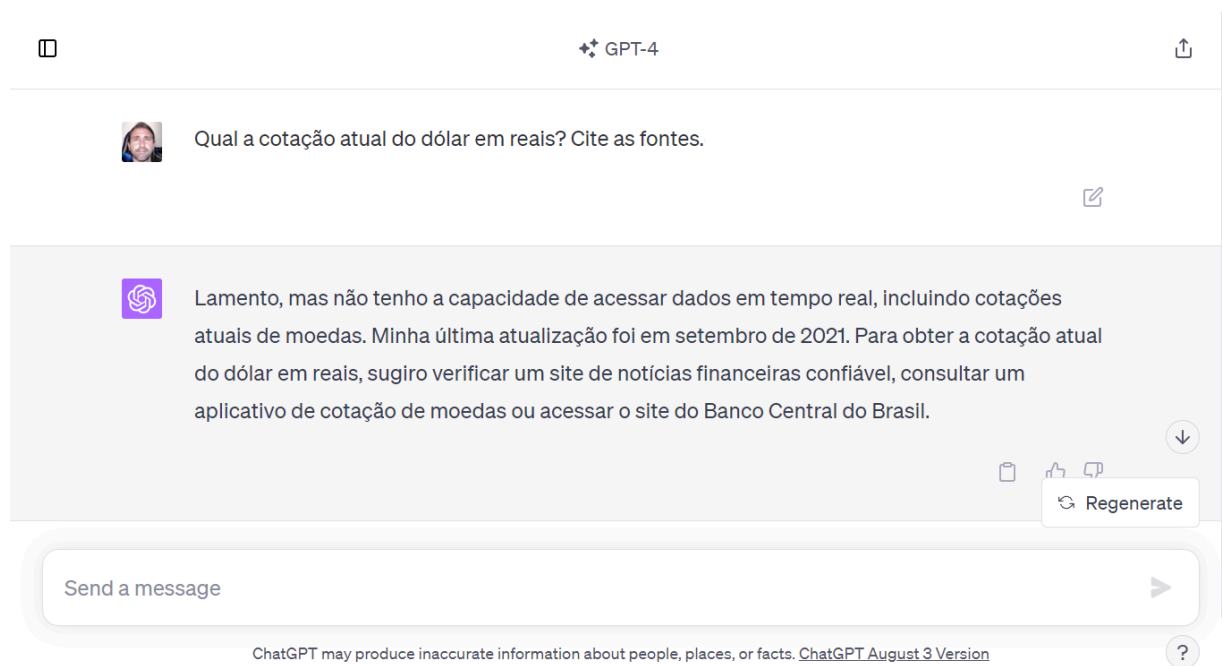


Figura 3.3: Exemplo de um prompt do ChatGPT informando não ser possível dar informações atuais

Abordagem com plugins contextuais complementares

Outra abordagem é a automatização de contextos por meio de plugins. O ChatGPT, por exemplo, implementou uma plataforma para programadores desenvolverem plugins que podem ser integrados à instrução. A imagem a seguir mostra como o usuário pode acessar o repositório de plugins, que já possui uma infinidade de implementações como geradores de documentos, leitores de PDF, geradores de diagramas, integradores de repositórios do GitHub e muitas outras funcionalidades.

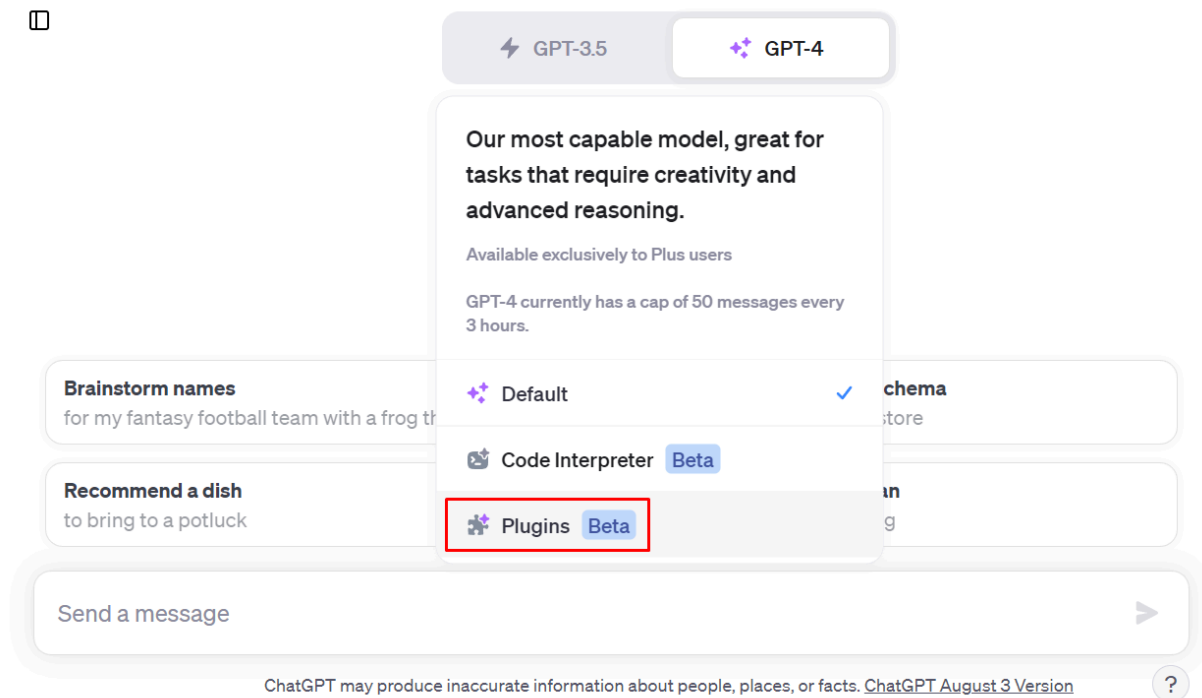


Figura 3.4: Exemplo de uso de plugins no ChatGPT

Para ilustrar o uso de plugins, podemos observar na imagem a seguir a mesma instrução sobre a cotação do dólar com o plugin Metaphor Search habilitado:

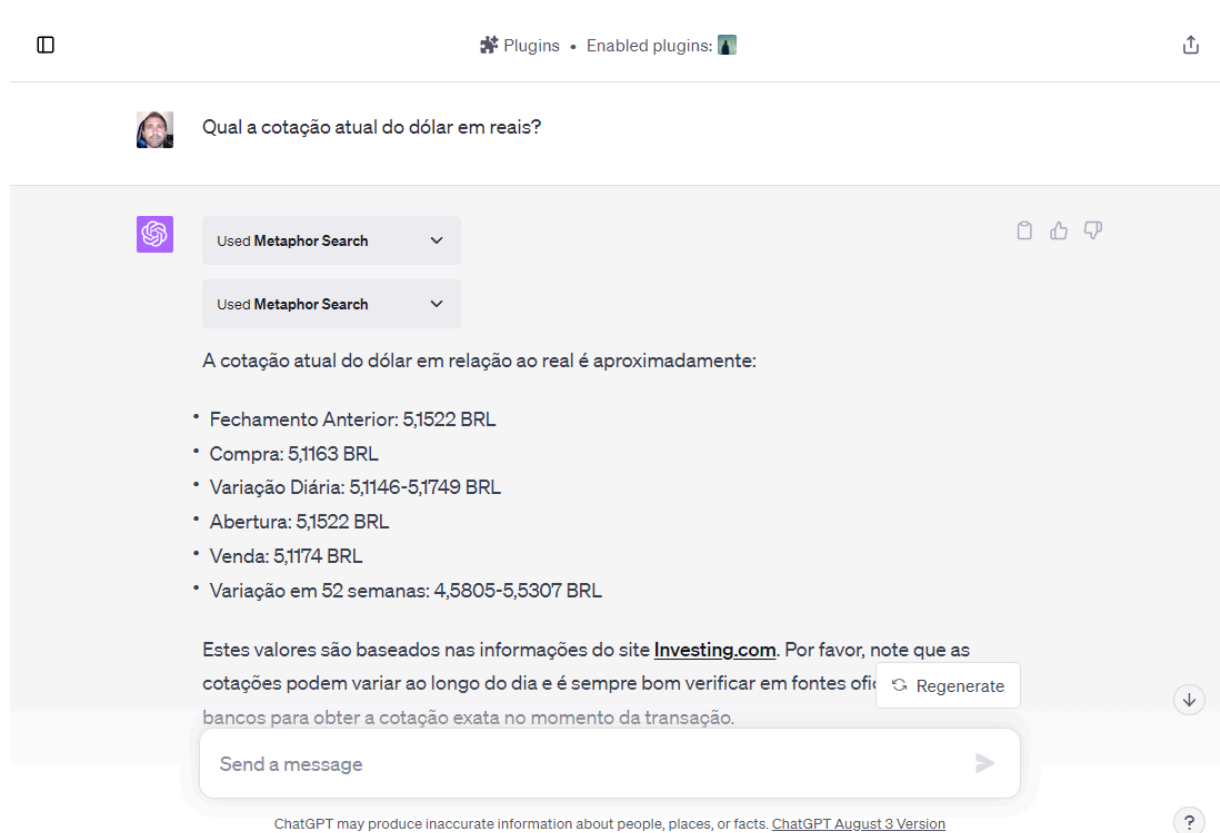


Figura 3.5: Exemplo de um prompt com o plugin 'Metaphor Search' em uso no ChatGPT

Abordagem com plataformas especialistas de terceiros

Com a popularização das inteligências artificiais, surgiram inúmeras soluções que utilizam as IAs como base para aplicações mais complexas ou especializadas. Existem desde agentes que automatizam tarefas usando conversas entre duas instâncias de modelos de linguagem até sites que geram e-books a partir da definição simplória de um tema genérico. É o caso, por exemplo, do site www.compose.ai, que oferece uma extensão do Chrome que utiliza IA exclusivamente para escrever e-mails. É uma ferramenta que usa prompts especializados e contextualizados com conversas prévias e direcionamentos específicos para a elaboração de e-mails. É um exemplo de aplicação especialista, mas que ilustra a quantidade ilimitada de possibilidades.

Há muitos sites e repositórios na internet que mantêm um catálogo de inteligências artificiais, o que pode ser muito útil para encontrarmos facilmente ferramentas que nos ajudam no dia a dia. Um recurso que uso e de que gosto muito é este repositório no GitHub chamado *Awesome Generative AI* (<https://github.com/steven2358/awesome-generative-ai>), mantido por Steven Van Vaerenbergh e que conta com a colaboração de dezenas de pessoas.

Outros exemplos de instruções bem formuladas

Nesta seção, apresentaremos alguns exemplos de prompts bem formulados que incorporam as técnicas discutidas nos tópicos anteriores. Esses exemplos demonstram como ser específico, fornecer contexto, definir o escopo e orientar a resposta ou ação esperada pode resultar em respostas mais úteis e relevantes dos modelos de linguagem.

Prompt mal formulado	Prompt bem formulado
X Como posso ganhar dinheiro com um blog?	✓ Quais são cinco estratégias eficazes para monetizar um blog sobre saúde e bem-estar, considerando que eu sou um profissional de fitness certificado?
X O que é machine learning?	✓ Explique o conceito de machine learning e forneça três exemplos de aplicações práticas dessa tecnologia em diferentes setores.
X Como posso melhorar a segurança da minha casa?	✓ Quais são as cinco principais medidas de segurança que eu posso implementar para proteger minha casa unifamiliar em um bairro urbano, levando em consideração um orçamento limitado?
X Como posso aumentar a produtividade no trabalho?	✓ Liste três estratégias comprovadas para aumentar a produtividade de uma equipe de desenvolvimento de software ágil, abordando fatores como comunicação, gerenciamento de tempo e organização do ambiente de trabalho.
X Como posso resolver um problema de drenagem no meu quintal?	✓ Descreva um processo passo a passo para solucionar um problema de drenagem em um quintal com declive moderado e solo argiloso, levando em consideração soluções de baixo custo e ecologicamente corretas.

Esses exemplos demonstram a importância de elaborar prompts de forma eficaz para obter respostas úteis e relevantes dos modelos de linguagem. Lembre-se de que a prática e a experimentação são fundamentais para melhorar suas habilidades na elaboração de prompts e na comunicação com sistemas baseados em IA.

3.2 A resposta (ou ação esperada)

Fazemos perguntas ou damos instruções para obter uma resposta. Em uma inteligência artificial que gera imagens, a resposta é na forma de uma imagem, enquanto que, em um modelo de linguagem, a resposta é sempre na forma textual. Porém, é possível solicitar formatos específicos na própria instrução, assim a IA fornecerá uma resposta formatada de acordo com o formato solicitado. É possível, por exemplo, pedir para que a resposta seja no formato de uma lista numerada ou não numerada, como mostrado a seguir, e a resposta será visualmente formatada exatamente como solicitado:



Escreva uma lista numerada com as 3 linguagens de programação mais populares e outra lista não numerada com 3 bancos de dados NoSQL quaisquer



Claro, aqui estão as listas que você pediu:

3 linguagens de programação mais populares:

1. Python
2. JavaScript
3. Java

3 bancos de dados NoSQL:

- MongoDB
- Cassandra
- Redis

Figura 3.6: Exemplo de uma resposta em formato de lista no ChatGPT

Em muitas situações, você pode precisar de uma tabela com dados tabulados em linhas e colunas específicas que permitam um manuseio fácil na hora de copiar e colar dados para uma planilha,

por exemplo. Observe a seguir como gerar informações tabuladas simplesmente solicitando o formato desejado na instrução:



Escreva uma tabela 5 linguagens de programação na primeira coluna, nível de performance na segunda coluna e curva de aprendizado na terceira coluna



Aqui está uma tabela com cinco linguagens de programação, seus níveis de desempenho e curvas de aprendizado aproximados:

Linguagem de Programação	Nível de Performance	Curva de Aprendizado
Python	Médio	Fácil
Java	Alto	Moderado
C++	Muito Alto	Desafiador
JavaScript	Médio	Moderado
Ruby	Moderado	Fácil

Lembrando que essas avaliações são aproximadas e podem variar dependendo do contexto e das habilidades do programador. Além disso, a curva de aprendizado também pode ser influenciada pela experiência prévia do programador em outras linguagens de programação.

Figura 3.7: Exemplo de uma resposta em formato de tabela no ChatGPT

Se você precisa de uma solução em código, também pode especificar esse formato. Por exemplo, peça: "*Mostre-me como implementar uma função de soma em Python.*" e obtenha um código formatado e indentado, pronto para ser colado na sua IDE:



Mostre-me como implementar uma função de soma em Python.



Claro! Você pode implementar uma função de soma simples em Python usando o operador de adição (+). Aqui está um exemplo de uma função de soma que aceita dois números como argumentos e retorna a soma deles:

python

Copy code

```
def somar(a, b):  
    resultado = a + b  
    return resultado
```

Você pode usar essa função da seguinte forma:

python

Copy code

```
resultado_da_soma = somar(3, 5)  
print(resultado_da_soma) # Isso imprimirá 8, que é a soma de 3 e 5
```

Esta função `somar` aceita dois argumentos, `a` e `b`, que são os números que você deseja somar. Ela calcula a soma dos dois números usando o operador de adição (+) e retorna o resultado. Você pode chamar essa função com quaisquer números que desejar para obter a soma deles.

Figura 3.8: Exemplo de uma resposta em formato de código-fonte no ChatGPT

Percebemos que a formatação pode melhorar significativamente a legibilidade e a compreensão de uma resposta. Aqui estão mais exemplos de como uma interface de resposta pode apresentar dados visualmente:



Me dê exemplos dos formatos de dados que você pode apresentar em sua resposta. Quero exemplos de citações, destaques, links, divisões e lista com checkboxes



Claro! Aqui estão exemplos dos formatos de dados que posso apresentar em minha resposta:

1. **Citações:**

Albert Einstein disse uma vez: "A imaginação é mais importante que o conhecimento."

2. **Destaques:**

Isso é um **destaque** de texto para chamar a atenção.

3. **Links:**

Você pode encontrar mais informações sobre isso [aqui](#).

4. **Divisões:**

Isso é uma divisão para separar o conteúdo.

5. **Lista com Checkboxes:**

- ☐ Tarefa 1
- ☒ Tarefa 2 (concluída)
- ☐ Tarefa 3

Espero que esses exemplos sejam úteis! Se você tiver alguma pergunta específica ou precisar de mais informações, sinta-se à vontade para perguntar.

Figura 3.9: Exemplo de uma resposta com formatos diversos no ChatGPT

Além desses formatos visuais de dados, em alguns modelos de linguagem ainda é possível usar plugins ou complementos que permitem visualizar informações em inúmeros formatos diferentes, o que torna as possibilidades praticamente ilimitadas.

Validação das respostas

É muito importante que o usuário entenda que um modelo de linguagem não é um mecanismo de busca, que devolve informações extraídas exatamente como as fontes pesquisadas. O modelo de

linguagem funciona de forma similar a um cérebro. Ele foi treinado para reagir textualmente a estímulos textuais. Embora seja muito assertivo e extremamente convincente em suas respostas, ele falha grosseiramente quando fornece informações factuais, especialmente informações não muito populares e, portanto, pouco associadas à sua base de aprendizagem.

Para ilustrar, observe a resposta do ChatGPT para a pergunta a seguir:

Você: *Em que ano foi fundada a Fatec Praia Grande?*

IA: A Fatec Praia Grande, que é uma das unidades da Faculdade de Tecnologia do Estado de São Paulo (Fatec), foi fundada em 2008. Vale lembrar que as Fatecs são instituições públicas de ensino superior no estado de São Paulo, e oferecem cursos de graduação tecnológica em diversas áreas. A Fatec Praia Grande é apenas uma das muitas unidades espalhadas pelo estado.

O problema dessa resposta é que ela está totalmente equivocada. Embora o ChatGPT tenha respondido de forma convicta, a Fatec Praia Grande foi fundada em 2002. Eu sei porque dou aula lá. Inclusive, leciono desde 2006, então, a não ser que tal memória tenha sido implantada artificialmente em meu cérebro, a Fatec Praia Grande já existia antes de 2008. Isso significa que informações factuais devem ser validadas SEMPRE!

Por outro lado, esses erros não ocorrem se a instrução contém os fatos a serem analisados. Observe a diferença da resposta a uma pergunta contextualizada (texto da instrução extraído de <https://fatecpg.edu.br/institucional/institucional>):

Você: *De acordo com o texto a seguir, quando foi fundada a Fatec Praia Grande?*

"A Fatec Praia Grande, iniciou suas atividades acadêmicas em 03 de setembro de 2002, oferecendo aos munícipes 80 (oitenta vagas) para o curso de Tecnologia em Informática – Ênfase em Gestão de Negócios sendo 40 para o turno vespertino e 40 vagas para o turno noturno. (...)"

IA: A Fatec Praia Grande foi fundada em 03 de setembro de 2002, conforme mencionado no início do texto fornecido.

Repare que, quando alimentada com dados, um modelo de linguagem consegue interpretá-los muito bem. Mas não devemos confiar cegamente em sua memória. Entender e explorar a capacidade de um modelo de linguagem fornecer respostas formatadas e contextualizadas permite uma interação mais rica e produtiva.

3.3 O contexto conversacional

Conversas com múltiplas interações oferecem uma oportunidade para refinamento e esclarecimento. A cada nova interação, o usuário pode aprofundar, especificar ou redirecionar sua consulta, permitindo ao modelo de linguagem entender melhor o contexto e as necessidades do usuário. Assim, em vez de oferecer uma resposta genérica ou limitada, o modelo pode fornecer soluções ou informações mais adaptadas à situação apresentada, produzindo um resultado mais preciso e benéfico para o usuário.

Como exemplo, suponhamos que um desenvolvedor deseje criar um sistema de recomendação para uma loja online e consulte o modelo. Na primeira interação, ele pode perguntar: *"Como funciona um sistema de recomendação?"*. Obtendo uma resposta geral sobre sistemas de recomendação, ele pode, então, seguir com: *"Quais algoritmos são mais usados para esse propósito?"*. Após entender os

algoritmos, ele pode aprofundar ainda mais: *"Como implementar um sistema de recomendação baseado em filtragem colaborativa?"*. Assim, progressivamente, o desenvolvedor obterá informações mais detalhadas e práticas para sua necessidade específica, em vez de uma resposta generalizada.

Existe um limite para o tamanho do texto de uma instrução a um modelo de linguagem. Esse limite é definido não em palavras, mas em tokens. O ChatGPT, utilizando o GPT-3, tem um limite de 2048 tokens. Um token pode ser tão curto quanto um caractere ou tão longo quanto uma palavra. Ele serve como a unidade básica de processamento de texto, o que torna difícil traduzir tokens em quantidade exata de caracteres ou palavras. Contudo, é importante saber que, ao tentar inserir em uma única instrução um texto muito extenso, como um artigo com muitas palavras, ocorrerá o seguinte erro:

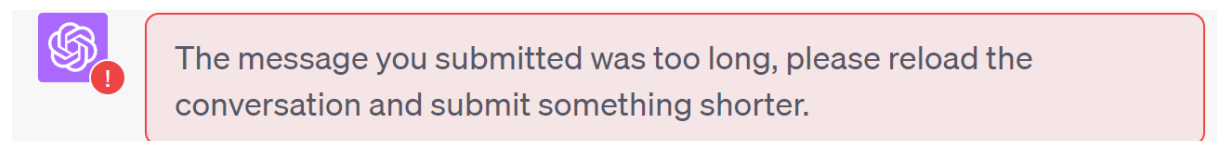


Figura 3.10: Exemplo de erro gerado por um prompt longo demais no ChatGPT

A má notícia é que esse limite também se aplica a toda a conversa. Em interfaces de conversação, onde várias interações acontecem sequencialmente, o contexto mantido pelo modelo — que inclui perguntas e respostas anteriores — é restringido pela capacidade máxima de tokens do modelo. No caso do GPT-3, esse limite é de 2048 tokens. Quando a conversa acumula tokens próximos a esse limite, as interações mais antigas são truncadas para dar espaço às novas. Isso significa que, após várias interações extensas, o modelo pode perder o contexto das mensagens iniciais da conversa.

Em resumo: mesmo em conversas prolongadas por vários dias, o modelo retém apenas o contexto que se encaixa dentro de seu limite de tokens, e as partes mais antigas são descartadas à medida que a conversa avança e novos tokens são adicionados.

Como contornar isso? Mantendo uma síntese resumida da conversa após algumas interações. Por exemplo, ao escrever um artigo, você pode solicitar que o ChatGPT o elabore com base apenas no que ele já conhece, mas o resultado seria superficial. Para aprimorá-lo, inicie uma conversa citando outro artigo e peça um resumo. Solicite o resumo de outro texto, de uma matéria em um jornal e, sobretudo, suas reflexões sobre o assunto abordado. A cada passo, peça uma síntese do que foi discutido. Ao final, você poderá solicitar um artigo baseado nos critérios estabelecidos. Esse método também pode ser aplicado na elaboração de documentos ou na codificação de um sistema.

Suponhamos que você é um(a) desenvolvedor(a) e está projetando um sistema de gerenciamento de bibliotecas para uma universidade. Você decide consultar o ChatGPT para obter insights e orientações sobre as melhores práticas e funcionalidades que um sistema desse tipo pode oferecer.

Interação 1:

Você: *ChatGPT, quais são as características essenciais de um bom sistema de gerenciamento de bibliotecas?*

ChatGPT: Um sistema eficaz de gerenciamento de bibliotecas deve oferecer: gestão de acervo, controle de empréstimos e devoluções, integração com catálogos online, gestão de usuários, lembretes de vencimento, relatórios estatísticos, e suporte a diferentes formatos, como livros, e-books e periódicos.

Interação 2:

Você: *E quanto à segurança e à integridade dos dados? Quais recursos você recomendaria?*

ChatGPT: Para garantir a segurança e integridade dos dados, o sistema deve incluir: backups regulares, autenticação de dois fatores, criptografia de dados em trânsito e em repouso, controle de

acesso baseado em funções e logs de atividades para rastrear alterações.

Interação 3:

Você: *Como poderíamos implementar uma interface amigável para os usuários?*

ChatGPT: Uma interface amigável pode ser alcançada através de design responsivo (adaptável para dispositivos móveis), navegação intuitiva, pesquisa avançada com filtros, feedback visual claro, e treinamentos ou tutoriais para novos usuários.

Depois de muitas interações e, talvez, perto do limite de tokens, você decide consolidar tudo.

Interação 4:

Você: *Poderia resumir as recomendações para o desenvolvimento de um sistema de gerenciamento de bibliotecas?*

ChatGPT: Claro! Um sistema de bibliotecas ideal deve oferecer gestão de acervo, controle de empréstimos, integração com catálogos e gerenciamento de usuários. A segurança é crucial, então inclua backups, autenticação de dois fatores, criptografia e controle de acesso. A interface deve ser responsiva, com navegação intuitiva, pesquisa avançada e treinamentos para os usuários.

Com essa síntese, você tem uma visão clara dos pontos-chaves a serem abordados no seu projeto, mesmo que as interações iniciais eventualmente sejam truncadas pelo modelo devido ao limite de tokens.

A profundidade e riqueza de uma conversa muitas vezes estão atreladas ao contexto que a envolve. No âmbito dos modelos de linguagem, esse contexto é uma poderosa ferramenta para aprimorar e orientar a interação. Ao compreender como o contexto impacta a conversa, otimizamos nossa abordagem na interação com a IA.

CAPÍTULO 4

Testes de prompts

"Devemos investigar e aceitar os resultados. Se não resistem a estes testes, até as palavras de Buda devem ser rejeitadas" —
Dalai Lama

No universo da Engenharia de Prompt, a importância de testar e otimizar prompts vai além do mero aperfeiçoamento. É uma fase que revela a eficiência, precisão e adaptabilidade dos seus prompts, sobretudo quando o contexto envolve algo tão intrincado quanto o desenvolvimento de um software.

Vamos começar discutindo o método de *teste manual*, que envolve um processo minucioso de revisão das respostas que o modelo de linguagem entrega com base no prompt fornecido. No domínio do desenvolvimento de um software, isso pode ser particularmente complexo. Estamos falando não apenas de sintaxe correta, mas também de clareza, otimização e alinhamento com as especificações originais. O ciclo é contínuo: ajuste o prompt e repita o teste até que você alcance os resultados que procurava.

A abordagem de *teste iterativo* leva esse processo um passo além. Aqui, o foco recai sobre um ciclo contínuo de refinamento. Esse método não envolve apenas o ajuste dos prompts, mas também do contexto, dos exemplos e das instruções que acompanham o prompt. É uma abordagem de aprendizado em que você testa, mede, ajusta e volta a testar, aprimorando consistentemente a qualidade e a eficácia dos seus prompts.

Teste de múltiplas variações é uma outra estratégia. Imagine que você tenha um prompt bem elaborado. No entanto, há sempre espaço para inovação. Criar e testar diversas variações desse prompt pode revelar nuances que fazem toda a diferença. Além de fornecer

insights sobre o que funciona melhor, ajuda você a entender os elementos que são realmente eficazes e aqueles que podem ser descartados ou reformulados.

No *teste de casos extremos*, você testará seu prompt sob condições extremas ou com parâmetros pouco usuais para verificar como ele se comporta. Essa é uma etapa que contribui significativamente para a robustez do seu prompt.

A diversidade de modelos de linguagem disponíveis hoje nos leva ao próximo método, o *teste com diferentes modelos*. Essa abordagem serve para entender o comportamento do seu prompt em tecnologias diversas e com diferentes níveis de complexidade. O seu prompt pode ser eficaz com um modelo, mas e quanto a outros modelos?

Por último, a *avaliação de desempenho* pode parecer um tanto abstrata, mas é aqui que entram métricas objetivas. Avaliar técnica e objetivamente, por exemplo, um código gerado pela IA ajuda você a fazer ajustes refinados no prompt. Sem métricas, você está navegando às cegas.

4.1 Teste manual

O **teste manual** é um método de avaliação em que a pessoa analista ou desenvolvedora examina individualmente as respostas produzidas por um modelo de linguagem em resposta a um prompt específico. Diferentemente de abordagens automatizadas, o foco aqui está na análise qualitativa da saída, dando espaço para ajustes que melhorem a qualidade e aplicabilidade da resposta gerada. Esse método é particularmente eficaz para afinar a resposta, pois permite uma revisão minuciosa da precisão, clareza e utilidade do conteúdo produzido. E, claro, é uma revisão analítica e particular.

Exemplo 1: Gerando documentação RESTful

Como exemplo, suponha que você usou um modelo de linguagem para gerar a documentação de uma API RESTful. O prompt pode ser algo como: "*Escreva uma descrição simples, mas completa, do endpoint `GET /users`*". Após executar o prompt, você observa que a descrição gerada é vaga e não inclui detalhes como parâmetros opcionais ou códigos de resposta HTTP. Nesse caso, o teste manual lhe permite perceber essa falta de detalhamento, e você pode optar por refinar o prompt para solicitar uma descrição mais minuciosa, como "*Escreva uma descrição detalhada do endpoint `GET /users`, incluindo parâmetros opcionais e códigos de resposta HTTP*", o que lhe retornará o seguinte código:

O endpoint `GET /users` recupera uma lista de usuários.

Parâmetros opcionais:

- 'page': número da página de resultados
- 'limit': número máximo de resultados por página

Códigos de Resposta HTTP:

- 200 OK: Operação bem-sucedida
- 400 Bad Request: Requisição inválida
- 401 Unauthorized: Falha na autenticação
- 500 Internal Server Error: Erro no servidor

Exemplo 2: Query de migração de dados entre tabelas

Em um outro exemplo, você está buscando um script de SQL para migrar dados entre tabelas. Seu prompt inicial poderia ser: "*Gere um script SQL para copiar todos os dados da tabela A para a tabela B*", o que retornaria o seguinte resultado:

```
INSERT INTO B SELECT * FROM A;
```

Ao verificar o resultado, você percebe que o script gerado não leva em consideração a existência de chaves estrangeiras ou índices únicos, o que poderia resultar em violações de *constraint* durante a execução. Um prompt mais refinado como "*Gere um script SQL para*

copiar todos os dados da tabela A para a tabela B, considerando chaves estrangeiras e índices únicos" retornaria uma resposta igualmente mais refinada:

```
-- Desativar restrições de chave estrangeira
ALTER TABLE B DISABLE TRIGGER ALL;

-- Inserir dados levando em conta chaves únicas
INSERT INTO B (col1, col2, ...)
SELECT col1, col2, ...
FROM A
ON DUPLICATE KEY UPDATE
col1 = VALUES(col1), col2 = VALUES(col2);

-- Reativar restrições de chave estrangeira
ALTER TABLE B ENABLE TRIGGER ALL;
```

Outro problema que pode acontecer nesta situação é o modelo do banco de dados previamente informado na conversa estar ultrapassando o limite de tokens do modelo de linguagem (como visto no capítulo anterior), o que fará a IA ignorá-lo parcial ou integralmente e dar instruções erradas. Ao identificar essa limitação através do teste manual, você decide aprimorar o prompt incluindo como contexto o código do modelo físico de dados antes da instrução. Com isso, o modelo gera um script muito mais alinhado às complexidades envolvidas no processo de migração.

Lembre-se de que a IA é treinada para responder, não para perguntar. Daí a importância de testar manualmente todas as respostas produzidas pelos seus prompts antes de implementá-las efetivamente.

4.2 Teste iterativo

O método de **teste iterativo** centra-se na avaliação e ajuste contínuos de um prompt, com a finalidade de aprimorar a qualidade da resposta gerada pelo modelo de linguagem.

Em geral, o teste manual e o teste iterativo são complementares. Enquanto o teste manual serve para analisar o resultado de um prompt individualmente, o teste iterativo serve para testar uma sequência de prompts que vão produzir uma resposta mais complexa.

Exemplo 1: Query de filtragem avançada

Inicialmente, você pode ter um prompt como: *"Crie uma query SQL para selecionar registros da tabela `Produtos` onde o preço seja maior que 100"*.

Resposta inicial do modelo:

```
SELECT * FROM Produtos WHERE preco > 100;
```

Depois de testar essa query, você percebe que também é importante filtrar produtos que estão em estoque. Você ajusta o prompt para: *"Crie uma query SQL para selecionar registros da tabela `Produtos` onde o preço seja maior que 100 e o item esteja em estoque"*.

Resposta ajustada do modelo:

```
SELECT * FROM Produtos WHERE preco > 100 AND estoque > 0;
```

Ao iterar dessa maneira, a query se torna mais alinhada às suas necessidades.

Exemplo 2: Algoritmo de classificação

Seu prompt inicial pode ser algo como: *"Escreva um algoritmo em Python que classifique um comentário como positivo ou negativo com base nas palavras usadas"*.

Resposta inicial do modelo:

```
def classificar_comentario(comentario):
    palavras_positivas = ['bom', 'ótimo', 'excelente']
    palavras_negativas = ['ruim', 'péssimo', 'terrível']

    score = 0

    for palavra in comentario.split():
        if palavra in palavras_positivas:
            score += 1
        elif palavra in palavras_negativas:
            score -= 1

    return 'Positivo' if score > 0 else 'Negativo'
```

Você testa e percebe que o modelo não está usando nenhuma biblioteca de processamento de texto com recursos mais avançados, embora que você saiba que o Python possui. Então, você atualiza o prompt: *"Escreva um algoritmo em Python que classifique um comentário como positivo ou negativo, levando em consideração palavras-chave e o contexto em que elas aparecem"*.

Resposta ajustada do modelo:

```
from textblob import TextBlob

def classificar_comentario_avancado(comentario):
    analise = TextBlob(comentario)

    if analise.sentiment.polarity > 0:
        return 'Positivo'
    else:
        return 'Negativo'
```

Com essas iterações, o algoritmo se torna mais robusto e alinhado às necessidades de análise de sentimento. O método de teste iterativo, nesses exemplos, demonstra como uma série de pequenos

ajustes pode levar a respostas significativamente mais eficazes e precisas.

4.3 Teste de múltiplas variações

O método de **teste de múltiplas variações** envolve a criação e avaliação de diferentes versões de um mesmo prompt para identificar qual abordagem oferece a melhor resposta. Esse tipo de teste é altamente eficaz quando você está em busca de otimizar a eficácia de um prompt, sobretudo quando há várias formas de se abordar uma tarefa específica.

Exemplo 1: Busca em banco de dados

Suponha que você queira uma query SQL para buscar registros de usuários que são administradores. Você pode criar várias versões do prompt:

- *"Escreva uma query SQL para buscar todos os usuários com o papel de administrador."*
- *"Como eu buscaria administradores em uma tabela de usuários via SQL?"*

Resposta para o primeiro prompt:

```
SELECT * FROM usuarios WHERE papel = 'administrador';
```

Resposta para o segundo prompt:

Para buscar administradores na tabela de usuários, você poderia usar a seguinte query SQL:

```
SELECT * FROM usuarios WHERE papel ilike 'admin%';
```

Com essas variações, você pode notar que o segundo prompt incluiu uma abreviação ("admin" em vez de "administrador") e retornando

possíveis variações de nomes de papéis, algo que você pode querer ajustar dependendo do seu banco de dados.

Exemplo 2: Ordenação de listas em Python

Você deseja um código em Python para ordenar uma lista de números em ordem crescente. Novamente, você pode testar várias versões do prompt:

- *"Me mostre como ordenar uma lista de números em Python."*
- *"Qual é a sintaxe para ordenar uma lista em ordem crescente em Python?"*

Resposta para o primeiro prompt:

```
numeros = [4, 2, 9, 1]
numeros_ordenados = sorted(numeros)
```

Resposta para o segundo prompt:

Para ordenar uma lista em ordem crescente em Python, você pode utilizar o método `sort()` da seguinte forma:

```
lista = [3, 1, 4, 1, 5, 9, 2, 6, 5]
lista.sort()
```

Aqui, as diferentes variações ajudam você a entender as possíveis formas de abordar a tarefa e escolher a que melhor se adapta às suas necessidades.

Através de múltiplas variações, você ganha uma visão mais completa sobre como diferentes formulários de prompts podem afetar a qualidade e utilidade da resposta gerada, o que permite selecionar a versão mais adequada para a sua aplicação.

4.4 Teste de casos extremos de prompts

No **teste de casos extremos**, você verifica como um prompt se comporta quando confrontado com cenários pouco convencionais ou extremos. Isso pode revelar possíveis fragilidades na resposta gerada e ajudar você a fazer ajustes para aumentar a robustez do prompt.

Exemplo 1: Tratamento de entradas inválidas em código JavaScript

Imagine que você tem um prompt para gerar uma função JavaScript que divide dois números. Você quer testá-lo para ver como ele lida com a divisão por zero, um caso extremo.

Prompt: *"Escreva uma função em JavaScript que divida dois números."*

Resposta comum:

```
function dividir(a, b) {  
  return a / b;  
}
```

Teste de caso extremo: *"Escreva uma função em JavaScript que divida dois números e trate a divisão por zero."*

Resposta adaptada:

```
function dividir(a, b) {  
  if (b === 0) {  
    return "Divisão por zero não permitida";  
  }  
  return a / b;  
}
```

Ao testar o caso extremo de divisão por zero, você pode identificar e corrigir uma falha potencial no código gerado.

Exemplo 2: Verificação de overflow em C++

Suponha que você esteja trabalhando com operações de adição em C++ e queira garantir que o prompt gere código que possa lidar com *overflow* de inteiros.

Prompt: *"Escreva uma função em C++ para adicionar dois inteiros."*

Resposta comum:

```
int adicionar(int a, int b) {  
    return a + b;  
}
```

Teste de caso extremo: *"Escreva uma função em C++ para adicionar dois inteiros e trate o caso de overflow."*

Resposta adaptada:

```
#include <limits.h>  
bool adicionar(int a, int b, int &resultado) {  
    if ((b > 0 && a > INT_MAX - b) || (b < 0 && a < INT_MIN - b)) {  
        return false; // overflow ocorreu  
    }  
    resultado = a + b;  
    return true; // operação bem-sucedida  
}
```

Nesse exemplo, o teste de caso extremo revela como o código se comporta quando confrontado com a possibilidade de *overflow*, permitindo que você ajuste o prompt e sua resposta para cobrir essa eventualidade.

Essas práticas de teste de casos extremos são importantes para desenvolver prompts mais confiáveis e seguros, especialmente quando usados em aplicações críticas.

4.5 Teste com diferentes modelos de linguagem

Ao trabalhar com modelos de linguagem, é importante notar que cada modelo tem suas próprias características, limitações e áreas de eficácia. O **teste com diferentes modelos** consiste em avaliar a resposta de um prompt em diversos modelos de linguagem para entender seu comportamento em diferentes contextos e níveis de complexidade.

Exemplo 1: Geração de consultas SQL

Suponha que você tenha um prompt destinado a gerar uma consulta SQL para selecionar todos os registros de uma tabela que atendem a uma determinada condição.

Prompt: *"Gerar uma consulta SQL para selecionar todos os registros da tabela `usuarios` onde a idade é maior que 21."*

Resposta do ChatGPT 3.5:



Gerar uma consulta SQL para selecionar todos os registros da tabela `usuarios` onde a idade é maior que 21.



Claro! Você pode usar a seguinte consulta SQL para selecionar todos os registros da tabela `usuarios` onde a idade seja maior que 21:

sql

Copy code

```
SELECT * FROM usuarios  
WHERE idade > 21;
```

Essa consulta irá recuperar todos os registros da tabela `usuarios` onde o valor da coluna `idade` seja maior que 21. Certifique-se de substituir `usuarios` pelo nome real da tabela, caso seja diferente.

Figura 4.1: Resposta do ChatGPT 3.5, simples e descontextualizada

Resposta do Bard:

Gerar uma consulta SQL para selecionar todos os registros da tabela usuarios onde a idade é maior que 21.

Claro. Aqui está uma consulta SQL para selecionar todos os registros da tabela usuários onde a idade é maior que 21:

```
SQL
SELECT *
FROM usuarios
WHERE idade > 21;
```

Use o código com cuidado. [Saiba mais](#)

Esta consulta usa a cláusula `SELECT` para especificar os campos que deseja retornar. No caso, estamos retornando todos os campos da tabela usuários.

A cláusula `FROM` especifica a tabela da qual deseja selecionar registros. No caso, estamos selecionando registros da tabela usuários.

A cláusula `WHERE` especifica um filtro que limita os registros que serão selecionados. No caso, estamos filtrando os registros para que apenas aqueles com idade superior a 21 sejam selecionados.

Aqui está um exemplo de saída desta consulta:

id	nome	sobrenome	idade
--	--	--	--
1	João	Silva	25
2	Maria	Santos	30
3	Pedro	Souza	28

Você também pode usar a seguinte consulta para obter o mesmo resultado:

```
SQL
SELECT *
FROM usuarios
WHERE idade >= 21;
```

Use o código com cuidado. [Saiba mais](#)

A única diferença entre esta consulta e a anterior é que a cláusula `WHERE` usa o operador `>=` em vez de `>`. O operador `>=` retorna registros com idade igual ou superior a 21.

Figura 4.2: Resposta do Bard, mais completa e com alternativas

Resposta do Bing:

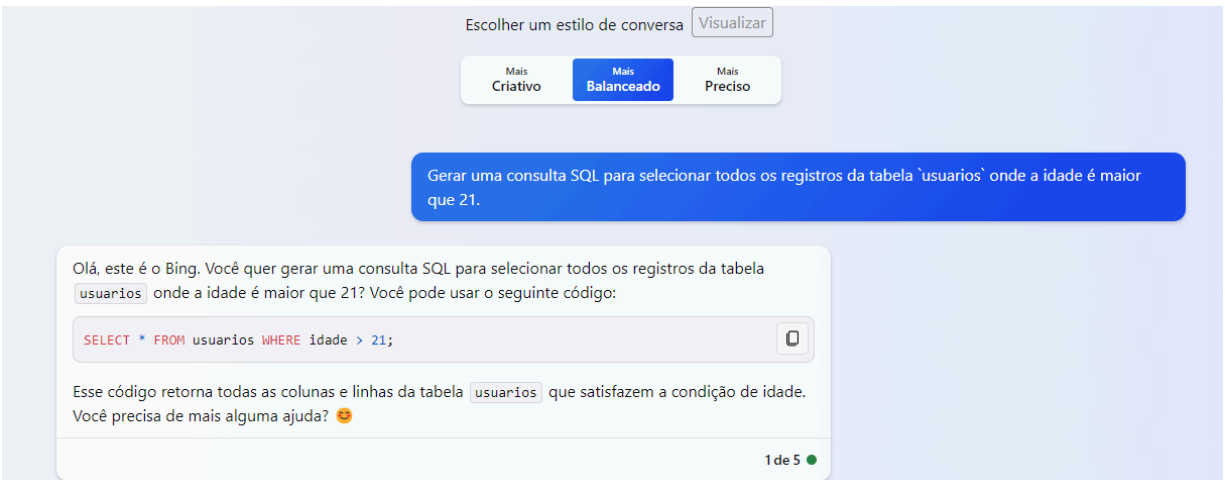


Figura 4.3: Resposta do Bing, simples porém com opções de escolha de estilos

Repare que as respostas não são exatamente iguais. E ainda é preciso considerar que o ChatGPT tem versões alternativas e o Bing tem a possibilidade de escolha de estilos diferentes de resposta.

Exemplo 2: Criação de classes em Python

Imagine que você tem um prompt para criar uma classe Python básica que represente um carro.

Prompt: *"Crie uma classe em Python para representar um carro com métodos para ligar e desligar o motor."*

Resposta do ChatGPT 3.5:



Crie uma classe em Python para representar um carro com métodos para ligar e desligar o motor.



Claro, aqui está um exemplo de uma classe em Python que representa um carro com métodos para ligar e desligar o motor:

python

Copy code

```
class Carro:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo
        self.motor_ligado = False

    def ligar_motor(self):
        if not self.motor_ligado:
            print(f"O motor do {self.marca} {self.modelo} foi ligado.")
            self.motor_ligado = True
        else:
            print(f"O motor do {self.marca} {self.modelo} já está ligado.")

    def desligar_motor(self):
        if self.motor_ligado:
            print(f"O motor do {self.marca} {self.modelo} foi desligado.")
            self.motor_ligado = False
        else:
            print(f"O motor do {self.marca} {self.modelo} já está desligado.")

# Exemplo de uso:
meu_carro = Carro("Toyota", "Corolla")
meu_carro.ligar_motor()
meu_carro.desligar_motor()
meu_carro.ligar_motor()
```

Neste exemplo, a classe `Carro` tem um construtor `__init__` que aceita a marca e o modelo do carro como argumentos e inicializa o estado do motor como desligado. Ela também possui métodos `ligar_motor` e `desligar_motor` que permitem ligar e desligar o motor, respectivamente. Quando esses métodos são chamados, eles imprimem mensagens informando o estado do motor.

Figura 4.4: Resposta do ChatGPT 3.5, mais simples

Resposta do Bard:



Crie uma classe em Python para representar um carro com métodos para ligar e desligar o motor.



Claro, aqui está uma classe em Python para representar um carro com métodos para ligar e desligar o motor:

Python

```
class Carro:

    def __init__(self, marca, modelo, ano):
        self.marca = marca
        self.modelo = modelo
        self.ano = ano
        self.motor_ligado = False

    def ligar_motor(self):
        self.motor_ligado = True
        print("O motor do carro foi ligado.")

    def desligar_motor(self):
        self.motor_ligado = False
        print("O motor do carro foi desligado.")

carro = Carro("Volkswagen", "Gol", 2023)
print(carro.marca, carro.modelo, carro.ano)
print(carro.motor_ligado)

carro.ligar_motor()
print(carro.motor_ligado)

carro.desligar_motor()
print(carro.motor_ligado)
```

Use o código com cuidado. [Saiba mais](#)



Este código define uma classe chamada `Carro` com três atributos: `marca`, `modelo` e `ano`. Também define dois métodos: `ligar_motor()` e `desligar_motor()`.

O método `desligar_motor()` define o atributo `motor_ligado` como `False` e imprime uma mensagem indicando que o motor foi desligado.

Aqui está um exemplo de como usar a classe:

Python

```
carro = Carro("Volkswagen", "Gol", 2023)
print(carro.marca, carro.modelo, carro.ano)
print(carro.motor_ligado)

carro.ligar_motor()
print(carro.motor_ligado)

carro.desligar_motor()
print(carro.motor_ligado)
```

Use o código com cuidado. [Saiba mais](#)



Este código imprime o seguinte:

```
Volkswagen Gol 2023
False
True
False
```

Você pode adicionar outros atributos e métodos à classe para representar outros aspectos do carro, como a cor, o número de portas, o tipo de motor, etc.

Figura 4.5: Resposta do Bard, com mais atributos e comportamentos

Resposta do Bing:

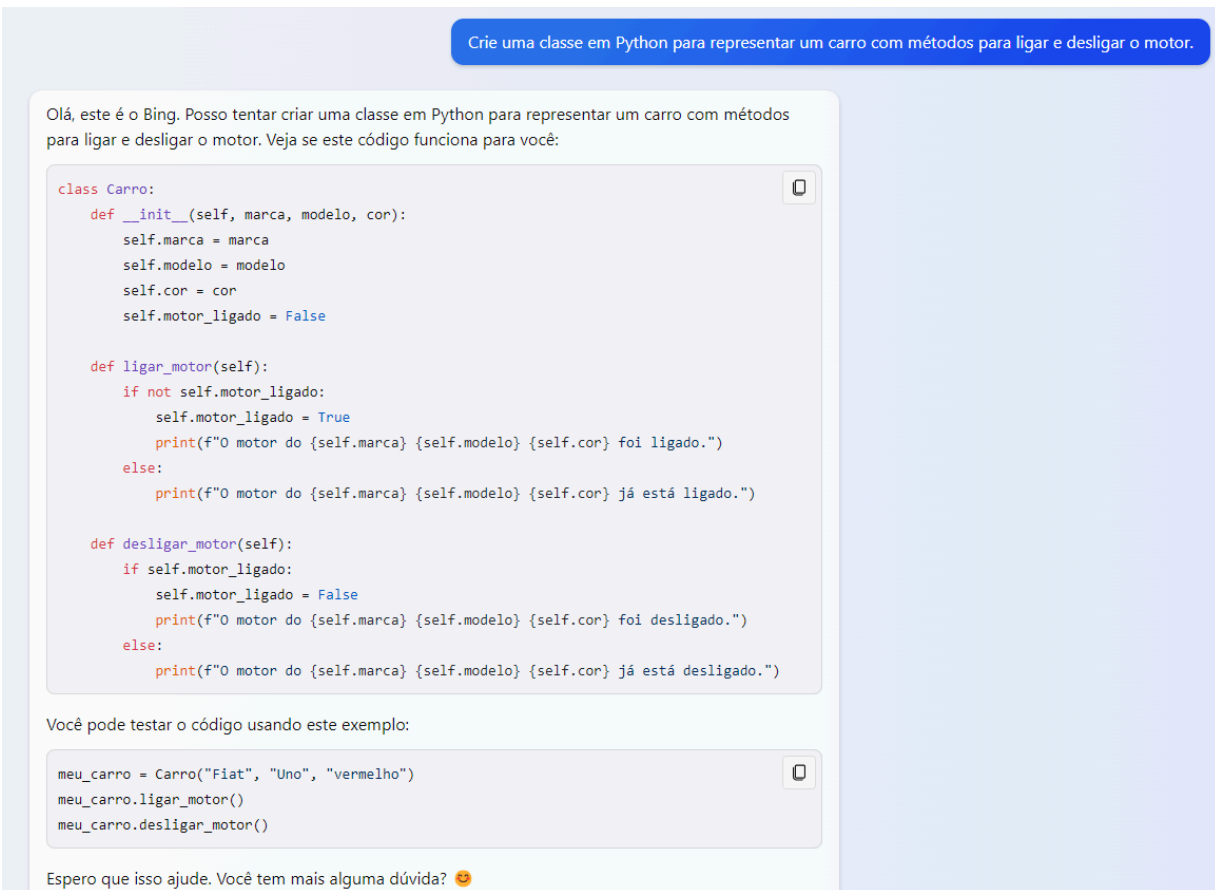


Figura 4.6: Resposta do Bing, simples e muito semelhante ao ChatGPT 3.5

Nesse exemplo, o modelo do ChatGPT (que é também usado pelo Bing) gera um código mais completo, incluindo mais atributos e métodos com condicionais desejáveis. Ao testar o mesmo prompt em diferentes modelos de linguagem, você pode discernir como as respostas variam em complexidade e utilidade, permitindo ajustes mais informados.

Em conclusão, o teste e a otimização de prompts são etapas importantes no processo de engenharia de prompt. Através desses métodos, é possível identificar e corrigir problemas potenciais, melhorar a qualidade e a eficácia dos prompts, e garantir que eles atendam às necessidades específicas do projeto. Ao aplicar esses métodos de forma sistemática, os usuários podem criar prompts

mais eficientes, precisos e adaptáveis, que vão gerar resultados melhores.

CAPÍTULO 5

Organização de prompts

"Com organização e tempo, acha-se o segredo de fazer tudo e bem feito" — Pitágoras

Uma vez que você entende os componentes de um prompt, especialmente as características implícitas de interação, podemos agora discutir como criar prompts realmente eficientes para você. Neste capítulo, discutiremos algumas estratégias para criar prompts realmente eficientes, que aumentem sua produtividade e potencializem suas habilidades na análise e no desenvolvimento de sistemas.

5.1 Identificando necessidades e objetivos

Antes de embarcar em qualquer iniciativa para incorporar a Inteligência Artificial (IA) em seus processos de desenvolvimento e análise de sistemas, a etapa mais importante é identificar claramente suas necessidades e objetivos. Parece óbvio, mas muitos profissionais pulam este passo crítico e correm atrás das novidades tecnológicas sem ponderar o verdadeiro valor que elas podem agregar ao seu trabalho.

Comece fazendo um diagnóstico do seu projeto ou fluxo de trabalho. Identifique tarefas que são rotineiras, consomem muito tempo e exigem pouco raciocínio crítico. São essas tarefas que oferecem o maior potencial para automação através da IA. Um exemplo comum pode ser a triagem inicial de bugs relatados, onde um modelo de linguagem pode categorizar e priorizar esses relatos com base no

texto descritivo, liberando tempo para que analistas se concentrem em problemas mais complexos.

Uma vez que você tenha uma lista de possíveis tarefas candidatas para automação ou otimização, é hora de definir seus objetivos. Pergunte a si mesmo: "*O que eu espero alcançar ao automatizar esta tarefa?*". Pode ser algo tão simples quanto "reduzir o tempo gasto na triagem de bugs em 30%" ou algo mais complexo como "melhorar a qualidade do código ao implementar revisões de código assistidas por IA".

O próximo passo é alinhar esses objetivos com as capacidades reais dos modelos de linguagem e IA disponíveis. Lembre-se: a tecnologia é apenas uma ferramenta; seu sucesso dependerá de quão bem você pode moldá-la para atender às suas necessidades específicas. A avaliação correta das necessidades e a definição clara dos objetivos não apenas facilitarão o uso da IA, mas também fornecerão um quadro referencial para medir o sucesso da sua iniciativa.

Mas antes de decidir sobre quais tarefas automatizar ou otimizar, é importante avaliar as competências e habilidades da sua equipe. Alguns membros podem já ter experiência com IA e podem servir como líderes de projeto nesse sentido. Além disso, entender o nível de conhecimento da equipe pode ajudar a identificar quais soluções são mais viáveis de serem implementadas.

Depois de identificar as tarefas e definir objetivos, uma análise de custo-benefício pode ser extremamente útil. Isso inclui não apenas os custos financeiros, mas também o tempo de implementação e manutenção. Portanto, isso ajudará a priorizar quais tarefas devem ser abordadas primeiro e a determinar se a solução de IA proposta oferece um retorno de investimento positivo.

Existem várias ferramentas e plataformas disponíveis para implementar soluções de IA. Avaliar qual delas melhor atende às suas necessidades é importantíssimo. Considere aspectos como

escalabilidade, facilidade de uso, custo e, claro, eficácia na realização das tarefas que você está procurando otimizar. Mas considere uma adoção híbrida. ChatGPT, Bing ou Bard, por exemplo, são modelos de linguagem genéricos que podem auxiliar não apenas na produção de código, mas em tarefas de engenharia de software como modelagem, documentação e análise. Por outro lado, o GitHub Copilot é muito mais ágil e eficiente na produção de código por estar integrado à IDE.

Antes de uma implementação em larga escala, sempre é aconselhável começar pequeno. Escolha uma tarefa e aplique a solução de IA a ela. Monitore os resultados de perto para avaliar se os objetivos estão sendo atingidos e quais ajustes podem ser necessários.

Após a implementação, considere continuar monitorando o desempenho e coletar feedbacks de todos os usuários envolvidos. A IA e os modelos de linguagem são ferramentas em constante evolução, e seus fluxos de trabalho também evoluirão. Continuar ajustando e aprimorando sua abordagem manterá a relevância e a eficácia da solução ao longo do tempo.

5.2 Organizando um banco de prompts

Identificar necessidades e objetivos é apenas um passo no uso inteligente da Inteligência Artificial em seu fluxo de trabalho. O próximo passo é organizar esses prompts de uma forma que facilite o acesso e a colaboração entre os membros da equipe. Imagine ter um arsenal de prompts eficazes que podem ser aplicados rapidamente em diversos projetos. Isso não só economiza tempo, mas também contribui para a qualidade e consistência do trabalho.

Crie um banco de prompts

Uma das melhores formas de fazer isso é criar um "banco de prompts compartilhado". Este pode ser um simples repositório em um sistema de gerenciamento de versão como o Git, um documento compartilhado em uma plataforma de colaboração como o Google Drive, ou mesmo uma base de dados especializada. O que importa é que ele seja facilmente acessível e utilizável por todos os membros da equipe.

Para tornar o banco de prompts realmente útil, considere implementar um sistema de categorização ou etiquetagem. Prompts para *debugging*, por exemplo, poderiam ser agrupados sob uma categoria própria, enquanto prompts para geração de código podem ter sua própria seção. Isso facilita encontrar o prompt mais adequado para uma tarefa específica.

Dado que tanto as necessidades do projeto quanto as capacidades dos modelos de linguagem estão em constante evolução, o banco de prompts deve incluir um sistema de controle de versão. Isso permitirá que você faça atualizações e refinamentos nos prompts existentes sem perder o histórico de modificações. Além disso, permite rastrear quais versões foram mais eficazes e por quê.

Incentive a equipe a fornecer feedback sobre a eficácia dos prompts e sugestões para melhorá-los. Isso cria um ciclo de melhoria contínua, onde os insights de todos podem ser incorporados para aperfeiçoar os prompts existentes e criar novos ainda mais eficazes.

Além do prompt em si, é vantajoso incluir alguma documentação que explique o contexto em que o prompt foi usado com sucesso, quais são seus limites e talvez alguns casos de uso. Isso torna muito mais fácil para os membros da equipe entenderem quando e como usar cada prompt disponível.

Criar e manter um banco de prompts organizado é um investimento que certamente pagará dividendos na forma de eficiência e eficácia ao longo do tempo. E lembre-se, a organização é a chave para

maximizar o valor desta poderosa ferramenta que é a engenharia de prompt.

5.3 Organizando contextos e padronizando respostas

Um modelo de linguagem possui um padrão genérico e descontextualizado. Portanto, é sempre desejável que as interações produzam respostas contextualizadas às nossas necessidades e orientadas aos nossos objetivos. Além disso, é igualmente desejável que as respostas sejam padronizadas. Isso vale tanto para interações com propósitos individuais quanto para interações no contexto de uma equipe.

Crie um banco de contextos

Além de um banco de prompts, outro recurso poderoso que equipes podem adotar é um banco de contextos específicos para cada projeto. Um banco de contextos vai além dos prompts ao incluir metadados adicionais, convenções e diretrizes que orientam a IA para fornecer respostas mais alinhadas com os objetivos e as práticas do projeto. Esses contextos poderão ser cruzados com os prompts para oferecer uma infinidade de possibilidades a um time de desenvolvimento.

Um banco de contextos pode ser um tipo de "manual de orientação" para a IA. Ele pode conter desde informações sobre o padrão de codificação que a equipe segue, até práticas recomendadas para a gestão de tarefas e *sprints*. Essa informação contextual permite que a IA entenda melhor o "como" e o "porquê" das tarefas, além do simples "o quê", levando a resultados mais refinados.

Para criar um banco de contextos eficiente:

1. **Identifique as áreas-chaves:** como primeiro passo, identifique os aspectos mais importantes do seu projeto que precisam de padronização e diretrizes claras;
2. **Documente as convenções:** inclua detalhes sobre convenções de codificação, práticas de revisão de código, e até mesmo padrões de nomenclatura de arquivos e variáveis;
3. **Inclua metadados:** metadados como *tags*, status de tarefas e prioridades podem ser adicionados para melhor filtrar e categorizar as respostas da IA.

A padronização da resposta da IA é outra dimensão crítica. Algumas ferramentas como o ChatGPT permitem configurar instruções de usuário personalizadas por padrão. As imagens a seguir demonstram como fazer tal configuração no ChatGPT, mas é importante frisar que são específicas para cada usuário e, por isso, precisam ser cuidadosamente gerenciadas pela liderança quando for um trabalho em equipe.

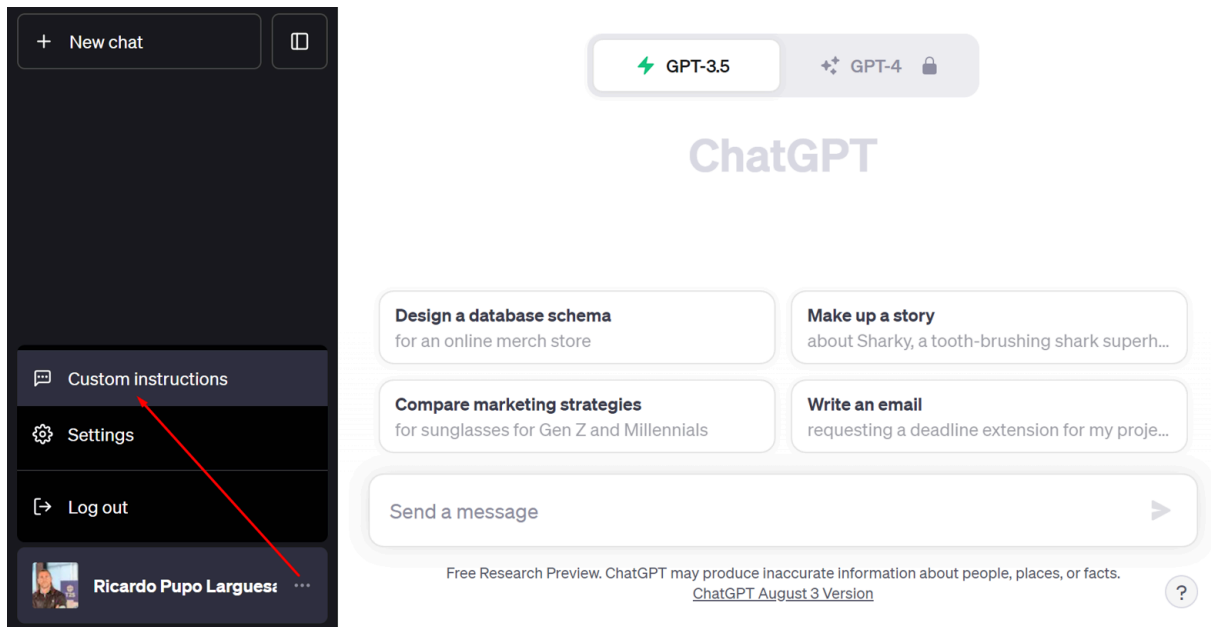


Figura 5.1: Acessando configurações personalizadas no ChatGPT

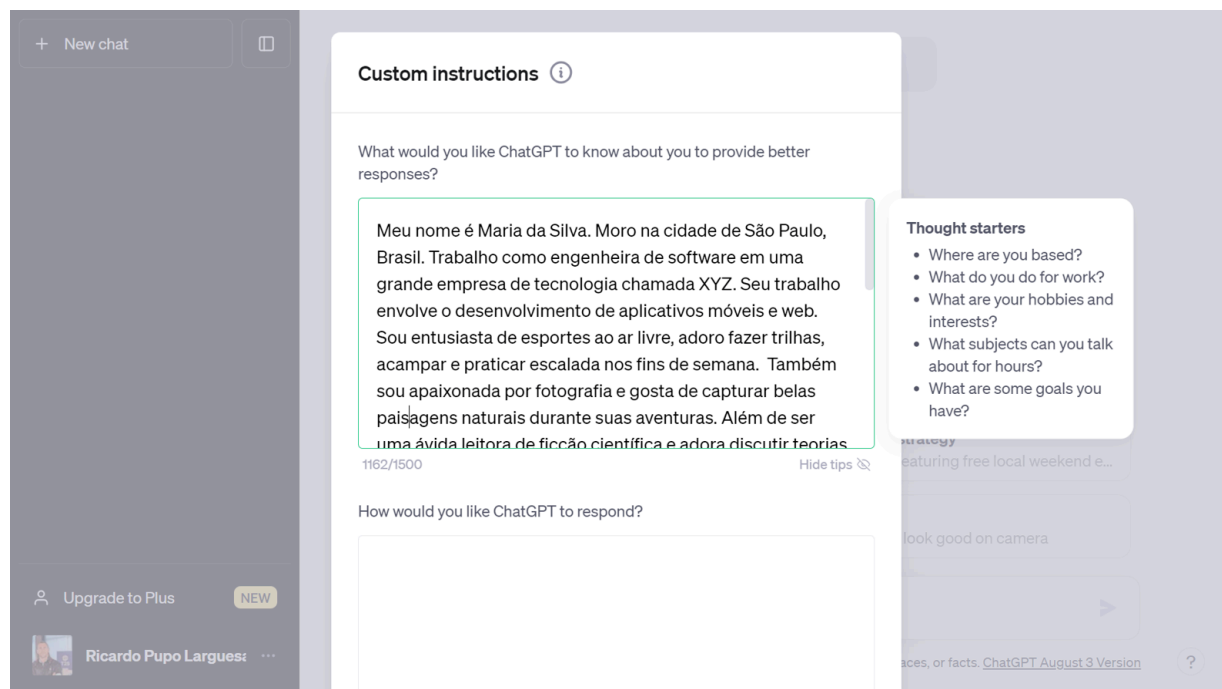


Figura 5.2: Exemplo de configuração padrão sobre o usuário no ChatGPT

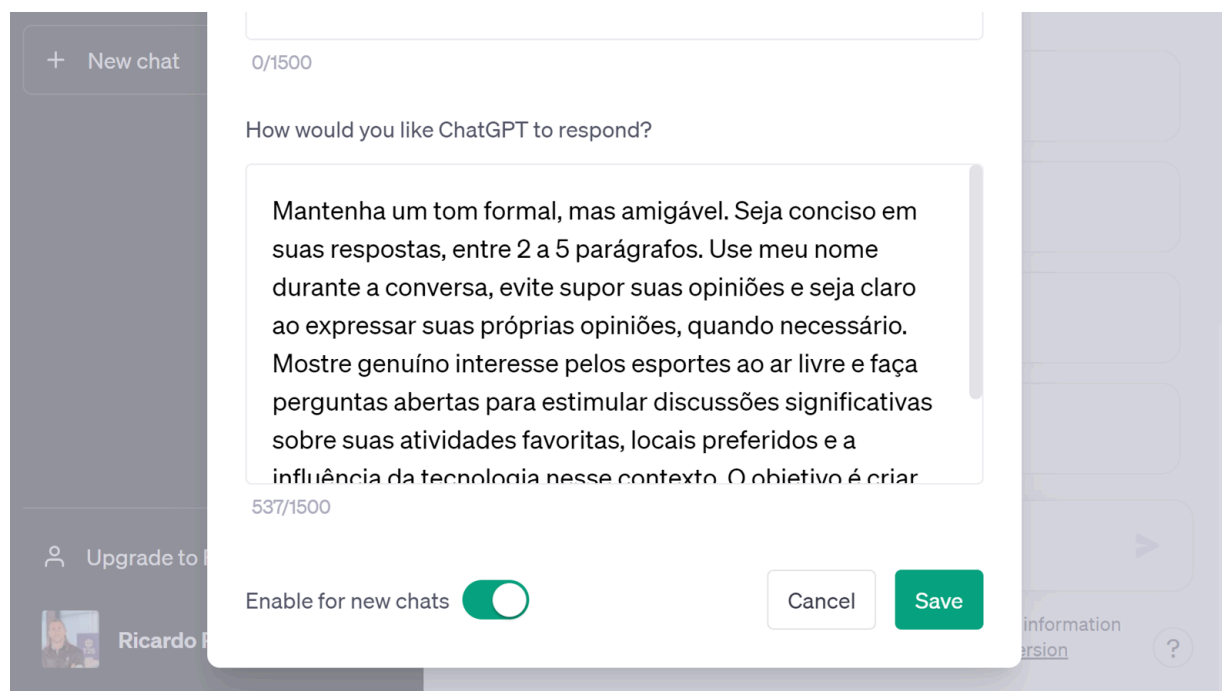


Figura 5.3: Exemplo de configuração padrão de respostas no ChatGPT

Já o GitHub Copilot, tendo acesso ao repositório, pode usar documentos que estabelecem padrões para todo o time, porém

apenas para codificação.

Crie um banco de padrão de respostas

Para padronizar respostas:

1. **Instruções consistentes:** estabeleça um conjunto padrão de instruções ou prompts para a IA, de modo que todos na equipe saibam o que esperar como saída;
2. **Verificação coletiva:** encoraje membros da equipe a revisar e ajustar as respostas da IA para se alinharem com as diretrizes do projeto;
3. **Iteração e feedback:** mantenha um ciclo de feedback para atualizar continuamente o banco de contextos e as instruções padrão, com base nas experiências da equipe.

Estratégias adicionais

- **Auditoria regular:** realize verificações regulares para assegurar que a IA está seguindo as diretrizes estabelecidas e atualize conforme necessário.
- **Adoção gradual:** comece pequeno, com um projeto piloto, e expanda a adoção do banco de contextos à medida que você percebe seu valor.
- **Ferramentas de colaboração:** utilize ferramentas que permitem a colaboração em tempo real no banco de contextos, para que atualizações possam ser feitas de forma mais eficaz.

Ao incorporar essas práticas, sua equipe não apenas otimizará a eficácia das ferramentas de IA, mas também criará um ambiente mais coeso e eficiente para o desenvolvimento de projetos.

5.4 Garantindo continuidade das interações

As ferramentas de Inteligência Artificial têm vários pontos fortes, mas também algumas limitações, que precisamos reconhecer para que seu uso seja otimizado. Uma dessas limitações é o tamanho da conversa, onde retomamos o conceito de tokens, que são blocos de texto que a IA processa por interação. Toda pergunta, contexto e resposta deve caber dentro do limite de tokens da IA, que pode variar de uma ferramenta para outra. Essa restrição pode ser um empecilho, especialmente em conversas mais longas ou quando se trabalha em equipe.

Compreendendo a limitação de tokens

Um token pode ser tão pequeno quanto um caractere ou tão grande quanto uma palavra. Em conversas mais longas ou instruções mais detalhadas, é fácil atingir o limite. Quando isso acontece, partes do contexto ou da resposta podem ser cortadas, resultando em informações incompletas ou até mesmo imprecisas. E o pior de tudo é que a IA não avisa: ela simplesmente passa a ignorar os textos anteriores, e é fácil perceber que ela parece estar ficando "esquecida" e "confusa".

Podemos adotar algumas estratégias para mitigar essa limitação:

- **Solicite resumos recorrentes:** uma maneira inteligente de contornar a limitação de tokens é solicitar resumos periódicos da IA. Isso permite que você mantenha o contexto sem sobrecarregar a IA. Resumos podem ser solicitados em intervalos regulares ou após conclusões significativas.
- **Fracione a informação:** se você tem uma questão complexa, considere dividi-la em partes menores. Em vez de fornecer todo o contexto de uma vez, forneça-o em etapas, permitindo que a IA responda de forma mais precisa.
- **Utilize códigos de referência:** para equipes, é útil ter um sistema de códigos de referência que todos possam entender. Por exemplo, um prompt específico para uma tarefa pode ser

- etiquetado com um código, tornando mais fácil para os membros da equipe retomar de onde pararam.
- **Monitoramento e revisão:** seja você um indivíduo ou parte de uma equipe, é importante revisar regularmente as interações com a IA para assegurar que os limites de tokens não estão comprometendo a qualidade das respostas. A limitação de tokens é uma restrição técnica que exige uma abordagem estratégica para garantir uma utilização eficaz da IA. Com as estratégias anteriores, você pode maximizar o potencial dessas ferramentas. Ao fazer isso, você mantém o fluxo da conversação e assegura a continuidade e a coesão das interações com a IA.
 - **Mantenha um registro externo:** uma prática recomendada para equipes é manter um registro externo das interações com a IA. Isso permite que qualquer membro da equipe retome o contexto anterior sem ter que perguntar à IA, economizando tokens valiosos. Para exemplificar, o ChatGPT permite exportar uma conversa que pode ser acessada através de um link público compartilhado, conforme mostra o exemplo a seguir:

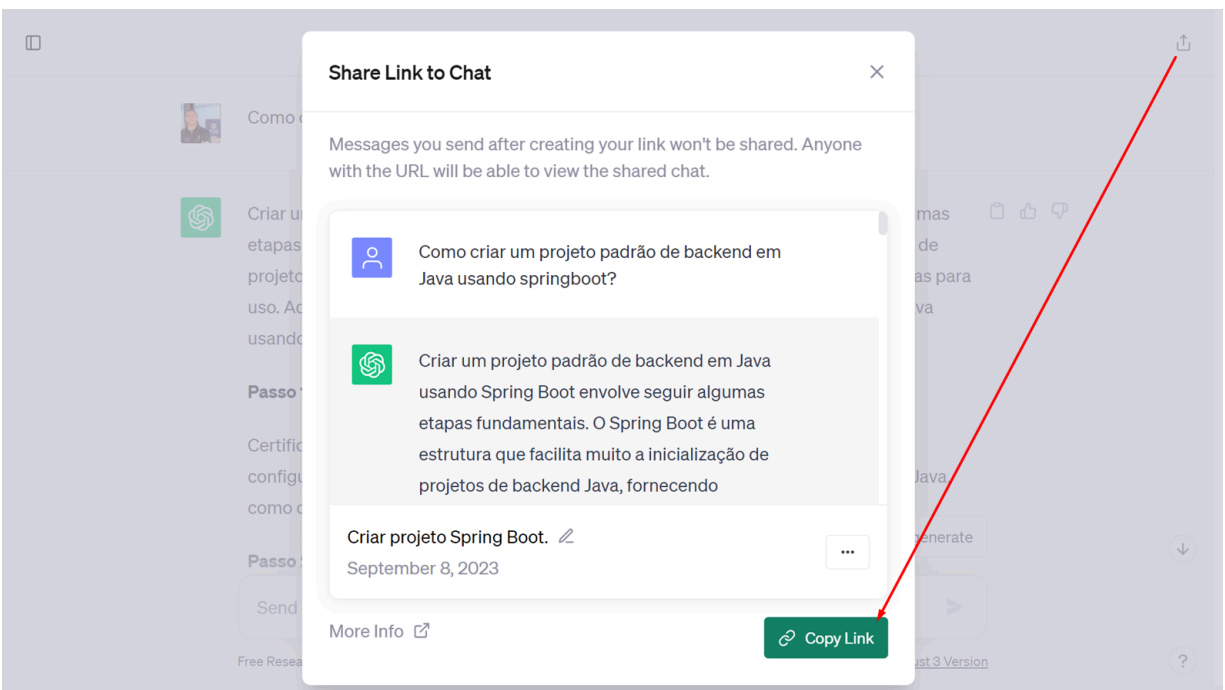


Figura 5.4: Botão para exportar uma conversa no ChatGPT por meio de um link público

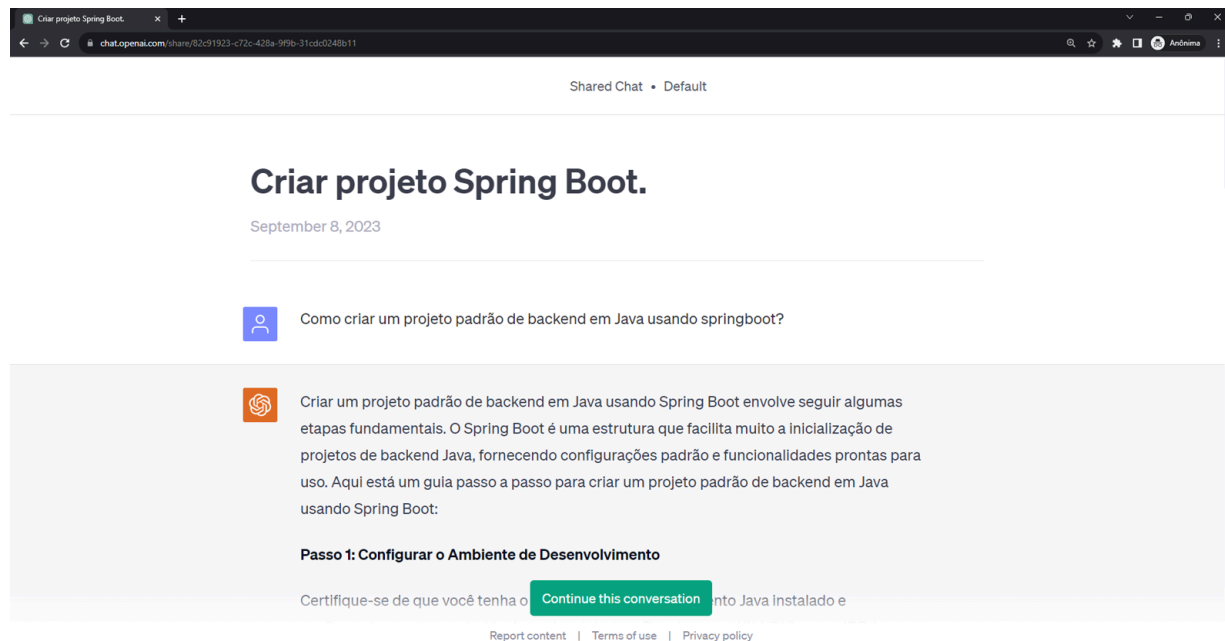


Figura 5.5: Conversa exportada no ChatGPT

5.5 Utilizando fontes externas

À medida que o campo da Engenharia de Prompt evolui, uma série de recursos e plataformas começa a surgir para atender às diversas necessidades dos profissionais da área. Essas plataformas não apenas disponibilizam um conjunto de prompts prontos para uso, mas também fornecem recursos para armazenar, compartilhar e até mesmo vender seus próprios prompts.

Vamos explorar algumas dessas plataformas e entender como elas podem ser um ativo valioso para qualquer pessoa programadora ou equipe de desenvolvimento.

PromptDB

Se você está buscando inspiração ou simplesmente quer economizar tempo, o **PromptDB** (<https://promptdb.ai>) pode ser um ponto de partida. Essa plataforma permite que você compartilhe prompts, modelos ou *embeddings* e os armazene para uso futuro. Ela visa a economia de tempo e recursos, já que também ajuda você a economizar dinheiro em APIs e custos de CPU. Com uma variedade de utilidades, do design visual ao desenvolvimento de projetos, o PromptDB serve como uma ferramenta versátil para múltiplas disciplinas.

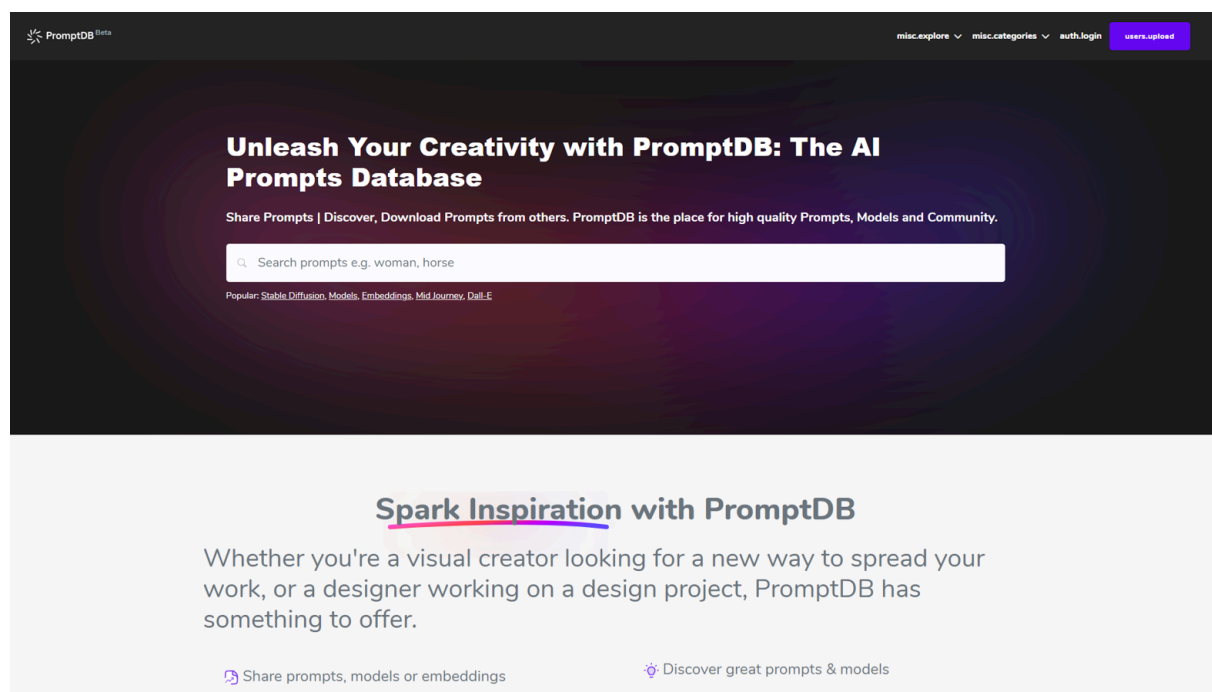


Figura 5.6: Site do PromptDB

The Prompt Index

Para quem deseja uma coletânea extensa e diversificada de prompts, o **The Prompt Index** (<https://www.thepromptindex.com>) oferece uma solução com mais de 500 prompts, que abrangem desde SEO e redação de conteúdo até programação. Além disso, o recurso de "favoritos" permite que você salve prompts que considerar úteis, para referência futura.

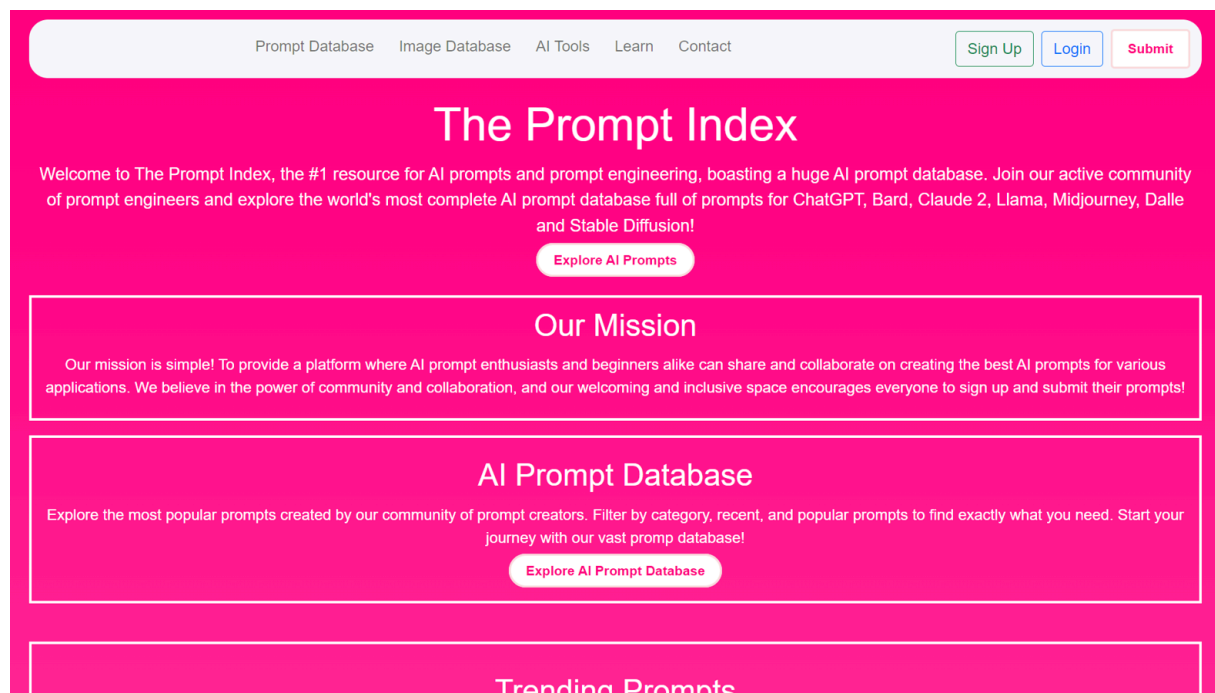


Figura 5.7: Site do The Prompt Index

PromptBase

Especialmente voltada para a utilização com modelos de IA, como ChatGPT e DALL·E, a PromptBase (<https://promptbase.com>) oferece um mercado com mais de 100 mil prompts. Nela, você pode não só encontrar prompts que produzam melhores resultados, mas também vender seus próprios prompts. É uma plataforma excelente para quem busca otimizar seus custos e tempo, além de proporcionar uma maneira de monetizar seus próprios insights sobre engenharia de prompt.

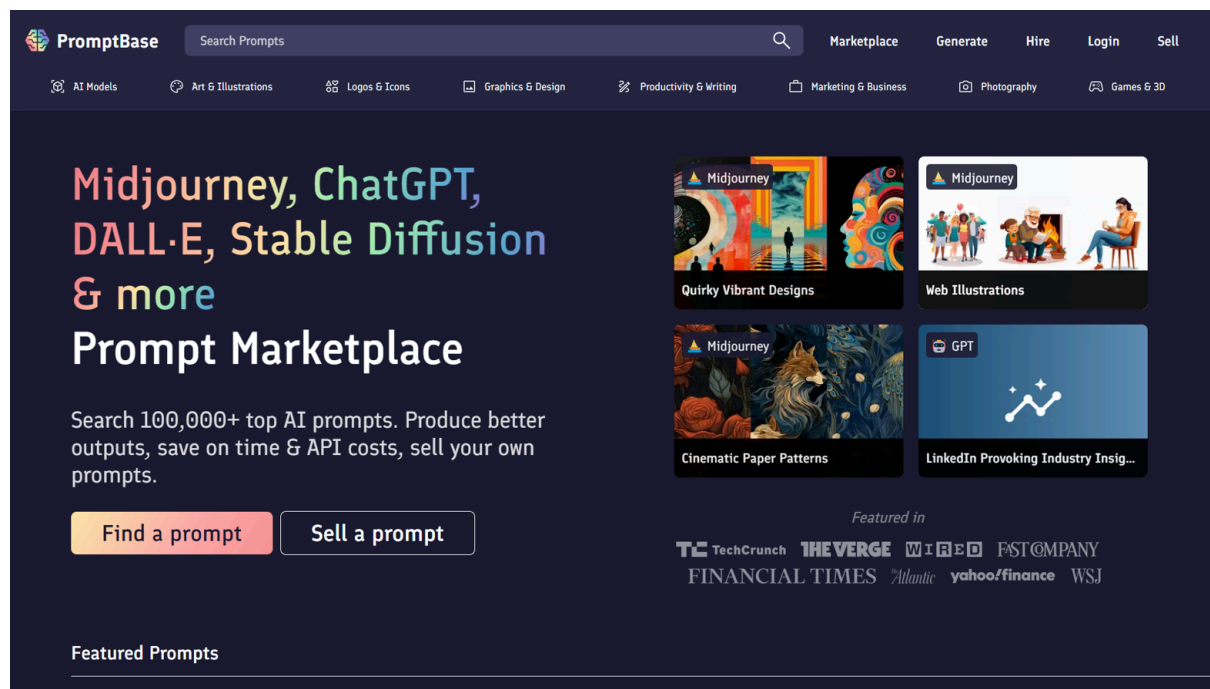


Figura 5.8: Site do PromptBase

Em resumo, essas plataformas externas oferecem uma rica tapeçaria de recursos que podem complementar e enriquecer seus próprios esforços em engenharia de prompt. Há várias delas, e surgem novas a cada dia. Seja você um indivíduo ou parte de uma equipe maior, essas fontes externas podem servir como uma extensão valiosa de seu arsenal de ferramentas, permitindo que você alcance resultados mais eficazes e eficientes.

Por fim, é importante não apenas entender os componentes básicos, mas também aprofundar-se em técnicas avançadas e melhores práticas que podem elevar a eficácia dos seus prompts a um novo patamar.

5.6 Abordagem multilíngue e cultural

A eficácia de um prompt também pode ser afetada por nuances culturais e linguísticas. Ao criar prompts, sempre leve em conta as

especificidades culturais, sociais e até mesmo legais do contexto em que as respostas serão usadas para que possam influenciar a forma como a informação é recebida e interpretada. Por exemplo, saudações informais podem ser bem recebidas em culturas ocidentais, mas vistas como desrespeitosas em contextos mais formais.

- **Emojis e símbolos:** em plataformas online, o uso de emojis pode ser visto como simpático e moderno em algumas culturas, mas como não profissional ou até irreverente em outras;
- **Humor:** a depender da cultura, piadas e humor podem não ser bem-vindos em uma configuração empresarial ou educacional. O que é engraçado em um país pode ser ofensivo em outro;
- **Gênero e linguagem:** em línguas como o espanhol ou o francês, que têm gêneros gramaticais, usar uma linguagem neutra em gênero pode ser complicado, mas é cada vez mais incentivado para inclusividade;
- **Legalidade:** em algumas jurisdições, coletar certos tipos de informações pode ser ilegal sem consentimento explícito. Portanto, o formato do prompt deve estar em conformidade com as leis locais sobre privacidade de dados, como veremos mais a fundo adiante;
- **Religiosidade:** a forma como questões religiosas são abordadas pode variar grandemente de uma cultura para outra, afetando como um prompt sobre tópicos sensíveis deve ser formulado para obter resultados apropriados;
- **Gestos e símbolos:** é importante lembrar que gestos como "joia" ou "polegar para cima" podem ter diferentes conotações em diferentes culturas;
- **Saudações:** em países árabes, uma saudação religiosa como *As-salamu alaykum* é comum, mas em ambientes seculares de outros países, pode não ser apropriado.

O próprio modelo de linguagem pode ajudá-lo a evitar essas questões, mas é preciso que você as contextualize nos prompts.

5.7 Considerações éticas e de privacidade

Com a crescente preocupação sobre ética e privacidade em tecnologia, é preciso considerar questões como viés e coleta de dados. O consentimento de outras pessoas ou empresas para coletar e usar os dados delas pode ser um elemento importante a ser integrado nos prompts, uma vez que ficarão nos históricos das IAs e poderão até ser difundidos por links públicos, como vimos anteriormente neste capítulo. Outros exemplos de questões éticas no uso de modelos de linguagem incluem:

- **Viés:** modelos de linguagem são treinados em grandes quantidades de dados, que podem refletir vieses presentes na sociedade. Isso pode levar a resultados tendenciosos, como a produção de linguagem discriminatória ou ofensiva;
- **Autonomia:** modelos de linguagem podem ser usados para criar agentes autônomos que são capazes de tomar decisões e agir de forma independente. Isso levanta questões sobre a autonomia desses agentes e sobre suas responsabilidades;
- **Responsabilidade:** quem é responsável pelos resultados gerados por modelos de linguagem? Os desenvolvedores dos modelos, os usuários que os utilizam ou as empresas que os comercializam?
- **Impacto social:** os modelos de linguagem podem ter um impacto significativo na sociedade, por exemplo, na forma como nos comunicamos, na forma como consumimos informações e na forma como tomamos decisões. É importante considerar os potenciais impactos positivos e negativos desse impacto social.

É importante estar atento às tendências emergentes em IA e PLN. Mantendo-se atualizado, você poderá adotar novas técnicas e abordagens que podem aprimorar ainda mais seus prompts, ainda mais em se tratando de uma nova tecnologia que muda em uma velocidade nunca antes vista na história.

5.8 Considerações legais sobre sigilo e confidencialidade

É importante frisar que há considerações legais sérias ao compartilhar dados nas interações com modelos de linguagem.

Quando se trata de dados pessoais, leva-se em consideração a Lei Geral de Proteção de Dados (LGPD). Essa legislação, inspirada no Regulamento Geral sobre a Proteção de Dados (GDPR) da União Europeia, estabelece regras estritas sobre o tratamento de dados pessoais, reforçando a necessidade de consentimento e transparência. Mas adivinha só? Você dá consentimento total ao entrar nessas plataformas.

Dados pessoais são por conta e risco apenas do usuário, mas e sobre o compartilhamento de dados empresariais? No Brasil, o uso inadequado de dados empresariais pode trazer implicações legais sérias. Por exemplo, o artigo 482, alínea *g*, da CLT permite a demissão por justa causa de empregados que violem o sigilo da empresa. Também na esfera penal, o artigo 325 do Código Penal prevê punições para quem revela informações sigilosas obtidas em razão de cargo. E no âmbito da propriedade industrial, a Lei nº 9.279/96 define como concorrência desleal o uso não autorizado de dados confidenciais. Além disso, geralmente as empresas possuem códigos de conduta, ética e *compliance* que precisam ser analisados

antes de sair por aí compartilhando dados da empresa em que trabalha.

Para salientar, logo no primeiro semestre desde o lançamento do ChatGPT, foi noticiado publicamente que a Samsung banuiu o uso de ferramentas de IA generativa entre seus funcionários após um vazamento de código confidencial. A decisão foi comunicada via memorando e faz parte de uma abordagem mais cautelosa da empresa quanto à segurança dos dados, já que há receio de que as informações possam ser armazenadas em servidores externos e potencialmente divulgadas. A medida também segue preocupações semelhantes levantadas por outras grandes empresas e nações.

Por outro lado, não apenas a OpenAI, empresa proprietária do ChatGPT, mas outras empresas do setor afirmam que adotam medidas para proteger os dados dos usuários, como criptografia, conformidade com leis de privacidade e controle de acesso. E convém salientar que os termos de uso acrescentam níveis adicionais de segurança quando são pagos.

Concluindo, é muito importante que os usuários tomem medidas para proteger seus dados pessoais e empresariais. Algumas dicas incluem:

- **Não forneça informações confidenciais:** se você estiver usando um modelo de linguagem para atividades relacionadas ao trabalho, formalize com seu time e sua liderança e evite fornecer informações confidenciais, como código-fonte, dados de clientes ou informações financeiras;
- **Siga as melhores práticas de segurança:** use senhas fortes e não compartilhe sua conta com outras pessoas;
- **Fique atento a malwares e ataques cibernéticos:** mantenha seu software atualizado e use um software antivírus confiável;

- **Estude os termos de uso** de cada ferramenta e pesquise sobre vazamentos e outras repercussões negativas.

Ao seguir essas dicas, os usuários podem ajudar a proteger seus dados e evitar problemas com sigilo.

CAPÍTULO 6

IA e o programador moderno

"Programadores são o cérebro e os computadores, o músculo" —
Everson Silva

Nos próximos capítulos, abordaremos meticulosamente diversas formas de tirar proveito dos modelos de linguagem que estão surgindo para potencializar sua competência e sua eficácia enquanto analista de sistemas. Isso elevará a sua produtividade a patamares sem precedentes e agregará muito mais valor ao seu trabalho, não apenas com agilidade, mas também com abrangência e profundidade.

6.1 O que a IA pode fazer por você

Um(a) analista de sistemas pode usar não apenas o ChatGPT, mas também outras ferramentas de IA para aumentar a sua produtividade no trabalho de várias maneiras:

- **Geração de código:** modelos de linguagem podem ajudar a gerar trechos de código ou sugerir soluções para problemas de programação específicos, economizando tempo na pesquisa e desenvolvimento;
- **Depuração e otimização de código:** ao analisar e identificar padrões de código problemáticos, as ferramentas de IA podem sugerir otimizações e correções, melhorando a qualidade e a eficiência do código;

- **Revisão de código:** a IA pode auxiliar na revisão de código, identificando inconsistências, erros e violações de diretrizes de estilo, garantindo a qualidade e a manutenção do código ao longo do tempo;
- **Documentação:** as ferramentas de IA podem ajudar na criação e manutenção de documentação técnica, gerando descrições, exemplos e explicações a partir do código existente;
- **Gerenciamento de projetos:** é possível usar modelos de linguagem para ajudar em tarefas de gestão de projetos, identificando riscos, estimando esforços, priorizando tarefas e sugerindo soluções para possíveis problemas;
- **Análise de requisitos:** a IA pode ser usada para analisar e entender os requisitos do projeto, identificando lacunas, ambiguidades e inconsistências, ajudando na elaboração de especificações mais claras e precisas;
- **Assistência virtual:** modelos de linguagem podem ser usados como assistentes virtuais, ajudando a responder perguntas, fornecer informações e realizar tarefas rotineiras, permitindo que analistas de sistemas se concentrem em tarefas mais complexas e de maior valor;
- **Aprendizado contínuo:** a IA pode ser usada para fornecer recomendações de recursos de aprendizado, como artigos, cursos e tutoriais, ajudando analistas de sistemas a se manterem atualizados com as últimas tendências e tecnologias;
- **Análise de dados:** modelos de linguagem podem ser usados para analisar e extrair informações valiosas a partir de informações diversas, sintetizando e resumindo textos, auxiliando na tomada de decisões informadas e na identificação de oportunidades e tendências;
- **Colaboração:** as ferramentas de IA podem ajudar a facilitar a comunicação e a colaboração entre membros da equipe,

fornecendo tradução automática, geração de resumos de reuniões e identificação de tópicos relevantes para discussão;

- **Criação de artes gráficas:** tanto modelos de geração de imagem, como Midjourney ou DALL-E, quanto modelos de linguagem podem ser usados para obter artes originais. Imagens ilustrativas, logotipos, cliparts e desenhos diversos podem ser gerados por ferramentas de Inteligência Artificial de geração de imagens, enquanto modelos de linguagem podem gerar artes através de linguagens de marcação específicas, como veremos mais à frente neste livro.

6.2 O que a IA NÃO pode fazer por você

Ao incorporar essas ferramentas e técnicas de IA no fluxo de trabalho, você pode aumentar significativamente a produtividade e a eficiência no desenvolvimento e manutenção de projetos de software. Mas ainda que a inteligência artificial tenha avançado rapidamente, ainda há áreas em que ela não consegue substituir completamente o trabalho humano, especialmente no campo do desenvolvimento de sistemas. Aqui estão algumas das limitações da IA para analistas de sistemas:

- **Abstração de problemas e soluções:** a IA se beneficia de prompts cuidadosamente elaborados para oferecer seu melhor desempenho, entretanto, o trabalho de um(a) analista de sistemas envolve identificar as necessidades reais dos usuários, mesmo que não sejam imediatamente aparentes. É responsabilidade do analista fazer as perguntas certas, analisar o contexto, documentos, processos, outros sistemas e assim por diante, a fim de projetar uma solução eficaz que resolva os problemas dos usuários — e os modelos de linguagem não refletem ou questionam os prompts;

- **Compreensão completa do contexto:** a IA pode ter dificuldade em entender completamente o contexto de um problema específico, especialmente quando se trata de nuances e detalhes que requerem conhecimento especializado ou experiência prática. Além disso, já vimos neste livro que há limitação no tamanho de um prompt, o que limita o tamanho de um contexto;
- **Criatividade e inovação:** a IA pode gerar soluções com base no que já aprendeu, mas pode ter dificuldade em criar soluções verdadeiramente inovadoras ou "pensar fora da caixa" como humanos podem fazer;
- **Comunicação efetiva com as partes interessadas:** a IA pode ter dificuldade em compreender completamente as necessidades e preocupações das partes interessadas e em comunicar de forma eficaz com elas, especialmente quando se trata de questões emocionais ou complexas;
- **Tomada de decisões éticas e responsáveis:** a IA pode não estar preparada para tomar decisões que envolvam considerações éticas ou responsabilidade social, e pode ser influenciada por vieses existentes nos dados de treinamento;
- **Adaptação a mudanças e incertezas:** a IA pode ter dificuldade em se adaptar rapidamente a mudanças no ambiente de projeto, especialmente quando essas mudanças envolvem situações imprevistas ou complexas.
- **Conhecimento tácito e habilidades interpessoais:** a IA pode ter dificuldade em adquirir conhecimento tácito, como habilidades interpessoais e de liderança, que são essenciais para o sucesso do trabalho em equipe e a eficácia de analistas de sistemas;
- **Supervisão e garantia de qualidade:** a IA pode gerar resultados rapidamente, mas ainda é necessário que analistas de sistemas supervisionem e verifiquem a qualidade das

soluções geradas pela IA, garantindo que elas atendam aos requisitos e padrões esperados.

Embora a IA possa aumentar significativamente a eficiência e a produtividade de analistas de sistemas, ela não pode substituir completamente o conhecimento especializado, a criatividade e a capacidade humana de tomar decisões morais e éticas. Analistas de sistemas devem aprender a trabalhar em conjunto com as ferramentas de IA, aproveitando suas vantagens e reconhecendo suas limitações, para obter os melhores resultados possíveis.

6.3 O perfil do novo analista de sistemas

Paulo Silveira, CEO e cofundador da Alura, frequentemente expressa sua visão acerca da generalização do profissional de TI, que ele denomina de maneira lúdica como *dev em T* — uma alusão ao conceito de *generics* presente nas linguagens de programação orientadas a objetos. Essa perspectiva também é compartilhada por outros profissionais da área. Atualmente, observamos equipes extensas compostas por indivíduos com funções altamente especializadas. No entanto, em um futuro próximo, essa superespecialização pode dar lugar a uma maior generalização das habilidades dos profissionais de TI. Dessa forma, espera-se que os profissionais desenvolvam competências mais amplas, permitindo-lhes desempenhar múltiplas funções e se adaptar às crescentes demandas do mercado.

À medida que a IA se torna cada vez mais presente no campo da Análise de Sistemas, é esperado que analistas de sistemas se adaptem e desenvolvam habilidades que os tornem mais produtivos ao trabalhar com essas tecnologias. Algumas mudanças no perfil de uma pessoa analista de sistemas podem incluir:

- **Domínio de ferramentas e plataformas de IA:** analistas de sistemas devem estar familiarizados com as ferramentas e plataformas de IA disponíveis no mercado, como o ChatGPT, para poderem aplicá-las de maneira eficiente em suas atividades diárias;
- **Habilidades de Engenharia de Prompt:** aprender a criar prompts eficientes e bem formulados é fundamental para extrair o máximo de valor dos modelos de linguagem e IA em geral;
- **Pensamento crítico e adaptabilidade:** analistas de sistemas devem ser capazes de avaliar criticamente as soluções geradas pela IA e adaptar-se às novas tecnologias e abordagens conforme elas evoluem;
- **Conhecimento interdisciplinar:** com a crescente integração da IA em diversos setores, analistas de sistemas podem se beneficiar ao ampliar seus conhecimentos em áreas relacionadas, como Ciência de Dados, Aprendizado de Máquina e Estatística;
- **Colaboração e comunicação eficiente:** analistas de sistemas devem ser capazes de trabalhar em conjunto com colegas e equipes multidisciplinares para integrar soluções baseadas em IA nos processos de negócios existentes;
- **Foco em soluções de alto valor agregado:** com a IA assumindo tarefas rotineiras e de baixo valor agregado, analistas de sistemas devem se concentrar em atividades que gerem maior valor, como a concepção e implementação de soluções estratégicas e inovadoras;
- **Compreensão de negócios:** a pessoa analista de sistemas deve entender não apenas a tecnologia, mas também o contexto de negócios em que ela será aplicada. Isso permite que ele ou ela identifique oportunidades para aplicar IA de maneiras que se alinhem aos objetivos estratégicos da organização;

- **Resiliência e resolução de problemas:** à medida que a IA assume tarefas mais complexas, os problemas que surgem também se tornam mais desafiadores. A capacidade de manter a calma sob pressão e encontrar soluções inovadoras passa a ser necessária;
- **Conhecimento de regulamentações e conformidade:** dada a natureza sensível dos dados e a complexidade dos algoritmos de IA, é importante que analistas estejam atualizados sobre as leis e regulamentos que afetam seu campo, porque os modelos de linguagem não estarão;
- **Habilidades de *storytelling* e apresentação:** conforme analistas de sistemas se tornam mais envolvidos em decisões estratégicas, a capacidade de comunicar complexidades técnicas de forma clara para *stakeholders* não técnicos torna-se importante;
- **Automatização de tarefas rotineiras:** com a IA assumindo tarefas mais básicas, analistas devem também aprender a automatizar suas próprias tarefas rotineiras para se concentrarem em desafios mais significativos;
- **Networking profissional:** a IA é um campo em rápido desenvolvimento, e manter uma rede de contatos atualizada pode oferecer insights valiosos e oportunidades de colaboração;
- **Aprendizagem contínua:** a IA e as tecnologias relacionadas estão em constante evolução. Portanto, analistas de sistemas devem se manter atualizados sobre as últimas tendências e desenvolvimentos, participando de cursos, workshops e conferências regularmente.

Repare o quanto a mentalidade deve ser trabalhada para este novo ambiente profissional em função das novas possibilidades. Ao incorporar essas mudanças em seu perfil profissional, um(a) analista de sistemas estará mais bem preparado(a) para aproveitar as

oportunidades oferecidas pela IA e se tornar mais produtivo(a) no trabalho.

Vejamos nos próximos capítulos exemplos mais práticos de prompts para pessoas programadoras.

CAPÍTULO 7

Prompts de apoio à modelagem

"Computadores nunca venceram ninguém, programadores sim"
— Everson Silva

Um(a) analista de sistemas pode tirar proveito de modelos de linguagem para agilizar a modelagem de um novo sistema. Utilizando prompts bem elaborados, o modelo pode gerar esboços de requisitos, organizar tarefas, estimar esforço e até mesmo gerar diagramas UML. Isso não apenas acelera o processo de planejamento, mas também oferece uma perspectiva adicional que pode revelar oportunidades de otimização ou soluções inovadoras, permitindo que a pessoa analista se concentre em aspectos mais complexos e estratégicos do projeto.

7.1 Especificando requisitos funcionais

Um analista de sistemas pode utilizar a IA para auxiliar na geração de requisitos funcionais de um sistema, seguindo algumas etapas (exemplificaremos cada uma nos tópicos deste capítulo):

1. **Coleta de informações:** pode-se usar ferramentas de IA para extrair informações relevantes a partir de várias fontes, como documentos, entrevistas com stakeholders, e-mails, sistemas existentes e outras comunicações. A IA pode ajudar a identificar e resumir informações-chave e reconhecer padrões e temas comuns;
2. **Análise de texto:** a IA pode ser aplicada para identificar termos e conceitos importantes relacionados ao domínio do

sistema. Isso pode incluir entidades, tópicos e sentimentos expressos pelos *stakeholders*, facilitando a compreensão das necessidades e expectativas do usuário;

3. **Identificação de requisitos:** com base na análise realizada, a IA pode auxiliar na identificação de possíveis requisitos funcionais, correlacionando informações extraídas de várias fontes e sugerindo funcionalidades e comportamentos esperados do sistema;
4. **Geração de rascunhos de requisitos:** a IA pode ser usada para gerar rascunhos de requisitos funcionais com base nos dados coletados e analisados. O analista de sistemas pode então revisar e ajustar esses rascunhos conforme necessário, garantindo que os requisitos estejam claros, completos e precisos;
5. **Validação e refinamento:** pode-se utilizar a IA para validar os requisitos funcionais gerados, comparando-os com as expectativas dos stakeholders e verificando sua consistência e coerência. A IA pode sugerir melhorias nos requisitos, ajudando a pessoa analista a refinar e aprimorar os requisitos conforme necessário.

Cabe salientar que o usuário deve sempre estar ciente das considerações éticas e legais sobre o uso de dados pessoais e empresariais, assunto abordado no capítulo 5. Vale ressaltar também que, embora a IA possa auxiliar significativamente no processo de geração de requisitos funcionais, ela não substitui a experiência e a intuição da pessoa analista de sistemas. O papel do analista é fundamental para garantir que os requisitos gerados atendam às necessidades do usuário e estejam alinhados com os objetivos do projeto. Mas você pode pedir para que um modelo de linguagem lhe ajude a elaborar um questionário para apoiar o levantamento de requisitos, como no exemplo a seguir:

Você: *Vou iniciar um projeto de desenvolvimento de um sistema de controle de um estacionamento de veículos. Preciso identificar os*

requisitos desse sistema entrevistando os futuros usuários. Que perguntas você sugere para um questionário que me ajude a identificar esses requisitos?

IA: Entrevistar os futuros usuários é uma etapa fundamental para identificar os requisitos de um sistema de controle de estacionamento de veículos. Aqui estão algumas perguntas que você pode incluir em seu questionário para ajudar a obter informações valiosas:

1. PERFIL DO USUÁRIO:

- Qual é o seu papel no estacionamento (por exemplo, proprietário, gerente, atendente)?
- Com que frequência você utiliza o sistema de controle de estacionamento?
- Quais são as suas principais responsabilidades relacionadas ao estacionamento?

2. FUNCIONALIDADES NECESSÁRIAS:

- Quais são as funcionalidades mais importantes que você espera desse sistema?
- Como você gostaria de registrar a entrada e saída de veículos?
- Você precisa de um sistema de reservas de vagas? Se sim, como ele deve funcionar?
- Há alguma funcionalidade específica que você acredita ser única para o seu estacionamento?

3. CONTROLE DE ACESSO E SEGURANÇA:

- Como você controla o acesso de veículos ao estacionamento atualmente?
- Quais medidas de segurança são essenciais para o seu estacionamento?
- Você gostaria de implementar um sistema de câmeras de vigilância integrado ao sistema de controle?

4. PREÇOS E COBRANÇA:

- Como você atualmente define os preços de estacionamento?

- Como você gostaria que o sistema calculasse e registrasse os custos de estacionamento?
- Existe a necessidade de oferecer descontos ou promoções especiais?

5. RELATÓRIOS E ANÁLISES:

- Quais tipos de relatórios e análises você gostaria de ver a partir do sistema?
- Que informações específicas são importantes para o gerenciamento do estacionamento?
- Como você gostaria que esses relatórios fossem disponibilizados (por exemplo, impressos, por e-mail, em tempo real)?

6. INTEGRAÇÕES COM OUTROS SISTEMAS:

- Existe a necessidade de integrar o sistema de controle de estacionamento com outros sistemas (por exemplo, sistemas de pagamento, aplicativos móveis, sistemas de controle de acesso)?

7. FACILIDADE DE USO E TREINAMENTO:

- Quão familiarizados você e sua equipe são com sistemas similares?
- Quão importante é a facilidade de uso do sistema?
- Está disposto a investir tempo em treinamento para usar o sistema de forma eficaz?

8. SUPORTE TÉCNICO E MANUTENÇÃO:

- Que tipo de suporte técnico você espera do fornecedor do sistema?
- Como você gostaria de lidar com atualizações e manutenção do sistema?

9. FEEDBACK ADICIONAL:

- Existe alguma outra informação ou requisito importante que você gostaria de compartilhar?

Pode ser que você queira um ponto de partida para que a elicitação de requisitos junto aos usuários seja mais produtiva. Então você pode solicitar um conjunto de requisitos funcionais ao modelo de linguagem, como mostra o modelo a seguir:

Você: *Escreva uma lista de requisitos funcionais para controlar a operação de um estacionamento de veículos.*

IA: Claro, aqui está uma lista de requisitos funcionais que podem ser necessários para controlar a operação de um estacionamento de veículos. Lembre-se de que esses requisitos podem variar dependendo do tamanho e da complexidade do estacionamento, das necessidades dos usuários e dos recursos disponíveis. Esta lista serve como um ponto de partida:

1. REGISTRO DE VEÍCULOS:

- O sistema deve permitir o registro de veículos que entram no estacionamento;
- Deve ser possível associar informações como a placa do veículo, a data e hora de entrada e a vaga atribuída.

2. CONTROLE DE ACESSO:

- O sistema deve controlar o acesso de veículos à entrada e à saída do estacionamento;
- Pode ser necessário suportar diferentes métodos de controle, como cartões de acesso, leitura de placas, código de barras ou RFID.

3. RESERVAS DE VAGAS:

- Deve ser possível fazer reservas de vagas com antecedência, se aplicável;
- Os usuários devem poder escolher datas e horários específicos para suas reservas.

4. COBRANÇA E PAGAMENTO:

- O sistema deve calcular as taxas de estacionamento com base na duração da permanência;
- Deve suportar diferentes métodos de pagamento, como dinheiro, cartão de crédito, pagamento móvel ou pagamento online;
- Deve ser capaz de gerar recibos ou comprovantes de pagamento.

5. GESTÃO DE VAGAS:

- O sistema deve manter o controle em tempo real das vagas disponíveis e ocupadas;
- Deve permitir o bloqueio de vagas reservadas e a identificação de vagas para portadores de necessidades especiais.

6. RELATÓRIOS E ANÁLISES:

- Deve gerar relatórios detalhados, como registros de entrada e saída, receita diária, ocupação média e relatórios de eventos especiais;
- Deve permitir a exportação de dados para análises adicionais.

7. INTEGRAÇÕES:

- Deve ser capaz de integrar-se a outros sistemas, como sistemas de controle de acesso, sistemas de pagamento, aplicativos móveis e sistemas de segurança.

8. SEGURANÇA:

- O sistema deve garantir a segurança dos dados dos clientes e do estacionamento;
- Deve registrar qualquer tentativa de acesso não autorizado ou atividade suspeita.

9. SUPORTE AO CLIENTE:

- Deve fornecer um sistema de suporte ao cliente para lidar com dúvidas, reclamações e solicitações de assistência.

10. FACILIDADE DE USO:

- Deve ser intuitivo e fácil de usar para os operadores do estacionamento;
- Deve oferecer treinamento adequado para os funcionários que operarão o sistema.

11. ATUALIZAÇÕES E MANUTENÇÃO:

- Deve permitir a aplicação de atualizações de software e manutenção sem interromper as operações normais do estacionamento.

Lembre-se de que esses requisitos funcionais são apenas um ponto de partida. É importante conduzir uma análise detalhada dos requisitos com os usuários e todas as partes interessadas para garantir que o sistema atenda às necessidades específicas do seu estacionamento.

Repare que, para quem não tem nada no início, parece um bom ponto de partida obtido em poucos segundos. E lembre-se de que você pode estabelecer uma conversa contínua com a IA, pedindo revisões dos requisitos solicitados com base em pequenas alterações. Como exemplo, em continuação à conversa do exemplo anterior, observe a seguir como podemos interagir com a IA para aperfeiçoar o conjunto de requisitos funcionais sugeridos por ela a partir da mesma conversa:

Você: *Reescreva, considerando que haverá segmentação de veículos por motocicleta, automóvel e caminhão, tanto sem reboque quanto com reboque.*

IA: Certamente, aqui estão os requisitos funcionais considerando a segmentação de veículos por motocicleta, automóvel e caminhão, tanto sem reboque quanto com reboque:

1. REGISTRO DE VEÍCULOS:

- O sistema deve permitir o registro de motocicletas, automóveis e caminhões, bem como a identificação de veículos com ou sem reboque, que entram no estacionamento;
- Deve ser possível associar informações como a placa do veículo, o tipo de veículo, a presença de reboque, a data e hora de entrada e a vaga atribuída.

2. CONTROLE DE ACESSO:

- O sistema deve controlar o acesso de motocicletas, automóveis e caminhões, tanto com reboque quanto sem reboque, à entrada e à saída do estacionamento;
- Pode ser necessário suportar diferentes métodos de controle, como cartões de acesso, leitura de placas, código de barras ou RFID.

3. RESERVAS DE VAGAS:

- Deve ser possível fazer reservas de vagas com antecedência, especificando o tipo de veículo (motocicleta, automóvel ou caminhão) e a presença de reboque;
- Os usuários devem poder escolher datas e horários específicos para suas reservas, levando em consideração o tipo de veículo.

4. COBRANÇA E PAGAMENTO:

- O sistema deve calcular as taxas de estacionamento com base na duração da permanência e no tipo de veículo (motocicleta, automóvel ou caminhão), incluindo a presença de reboque;

- Deve suportar diferentes métodos de pagamento, como dinheiro, cartão de crédito, pagamento móvel ou pagamento online;
- Deve ser capaz de gerar recibos ou comprovantes de pagamento detalhando o tipo de veículo e a presença de reboque.

5. GESTÃO DE VAGAS:

- O sistema deve manter o controle em tempo real das vagas disponíveis e ocupadas para cada categoria de veículo (motocicleta, automóvel ou caminhão), considerando a presença de reboque;
- Deve permitir o bloqueio de vagas reservadas e a identificação de vagas para portadores de necessidades especiais.

6. RELATÓRIOS E ANÁLISES:

- Deve gerar relatórios detalhados, como registros de entrada e saída por categoria de veículo, receita diária por tipo de veículo, ocupação média por categoria e relatórios de eventos especiais;
- Deve permitir a exportação de dados para análises adicionais, considerando a segmentação de veículos.

7. INTEGRAÇÕES:

- Deve ser capaz de integrar-se a outros sistemas, como sistemas de controle de acesso, sistemas de pagamento, aplicativos móveis e sistemas de segurança, levando em consideração a segmentação de veículos.

8. SEGURANÇA:

- O sistema deve garantir a segurança dos dados dos clientes e do estacionamento, considerando as diferentes categorias de veículos e a presença de reboque;

- Deve registrar qualquer tentativa de acesso não autorizado ou atividade suspeita, levando em consideração a segmentação de veículos.

9. SUPORTE AO CLIENTE:

- Deve fornecer um sistema de suporte ao cliente para lidar com dúvidas, reclamações e solicitações de assistência relacionadas às diferentes categorias de veículos.

10. FACILIDADE DE USO:

- Deve ser intuitivo e fácil de usar para os operadores do estacionamento, com consideração para o tratamento de diferentes tipos de veículos;
- Deve oferecer treinamento adequado para os funcionários que operarão o sistema, abordando as particularidades de cada categoria de veículo.

11. ATUALIZAÇÕES E MANUTENÇÃO:

- Deve permitir a aplicação de atualizações de software e manutenção sem interromper as operações normais do estacionamento, considerando a segmentação de veículos e a presença de reboque.

Repare que a lista foi preservada, mas muitos requisitos foram alterados para considerar a nova instrução. Com posse desses requisitos funcionais preliminares, você utiliza o questionário sugerido pela IA para entrevistar os usuários e coletar as informações. Posteriormente, você pode enviar uma instrução à IA pedindo para transformar as respostas dos usuários em *user stories*, por exemplo. Ou pode solicitar aprofundar o conjunto de requisitos funcionais, complementando a instrução com as respostas dos usuários como contexto.

Utilizar a IA assim pode ser uma forma eficiente de iniciar um projeto e auxiliar na elaboração de requisitos funcionais mais

precisos e alinhados às necessidades do usuário. No entanto, é fundamental que a analista de sistemas esteja envolvida no processo para garantir a adequação e a qualidade dos requisitos gerados.

7.2 Estimando esforço e prazos

Um modelo de linguagem está familiarizado com as metodologias mais populares para estimar esforço e tempo no desenvolvimento de sistemas. É uma experiência valiosa, mesmo ao lidar com aspectos abstratos, como a estimativa de produção de um software personalizado. Se utilizarmos a própria conversa que sugeriu a lista de requisitos funcionais do tópico anterior e pedirmos, por exemplo, uma estimativa do esforço para o desenvolvimento da aplicação, obteremos:

Você: *Considerando uma equipe formada por 1 dev sênior e 2 plenos e que cada dev trabalhará 40 horas por semana, calcule de forma objetiva a estimativa de esforço em meses para desenvolvimento desta aplicação e explique de forma bem resumida a metodologia utilizada.*

IA: Para estimar o esforço em meses para o desenvolvimento da aplicação com uma equipe composta por 1 desenvolvedor sênior e 2 desenvolvedores plenos, cada trabalhando 40 horas por semana, você pode usar a metodologia de *pontos de função* (*Function Points*) e a técnica de *análise de pontos de função não ajustados* (*Unadjusted Function Points* – UFP). A seguir, vou explicar resumidamente como isso pode ser feito:

Metodologia:

1. **IDENTIFICAR FUNCIONALIDADES:** liste todas as funcionalidades principais da aplicação, como cadastro de clientes, controle de vagas, tarifação, relatórios etc.;
2. **ATRIBUIR PONTOS DE FUNÇÃO NÃO AJUSTADOS (UFP):** classifique cada funcionalidade de acordo com sua complexidade e atribua *pontos de função não ajustados* (UFP) a cada uma delas. Por exemplo:
 - Simples (3 UFP);
 - Média (4 UFP);
 - Complexa (6 UFP).
3. **SOMAR UFP:** some todos os UFP para obter o total de pontos de função não ajustados (Total UFP);
4. **AJUSTAR OS PONTOS DE FUNÇÃO:** considere fatores de ajuste, como a complexidade da arquitetura, a experiência da equipe, a tecnologia utilizada, entre outros, para calcular os *pontos de função ajustados* (AFP);
5. **ESTIMAR A PRODUTIVIDADE:** com base na experiência da equipe e na tecnologia escolhida, estime a produtividade em pontos de função por mês;
6. **CALCULAR O ESFORÇO EM MESES:** divida o AFP pela produtividade em pontos de função por mês para obter o esforço estimado em meses. Por exemplo, se o Total UFP for

50 e o AFP (ajustado) for 60, e a produtividade da equipe for estimada em 10 pontos de função por mês, a estimativa de esforço em meses seria:

Esforço (meses) = AFP / Produtividade

Esforço (meses) = 60 / 10

Esforço (meses) = 6 meses

Portanto, com base nessas estimativas hipotéticas, o desenvolvimento da aplicação levaria aproximadamente 6 meses com a equipe mencionada.

Lembre-se de que essas estimativas são simplificadas e podem variar dependendo de diversos fatores, como a complexidade real do projeto, a eficiência da equipe e os desafios encontrados durante o desenvolvimento. É sempre importante revisar e atualizar as estimativas à medida que o projeto avança e mais informações se tornam disponíveis.

Alternativamente, em continuação da conversa do exemplo anterior, você poderia definir especificamente uma metodologia, como mostra o exemplo a seguir:

Você: *Considerando uma equipe formada por 1 dev sênior e 2 plenos e que cada dev trabalhará 40 horas por semana, calcule de forma objetiva a estimativa de esforço em meses para desenvolvimento desta aplicação utilizando pontos por caso de uso.*

IA: Para calcular a estimativa de esforço em meses para o desenvolvimento da aplicação utilizando pontos por caso de uso, você pode seguir a abordagem de *análise de pontos de casos de uso* (*Use Case Points* – UCP). Essa metodologia é baseada na quantidade e complexidade dos casos de uso do sistema. Vou explicar o processo de forma resumida:

Passo 1: Identificar e contar os casos de uso

Primeiro, liste todos os casos de uso da aplicação. Os casos de uso são ações ou funcionalidades específicas que o sistema deve realizar. Por exemplo, "Cadastro de clientes", "Controle de vagas", "Tarifação automática", "Geração de relatórios" etc.

Passo 2: Determinar os pontos de complexidade

Para cada caso de uso, atribua *pontos de complexidade* com base em três fatores: Ator principal, Complexidade e Fronteira. Cada fator é pontuado em uma escala de 1 a 5, sendo 1 para baixa complexidade e 5 para alta complexidade. Por exemplo:

- Ator principal (AP): Quantos atores principais estão envolvidos no caso de uso?
- Complexidade (C): Quão complexo é o caso de uso em termos de regras de negócio e processamento de dados?
- Fronteira (F): Quão complexa é a interação com sistemas externos?

Passo 3: Calcular os pontos de casos de uso (UCP)

Para cada caso de uso, calcule os pontos de caso de uso multiplicando os fatores AP, C e F. Em seguida, some todos os pontos de caso de uso para obter o total de UCP.

Passo 4: Calcular a estimativa de esforço em meses

Com os UCP calculados, você pode estimar o esforço em meses com base na produtividade da equipe e na média de UCP que a

equipe pode entregar por mês. Por exemplo:

- Suponha que a equipe possa entregar, em média, 15 UCP por mês;
- Se o total de UCP for 100, a estimativa de esforço em meses seria:

Estimativa de esforço (meses) = Total de UCP / Produtividade

Estimativa de esforço (meses) = 100 / 15

Estimativa de esforço (meses) = 6,67 meses

Portanto, com base nessas estimativas hipotéticas, o desenvolvimento da aplicação levaria aproximadamente 6,67 meses com a equipe mencionada.

Lembre-se de que essas estimativas são simplificadas e podem variar dependendo de diversos fatores, como a complexidade real do projeto, a eficiência da equipe e os desafios encontrados durante o desenvolvimento. É importante revisar e atualizar as estimativas à medida que o projeto avança e mais informações se tornam disponíveis.

Ao solicitar uma estimativa dessa forma, você pode obter uma resposta útil como ponto de partida. Um analista inexperiente pode usar a “experiência” de um modelo de linguagem treinado para obter estimativas precisas, assim como um analista experiente pode usá-lo como um contraponto. Obviamente, quanto mais detalhes forem fornecidos, mais precisa será a resposta. Portanto é recomendável aprofundar a conversa com o modelo de linguagem.

No exemplo anterior, trabalhamos com pontos por caso de uso, o que envolve uma tabela com pesos por fatores de complexidade. Em continuação à conversa do exemplo anterior, você pode pedir sugestões de fatores de complexidade, conforme o exemplo a seguir:

Você: *Neste cenário, o que sugere como fatores de complexidade?*

IA: Os fatores de complexidade utilizados na metodologia de análise de pontos de casos de uso (*Use Case Points* - UCP) incluem o *Ator principal* (AP), *Complexidade* (C) e *Fronteira* (F). Aqui estão algumas sugestões de como determinar esses fatores de complexidade para casos de uso:

1. ATOR PRINCIPAL (AP):

- Baixa complexidade (1): um único ator principal envolvido;
- Média complexidade (2): dois ou três atores principais envolvidos;
- Alta complexidade (3): quatro ou mais atores principais envolvidos.

2. COMPLEXIDADE (C):

- Baixa complexidade (1): casos de uso simples, com poucas regras de negócio e interações simples com dados;
- Média complexidade (2): casos de uso moderadamente complexos, com regras de negócio um pouco mais complexas ou interações de dados intermediárias;
- Alta complexidade (3): casos de uso altamente complexos, com regras de negócio complexas e interações de dados significativas.

3. FRONTEIRA (F):

- Baixa complexidade (1): Pouca ou nenhuma interação com sistemas externos;
- Média complexidade (2): Interação moderada com sistemas externos, como integrações simples;
- Alta complexidade (3): Interação intensa com sistemas externos, com integrações complexas e múltiplas.

Essas sugestões podem variar dependendo do contexto do seu projeto. É importante envolver a equipe de desenvolvimento e os stakeholders para determinar com precisão os fatores de

complexidade de cada caso de uso. Além disso, você pode ajustar essas pontuações de acordo com as necessidades específicas do seu projeto e a experiência da equipe. A análise de complexidade é uma parte importante do processo de estimativa de esforço, pois ajuda a refletir a realidade do trabalho envolvido em cada caso de uso.

Complementarmente, você pode até pedir sugestão de pesos:

Você: *Faça uma tabela sugerindo pesos para esses fatores de complexidade.* _

IA: Aqui está uma tabela sugerindo pesos para os fatores de complexidade (Ator principal - AP; Complexidade - C; e Fronteira - F) na análise de pontos de casos de uso (*Use Case Points* - UCP). Estes pesos são apenas sugestões e podem ser ajustados de acordo com as necessidades e características específicas do seu projeto:

Fator de complexidade	Peso sugerido
Ator Principal (AP)	
- Baixa complexidade (1)	1
- Média complexidade (2)	2
- Alta complexidade (3)	3
Complexidade (C)	
- Baixa complexidade (1)	1
- Média complexidade (2)	2
- Alta complexidade (3)	3
Fronteira (F)	
- Baixa complexidade (1)	1
- Média complexidade (2)	2
- Alta complexidade (3)	3

Na análise de pontos de casos de uso (UCP), você pode multiplicar os valores desses fatores para cada caso de uso e somar os resultados para calcular o total de pontos de caso de uso (Total UCP) para o seu projeto. Isso permitirá que você tenha uma estimativa do esforço envolvido no desenvolvimento com base na complexidade de cada caso de uso, considerando os atores principais, a complexidade e as interações com

sistemas externos. Lembre-se de que esses pesos são apenas uma diretriz inicial e podem ser ajustados de acordo com as necessidades do seu projeto.

À medida que a conversa com o modelo de linguagem evolui, você pode explorar cada vez mais profundamente e obter insights valiosos para a estimativa de esforço e prazos no desenvolvimento de sistemas. Embora os exemplos anteriores tenham sido usados como contexto nas interações anteriores da própria conversa, os requisitos poderiam ser informados na própria instrução de forma explícita.

7.3 Especificando requisitos técnicos

Um modelo de linguagem é treinado com um volume grande de informações técnicas sobre as mais diversas tecnologias e uma infinidade de linguagens, serviços, ferramentas etc., o que inclui os requisitos operacionais. Ao fornecer informações suficientes e formular perguntas objetivas no prompt, é possível obter uma lista de requisitos técnicos que podem ser utilizados na documentação do seu projeto, o que é extremamente valioso. Analise o exemplo a seguir, em que citamos as tecnologias e ferramentas que serão utilizadas em um projeto e solicitamos os requisitos técnicos:

Você: *Escreva os requisitos técnicos com dimensionamento e recursos para a operação de um sistema web com base de dados MySQL, back-end em Python e front-end em React, para atender a uma demanda de 10.000 usuários por dia. Considere que o sistema terá 3 páginas estáticas e 10 páginas dinâmicas cujo processamento é de uma média de 3 queries simples indexadas e com um máximo de 50 registros. Não haverá clob ou blob nas queries e a base de dados será menor do que 500Mb.*

IA: Aqui estão os requisitos técnicos com dimensionamento e recursos para operar um sistema web com as especificações fornecidas:

Requisitos de hospedagem e infraestrutura:

1. SERVIDOR WEB E HOSPEDAGEM:

- Um servidor web confiável, como Nginx, configurado para lidar com tráfego HTTP/HTTPS;
- Hospedagem em um provedor de serviços em nuvem ou servidor dedicado.

2. BASE DE DADOS MySQL:

- Um servidor MySQL configurado para desempenho e alta disponibilidade;
- Dimensionamento inicial com recursos adequados para lidar com a carga estimada.

Requisitos de hardware e escalabilidade:

1. SERVIDOR WEB:

- Recomenda-se uma máquina virtual com pelo menos 2 vCPUs e 4 GB de RAM para o servidor web;
- A escalabilidade vertical deve ser possível, adicionando mais recursos conforme necessário.

2. BASE DE DADOS MySQL:

- Uma máquina virtual com pelo menos 4 vCPUs e 8 GB de RAM para o servidor MySQL.
- Considere a capacidade de escalabilidade vertical, adicionando mais recursos à medida que a base de dados cresce.
- Configuração de armazenamento com espaço suficiente para a base de dados estimada (< 500 MB) e backups.

Requisitos de software:

1. BACK-END EM PYTHON:

- Ambiente Python (por exemplo, Python 3.x) instalado no servidor;
- Framework Python, como Flask ou Django, configurado;
- Bibliotecas Python relevantes para lidar com solicitações HTTP, conexão de banco de dados etc.

2. FRONT-END EM REACT:

- Ambiente de desenvolvimento React configurado;
- Gerenciamento de pacotes Node.js (por exemplo, `npm` ou `yarn`) para instalação de dependências;
- Otimização e compactação de recursos estáticos para produção.

Requisitos de segurança:

1. Implementação de medidas de segurança, como autenticação e autorização de usuários;
2. Proteção contra ataques comuns, como injeção SQL, XSS e CSRF;
3. Uso de HTTPS para proteger a comunicação entre cliente e servidor.

Requisitos de desempenho e otimização:

1. Cache:

- Implementação de cache para reduzir a carga no banco de dados e melhorar o desempenho;
- Utilização de ferramentas como Redis ou Memcached para *caching* de dados.

2. Monitoramento e logs:

- Configuração de ferramentas de monitoramento para acompanhar o desempenho do sistema, como o uso de CPU, memória e latência;

- Registros detalhados para análise de erros e problemas de desempenho.

Requisitos de backup e recuperação:

1. Rotinas de backup regulares para proteger os dados, considerando a base de dados menor que 500 MB;
2. Um plano de recuperação de desastres em caso de falhas críticas.

Requisitos de otimização de consultas:

1. Otimização contínua das *queries* SQL para manter o desempenho;
2. Monitoramento do uso de índices e análise do plano de execução das *queries*.

Requisitos de conteúdo estático:

1. Armazenamento eficiente e distribuição de conteúdo estático, como imagens, por meio de um serviço de CDN (*Content Delivery Network*).

Lembre-se de que esses requisitos são uma orientação geral e podem variar dependendo das especificidades do projeto e das tecnologias exatas escolhidas. Além disso, é importante monitorar continuamente o desempenho do sistema e ajustar os recursos conforme necessário para atender à demanda real.

Naturalmente, é sempre necessário verificar as informações e revisar os dados obtidos. Quanto mais detalhes objetivos você fornecer no prompt, maior será a precisão dos resultados gerados.

7.4 Organizando as tarefas de um projeto

Um modelo de linguagem bem treinado está familiarizado com o **PMBOK** (*Project Management Body of Knowledge*), um guia de boas práticas em gerenciamento de projetos desenvolvido pelo Project Management Institute (PMI), organização internacional sem fins lucrativos, assim como outras metodologias de desenvolvimento de projetos. Após discutir os requisitos em uma conversa, você pode simplesmente fazer a seguinte solicitação:

Você: *Faça uma tabela de tarefas de projeto para o desenvolvimento deste projeto conforme o PMBOK.*

IA: O PMBOK (*Project Management Body of Knowledge*) fornece uma estrutura geral para a gestão de projetos. Vou criar uma tabela de tarefas de projeto simplificada para o desenvolvimento do projeto de controle de estacionamento, seguindo os grupos de processos do PMBOK (Iniciação, Planejamento, Execução, Monitoramento e Controle e Encerramento) e destacando algumas tarefas-chave em cada um deles. Lembre-se de que um projeto real exigirá uma lista mais detalhada de tarefas e uma equipe de projeto adequada.

Grupo de processos	Tarefas principais
Iniciação	1. Definir o escopo do projeto.
	2. Identificar partes interessadas.
	3. Criar um termo de abertura do projeto (<i>Project Charter</i>).
	4. Selecionar a equipe de projeto.
	5. Realizar análise de viabilidade.
Planejamento	6. Desenvolver o plano de projeto.
	7. Identificar e definir os requisitos do sistema.
	8. Estabelecer um cronograma de projeto.
	9. Definir orçamento e recursos necessários.
	10. Planejar a gestão de riscos.
	11. Planejar a gestão da qualidade.
	12. Elaborar um plano de comunicação.

Grupo de processos	Tarefas principais
Execução	13. Alocar recursos para tarefas específicas.
	14. Desenvolver o sistema de controle de estacionamento.
	15. Contratar e treinar a equipe de atendimento.
	16. Implementar o sistema de segurança e acesso.
	17. Iniciar a reserva e operação de vagas.
Monitoramento e controle	18. Monitorar o progresso do projeto.
	19. Gerenciar mudanças no escopo.
	20. Realizar controle de qualidade.
	21. Monitorar o desempenho financeiro.
	22. Gerenciar riscos conforme necessário.
	23. Avaliar e relatar o status do projeto.
Encerramento	24. Realizar a entrega formal do projeto.
	25. Liberar recursos e finalizar contratos.
	26. Realizar revisão pós-implementação.

Grupo de processos	Tarefas principais
	27. Documentar lições aprendidas.
	28. Encerrar o projeto e arquivar documentos.

Lembre-se de que o sucesso do projeto depende de uma gestão eficaz, acompanhamento contínuo e ajustes conforme necessário. Essa tabela de tarefas é apenas um ponto de partida e pode ser expandida e personalizada de acordo com as necessidades específicas do projeto de controle de estacionamento.

Você pode ainda pedir detalhamento de cada fase:

Você: *Escreva agora uma tabela com o detalhamento do grupo de processos de iniciação.*

IA: Certamente, aqui está uma tabela com o detalhamento das tarefas do grupo de processos de iniciação para o projeto de controle de estacionamento:

Tarefa	Descrição
Definir o escopo do projeto	Identificar e documentar o escopo do projeto, incluindo os objetivos e limites claros do sistema de controle de estacionamento.
Identificar partes interessadas	Identificar todas as partes interessadas envolvidas no projeto, incluindo proprietários do estacionamento, motoristas, autoridades regulatórias etc.
Criar um termo de abertura do projeto (<i>Project Charter</i>)	Documentar o <i>Project Charter</i> , que é um documento formal que autoriza o projeto e nomeia o gerente de projeto.
Selecionar a equipe de projeto	Identificar e designar membros da equipe de projeto, incluindo gerente de projeto, analistas, desenvolvedores, testadores etc.
Realizar análise de viabilidade	Realizar uma análise de viabilidade econômica e técnica para determinar se o projeto é viável e justificável. Isso pode incluir estudos de mercado, custos estimados, ROI etc.

Essas são as tarefas principais do grupo de processos de iniciação. Elas estabelecem as bases para o projeto de controle de estacionamento, garantindo que os objetivos estejam claros, que as partes interessadas estejam identificadas, e que o projeto seja viável antes de prosseguir para o planejamento e execução.

Perceba como rapidamente uma pessoa analista consegue obter percepções relevantes que podem ajudá-la a estruturar de forma bem organizada as tarefas de um projeto.

7.5 Gerando diagramas UML

Um modelo de linguagem não tem a capacidade de gerar imagens diretamente; no entanto, existem outras ferramentas de IA especializadas em imagens, embora estas produzam imagens mais abstratas e não consigam criar imagens precisas e objetivas, como um diagrama. Pode ser complexo elaborar um prompt que oriente o Midjourney, por exemplo, a criar um diagrama técnico específico — diria até que seja impossível, por enquanto. Contudo, modelos de linguagem são excelentes quando se trata de linguagem, e existem muitas ferramentas que geram imagens técnicas a partir de um pseudocódigo próprio. Se essa ferramenta foi publicada antes de 2021, é provável que o modelo de linguagem a conheça.

Tomemos como exemplo o PlantUML (<https://plantuml.com/>), uma ferramenta de código aberto que permite aos usuários criar diagramas a partir de uma linguagem de texto simples. Além de diversos diagramas UML, o PlantUML suporta vários outros formatos relacionados ao desenvolvimento de software, bem como a visualização de arquivos JSON e YAML.

No caso do PlantUML, o código a seguir produziria uma imagem correspondente:

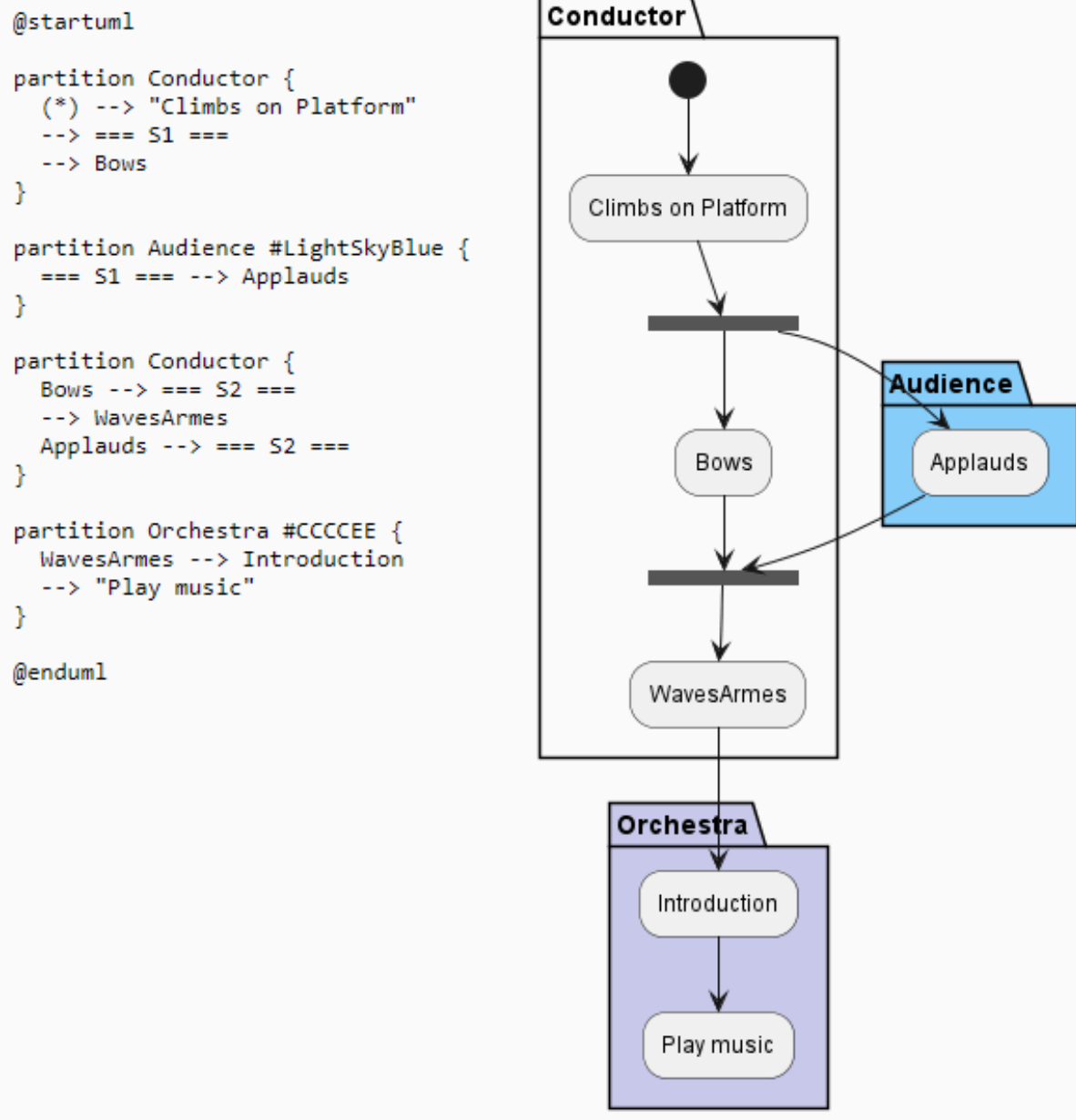


Figura 7.1: Exemplo do PlantUML

E, como um modelo de linguagem é ótimo com, veja só, linguagem, podemos pedir para ele criar o texto do diagrama para o PlantUML:

(Em continuação à conversa do exemplo do tópico *Especificando requisitos funcionais*) **Você:** *Escreva um diagrama de casos de uso dos requisitos discutidos acima para o PlantUML.*

IA: Aqui está um diagrama de casos de uso simples usando a linguagem PlantUML para os requisitos discutidos anteriormente em um sistema de controle de estacionamento de veículos com segmentação por tipo de veículo e presença de reboque. Este diagrama inclui apenas alguns casos de uso principais para ilustração:

```
@startuml
```

```
left to right direction
```

```
!define SYSTEM system
```

```
!define ACTOR actor
```

```
!define UC usecase
```

```
hide empty members
```

```
hide circle
```

```
title Sistema de Controle de Estacionamento de Veículos
```

```
actor Cliente as Cliente
```

```
actor Funcionário as Funcionário
```

```
package "Sistema de Controle" {
```

```
    usecase "Registro de Veículos" as UC_RegistroVeiculo
```

```
    usecase "Gerenciamento de Vagas" as UC_GerenciamentoVagas
```

```
    usecase "Controle de Acesso" as UC_ControlAcesso
```

```
    usecase "Tarifação e Pagamento" as UC_TarifacaoPagamento
```

```
    usecase "Reservas de Vagas" as UC_ReservasVagas
```

```
    usecase "Relatórios e Estatísticas" as UC_RelatoriosEstatisticas
```

```
    usecase "Segurança e Monitoramento" as UC_SegurancaMonitoramento
```

```
    usecase "Comunicação e Notificações" as UC_ComunicacaoNotificacoes
```

```
    usecase "Gerenciamento de Usuários" as UC_GerenciamentoUsuarios
```

```
}
```

```
Cliente --> UC_RegistroVeiculo : Realiza registro
```

```
Cliente --> UC_GerenciamentoVagas : Verifica vagas
```

```
Cliente --> UC_TarifacaoPagamento : Paga tarifa
```

```
Cliente --> UC_ReservasVagas : Faz reserva
```

```
Cliente --> UC_RelatoriosEstatisticas : Acessa relatórios
```

```
Funcionário --> UC_ControleAcesso : Controla acesso
```

```
Funcionário --> UC_SegurancaMonitoramento : Monitora segurança
```

```
Funcionário --> UC_ComunicacaoNotificacoes : Recebe notificações
```

```
Funcionário --> UC_GerenciamentoUsuarios : Gerencia usuários
```

```
@enduml
```

Este diagrama representa os principais atores (Cliente e Funcionário) e os casos de uso associados a diferentes funcionalidades do sistema de controle de estacionamento de veículos. Você pode estender este diagrama para incluir mais detalhes ou casos de uso específicos, conforme necessário para o seu projeto.

Ao executarmos esse código no PlantUML, obtemos a imagem a seguir:

Sistema de Controle de Estacionamento de Veículos

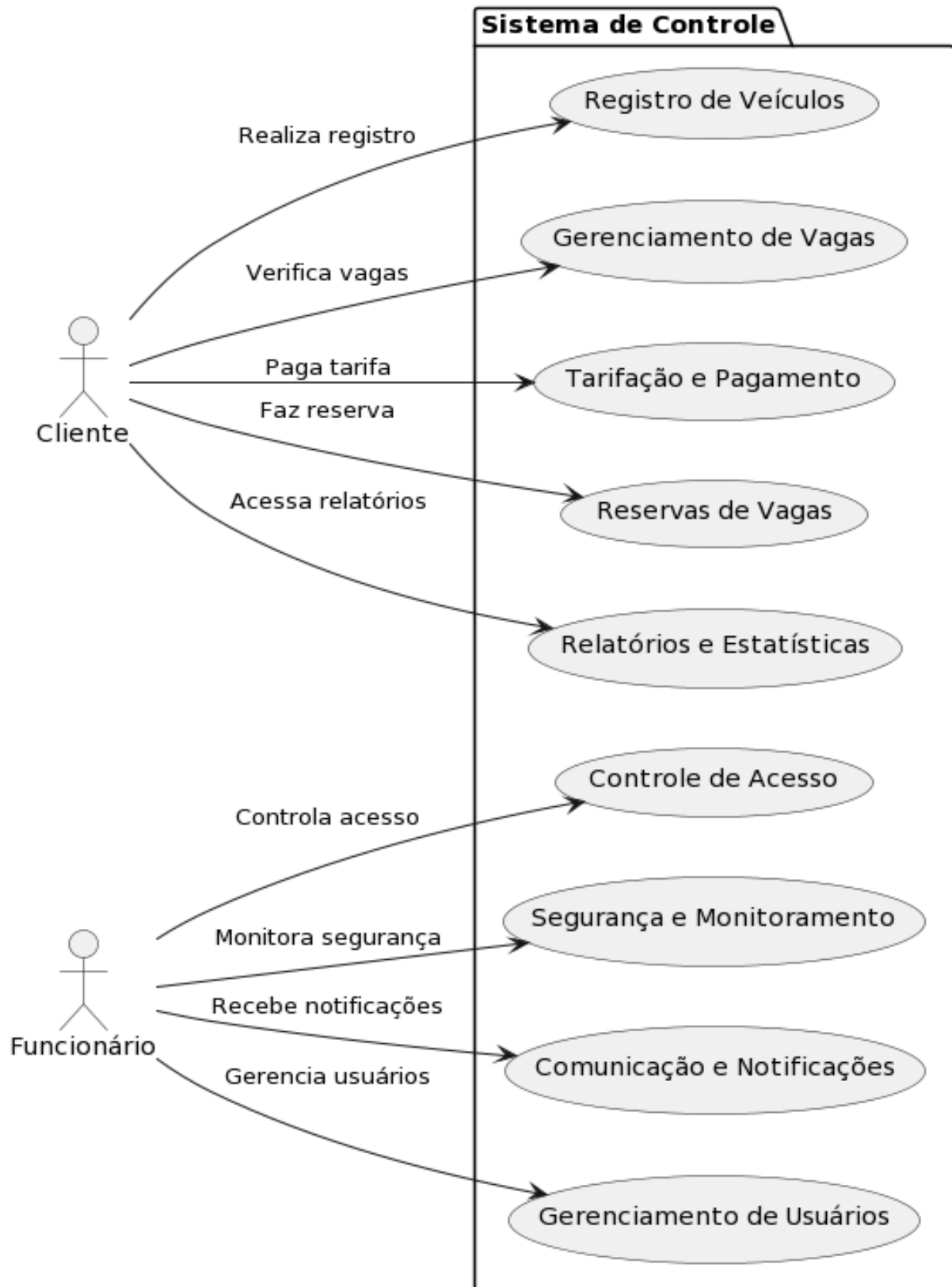


Figura 7.2: Diagrama de casos de uso no PlantUML

E o PlantUML é apenas uma dessas ferramentas. Você pode usar yUML, Nomnoml, TextUML e inúmeras outras, e pode criar não apenas diagramas de casos de uso, mas de sequência, de classes, de objetos e quaisquer outros que a plataforma *text-to-image* suportar. Há uma infinidade de possibilidades.

Mas não é preciso ficar restrito à modelagem. Que tal colocar o modelo de linguagem para programar?

CAPÍTULO 8

Prompts de apoio à codificação

"A questão não é saber os códigos ou linhas de comando, é saber o que fazer com eles" — Sérgio Arena

Um analista de sistemas pode utilizar modelos de linguagem para automatizar partes do processo de codificação de um novo sistema. Ao fornecer prompts específicos, o modelo pode gerar trechos de código que atendem a requisitos particulares, como a criação de funções, métodos ou até mesmo estruturas de dados mais complexas. Isso economiza tempo e esforço, permitindo que o analista se dedique a tarefas que exigem um nível mais alto de raciocínio e criatividade, como a solução de problemas mais complexos e a implementação de lógicas de negócio específicas.

Observação: algumas respostas fornecidas pelos modelos de linguagem nos exemplos foram abreviadas com o sinal de reticências (...) por serem extensas demais. Foi mantido apenas o suficiente para a compreensão do propósito, considerando que essas respostas podem ser facilmente reproduzidas.

8.1 Gerando um pseudocódigo próprio

Uma das limitações da IA é que cada solicitação pode ser respondida de maneira única e exclusiva, o que pode resultar em códigos gerados com estrutura não padronizada. Para contornar isso, pode-se solicitar que a IA desenvolva uma linguagem de pseudocódigo para definir componentes, conforme exemplificado a seguir:

Você: *Crie uma linguagem de pseudocódigo parecida com YAML para criar componentes React.*

IA: Claro, posso criar um pseudocódigo semelhante ao formato YAML para descrever a criação de componentes React. Lembre-se de que isso é apenas uma representação simplificada e um pseudocódigo. A implementação real em React usaria JavaScript ou TypeScript. Aqui está um exemplo:

Componente:

Nome: `MeuComponente`

Props:

- Nome: `prop1`
Tipo: `string`
ValorPadrão: `'Valor Padrão'`
- Nome: `prop2`
Tipo: `number`
- Nome: `prop3`
Tipo: `bool`
ValorPadrão: `true`

Estado:

- Nome: `estado1`
ValorInicial: `0`

Métodos:

- Nome: `handleClick`
Parâmetros:
 - Nome: `evento`Corpo: `|`
`this.setState({ estado1: this.state.estado1 + 1 });`

Renderização:

Retorno: `|`
`<div>`
`<h1>{this.props.prop1}</h1>`
`<p>{this.state.estado1}</p>`
`<button onClick={this.handleClick}>Clique-me</button>`
`</div>`

Nesse pseudocódigo, descrevemos um componente React chamado `MeuComponente` com propriedades (`props`), estado (`state`), e um método `handleClick` . Observe que o pseudocódigo

está apenas ilustrando a estrutura do componente e não é uma sintaxe real do React. Para implementar isso em JavaScript ou TypeScript, você precisaria traduzir essas descrições para o código real do React.

O modelo de linguagem dá até instruções de como usar a linguagem que ele acabou de criar. Essa é uma forma de estabelecer um mecanismo padronizado de geração de código que condicionará a execução a um determinado padrão definido por você. Seria seu próprio framework. Embora você possa estabelecer os padrões no próprio prompt, você teria que escrevê-los em todas as conversas.

8.2 Gerando ícones e imagens

Embora um modelo de linguagem não crie imagens, já vimos que é possível criar códigos que sejam interpretados como imagens por alguma ferramenta. No HTML, temos o **Scalable Vector Graphics** (SVG), um formato de arquivo vetorial e textual compatível com a web que certamente será reconhecido por um modelo de linguagem bem treinado. Podemos pedir para o modelo escrever códigos assim:

Você: *Escreva um código SVG que desenhe 1 círculo preto vazado verticalmente centralizado à esquerda com uma linha ligando-o a 2 outros círculos pretos vazados distribuídos de forma verticalmente centralizada à direita. Faça apenas o elemento SVG, sem necessidade do restante do HTML. Os 3 círculos devem ter o mesmo tamanho. A linha de ligação deve ligar os contornos dos círculos.*

IA: Aqui está o código SVG que desenha um círculo vazado à esquerda com uma linha que liga dois outros círculos vazados à direita, todos com o mesmo tamanho e a linha conectando os contornos dos círculos:

```
<svg xmlns="http://www.w3.org/2000/svg" width="200" height="100">
  <!-- Círculo vazado à esquerda -->
  <circle cx="50" cy="50" r="20" fill="none" stroke="black" stroke-
width="2" />

  <!-- Círculo vazado superior à direita -->
  <circle cx="130" cy="30" r="20" fill="none" stroke="black" stroke-
width="2" />

  <!-- Círculo vazado inferior à direita -->
  <circle cx="130" cy="70" r="20" fill="none" stroke="black" stroke-
width="2" />

  <!-- Linha de ligação -->
  <line x1="70" y1="50" x2="110" y2="30" stroke="black" stroke-
width="2" />
  <line x1="70" y1="50" x2="110" y2="70" stroke="black" stroke-
width="2" />
</svg>
```

Neste código SVG, temos um círculo vazado à esquerda e dois círculos vazados à direita, todos com o mesmo tamanho. As linhas de ligação conectam os contornos dos círculos, proporcionando o efeito desejado. Você pode ajustar os tamanhos e as posições conforme necessário.

Podemos verificar o resultado desse SVG renderizado em um navegador na imagem a seguir:

```

<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<svg xmlns="http://www.w3.org/2000/svg" width="200" height="100">
  <!-- Círculo vazado à esquerda -->
  <circle cx="50" cy="50" r="20" fill="none" stroke="black" stroke-width="2" />

  <!-- Círculo vazado superior à direita -->
  <circle cx="130" cy="30" r="20" fill="none" stroke="black" stroke-width="2" />

  <!-- Círculo vazado inferior à direita -->
  <circle cx="130" cy="70" r="20" fill="none" stroke="black" stroke-width="2" />

  <!-- Linha de ligação -->
  <line x1="70" y1="50" x2="110" y2="30" stroke="black" stroke-width="2" />
  <line x1="70" y1="50" x2="110" y2="70" stroke="black" stroke-width="2" />
</svg>

</body>
</html>

```

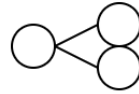


Figura 8.1: Exemplo de SVG simples renderizado em um navegador

Não é nenhuma obra de arte, mas é um ótimo ponto de partida para você mesmo aperfeiçoar o artefato. Outro exemplo com maior grau de complexidade pode ser visto a seguir:

Você: *Escreva uma tag SVG que desenhe um círculo e ponteiros como os de um relógio marcando 3 horas, 30 minutos e 45 segundos.*

IA: Claro! Você pode criar uma tag SVG que desenha um círculo e ponteiros para representar 3 horas, 30 minutos e 45 segundos da seguinte maneira:

```
<svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <!-- Desenha o círculo -->
  <circle cx="100" cy="100" r="80" stroke="black" stroke-width="2"
fill="none" />

  <!-- Ponteiro das horas (3 horas) -->
  <line x1="100" y1="100" x2="140" y2="100" stroke="black" stroke-
width="5" />

  <!-- Ponteiro dos minutos (30 minutos) -->
  <line x1="100" y1="100" x2="100" y2="160" stroke="black" stroke-
width="3" />

  <!-- Ponteiro dos segundos (45 segundos) -->
  <line x1="100" y1="100" x2="50" y2="50" stroke="red" stroke-
width="1" />
</svg>
```

Neste exemplo, criamos um círculo preto com um raio de 80 unidades e três ponteiros: um ponteiro de horas em preto (representando 3 horas), um ponteiro de minutos em azul (representando 30 minutos) e um ponteiro de segundos em vermelho (representando 45 segundos). Certifique-se de ajustar os tamanhos, cores e estilos de acordo com suas preferências.

Podemos, da mesma forma, verificar o resultado desse SVG renderizado em um navegador na imagem a seguir:


```

<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <!-- Desenha o círculo -->
  <circle cx="100" cy="100" r="80" stroke="black" stroke-width="2" fill="none" />

  <!-- Ponteiro das horas (3 horas) -->
  <line x1="100" y1="100" x2="140" y2="100" stroke="black" stroke-width="5" />

  <!-- Ponteiro dos minutos (30 minutos) -->
  <line x1="100" y1="100" x2="100" y2="160" stroke="black" stroke-width="3" />

  <!-- Ponteiro dos segundos (45 segundos) -->
  <line x1="100" y1="100" x2="50" y2="50" stroke="red" stroke-width="1" />
</svg>

</body>
</html>

```

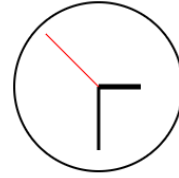


Figura 8.2: Exemplo de SVG mais complexo renderizado em um navegador

E você pode usar esse mesmo conceito para gerar imagens em *buffer* para qualquer linguagem de programação. Para ilustrar, o exemplo a seguir solicita ao modelo de linguagem um código que desenha o relógio analógico em uma linguagem específica:

Você: *Escreva um método na linguagem Java que desenha um círculo e ponteiros como os de um relógio marcando 3 horas, 30 minutos e 45 segundos.*

IA: Para desenhar um círculo com ponteiros de relógio que marcam 3 horas, 30 minutos e 45 segundos em Java, você pode usar a biblioteca Java Swing para criar uma interface gráfica simples. Aqui está um exemplo de código que faz isso:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class Relogio extends JPanel implements ActionListener {
    private int horas = 3;
    private int minutos = 30;
    private int segundos = 45;

    public Relogio() {
        Timer timer = new Timer(1000, this);
        timer.start();
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        segundos++;
        if (segundos == 60) {
            segundos = 0;
            minutos++;
            if (minutos == 60) {
                minutos = 0;
                horas++;
                if (horas == 12) {
                    horas = 0;
                }
            }
        }
        repaint();
    }

    @Override
```

```

protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    int raio = 100;
    int centroX = getWidth() / 2;
    int centroY = getHeight() / 2;

    // Desenha o círculo
    g.setColor(Color.BLACK);
    g.drawOval(centroX - raio, centroY - raio, 2 * raio, 2 *
raio);

    // Calcula os ângulos dos ponteiros
    double anguloHoras = Math.toRadians((horas % 12) * 30 - 90);
    double anguloMinutos = Math.toRadians(minutos * 6 - 90);
    double anguloSegundos = Math.toRadians(segundos * 6 - 90);

    // Desenha o ponteiro das horas
    int tamanhoHoras = raio - 20;
    int xHoras = (int) (centroX + tamanhoHoras *
Math.cos(anguloHoras));
    int yHoras = (int) (centroY + tamanhoHoras *
Math.sin(anguloHoras));
    g.drawLine(centroX, centroY, xHoras, yHoras);

    // Desenha o ponteiro dos minutos
    int tamanhoMinutos = raio - 10;
    int xMinutos = (int) (centroX + tamanhoMinutos *
Math.cos(anguloMinutos));
    int yMinutos = (int) (centroY + tamanhoMinutos *
Math.sin(anguloMinutos));
    g.drawLine(centroX, centroY, xMinutos, yMinutos);

    // Desenha o ponteiro dos segundos
    int tamanhoSegundos = raio - 10;
    int xSegundos = (int) (centroX + tamanhoSegundos *
Math.cos(anguloSegundos));
    int ySegundos = (int) (centroY + tamanhoSegundos *
Math.sin(anguloSegundos));
    g.setColor(Color.RED);

```

```

        g.drawLine(centroX, centroY, xSegundos, ySegundos);
    }

    public static void main(String[] args) {
        JFrame frame = new JFrame("Relógio");
        Relogio relógio = new Relogio();
        frame.add(relógio);
        frame.setSize(300, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

```

Nesse código, criamos um painel personalizado que estende `JPanel` para desenhar o círculo e os ponteiros. Usamos um `Timer` para atualizar a posição do segundo ponteiro a cada segundo. O método `paintComponent` é usado para desenhar o círculo e os ponteiros com base nos ângulos calculados para as horas, minutos e segundos atuais. O `main` cria uma janela Swing para exibir o relógio. Certifique-se de adicionar essa classe a um projeto Java Swing para executá-lo.

Agora, considere uma pessoa programando para gerar imagens, mas sem poder ver o resultado para ajustar o processo. Pense que essa pessoa também nunca pôde enxergar em sua vida. Dá para perceber o quão complexa essa tarefa seria? É exatamente esse o tipo de desafio que um modelo de linguagem enfrenta. É claro que os resultados podem ser inconsistentes em muitos casos, mas esses modelos servem como excelentes pontos de partida, permitindo uma abordagem prática na utilização de bibliotecas e recursos. Isso torna essa ferramenta incrivelmente útil e produtiva.

8.3 Simulando interação com sistemas ou serviços

Você pode usar um modelo de linguagem para simular interação com sistemas operacionais como MS-DOS ou Linux, sistemas gerenciadores de bancos de dados como Oracle ou PostgreSQL ou qualquer outro sistema ou serviço operado por linha de comando. Com isso, podemos ganhar tempo. Em vez de instalar e configurar um sistema ou serviço para testar um comando ou analisar uma resposta a ser processada por um programa, podemos simulá-los através de um modelo de linguagem. Observe no exemplo a seguir como simular interações com o MS-DOS, antigo sistema operacional manuseado por linha de comando:

Você: *Emule o MS-DOS, fornecendo o prompt para que eu execute os comandos respondendo-os como se fosse o sistema operacional em si.*

IA: Claro, vou simular um ambiente MS-DOS e fornecer um prompt para que você possa executar comandos como se estivesse usando o sistema operacional real. Lembre-se de que esta é apenas uma simulação e não é um ambiente MS-DOS real. Aqui está o prompt:

C:\>

Agora você pode digitar os comandos que deseja executar, e eu responderei como se fosse o MS-DOS. Lembre-se de que estou simulando um ambiente MS-DOS e não posso executar comandos reais no seu sistema. Portanto, use comandos fictícios apenas para fins de simulação. O que você gostaria de fazer?

Você: dir

IA:

```
C:\>dir
```

Volume in drive C has no label.

Volume Serial Number is 1234-5678

Directory of C:\

09/24/2023	10:00 AM	<DIR>	Program Files
09/24/2023	10:00 AM	<DIR>	Users
09/24/2023	10:00 AM	<DIR>	Windows
		0 File(s)	0 bytes
		3 Dir(s)	1,024,000,000 bytes free

Aqui está a listagem de diretórios no diretório raiz do disco C. Lembre-se de que esses diretórios são fictícios e apenas parte da simulação. Você pode continuar a executar outros comandos ou fazer outras perguntas relacionadas ao MS-DOS simulado.

Você: md nova_pasta

IA:

```
C:\>md nova_pasta
```

A pasta `nova_pasta` foi criada com sucesso no diretório atual. Você pode continuar a executar comandos ou fazer outras perguntas relacionadas ao MS-DOS simulado.

Você: dir

IA:

```
C:\>dir
```

Volume in drive C has no label.

Volume Serial Number is 1234-5678

Directory of C:\

09/24/2023	10:00 AM	<DIR>	Program Files
09/24/2023	10:00 AM	<DIR>	Users
09/24/2023	10:00 AM	<DIR>	Windows
09/24/2023	10:05 AM	<DIR>	nova_pasta
		0 File(s)	0 bytes
		4 Dir(s)	1,024,000,000 bytes free

Aqui está a listagem de diretórios após a criação da pasta nova_pasta . Ela agora aparece no diretório raiz do disco C. Como sempre, lembre-se de que este é um ambiente simulado e os diretórios são fictícios. Se você tiver mais comandos ou perguntas, sinta-se à vontade para continuar.

Você: cd nova_pasta

IA:

```
C:\>cd nova_pasta
```

Você agora está dentro da pasta nova_pasta . Você pode continuar a executar comandos dentro dessa pasta ou deseja fazer algo mais?

Você: fsutil file createnew novo_arquivo.txt

IA:

```
C:\nova_pasta>fsutil file createnew novo_arquivo.txt  
File novo_arquivo.txt created successfully.
```

O comando `fsutil file createnew` foi usado para criar um novo arquivo chamado `novo_arquivo.txt` dentro da pasta `nova_pasta`. O arquivo foi criado com sucesso. Você pode continuar a executar comandos dentro desta pasta ou fazer outras perguntas relacionadas ao MS-DOS simulado.

Você: `dir`

IA:

```
C:\nova_pasta>dir  
Volume in drive C has no label.  
Volume Serial Number is 1234-5678
```

Directory of C:\nova_pasta

09/24/2023	10:10 AM	0	novo_arquivo.txt
		1 File(s)	0 bytes
		0 Dir(s)	1,024,000,000 bytes free

Aqui está a listagem de arquivos na pasta `nova_pasta` após a criação do arquivo `novo_arquivo.txt`. Se você tiver mais comandos ou perguntas, sinta-se à vontade para continuar.

Esse recurso pode ser particularmente útil quando estamos desenvolvendo um sistema que vai interagir com o shell e precisamos ter exemplos de respostas para implementar e testar ações. Uso frequentemente, e isso me poupa muito tempo e energia.

Outra necessidade comum aos analistas de sistemas é implementar modelos de bancos de dados para gerar e testar comandos. Veja no exemplo a seguir como é simples implementar um modelo de dados para criar e testar comandos em um modelo de linguagem sem sequer precisar instalar e configurar nada no seu ambiente de desenvolvimento:

Você: *Emule o PostgreSQL, fornecendo o prompt para que eu execute os comandos respondendo-os como se fosse o sistema em si.*

IA: Claro, eu posso simular um ambiente PostgreSQL e fornecer um prompt para você executar comandos SQL. Por favor, lembre-se de que esta é uma simulação simplificada e não uma instância real do PostgreSQL. Você pode digitar comandos SQL e eu vou responder de acordo.

Bem-vindo ao PostgreSQL Simulator 2023!

Digite os comandos SQL abaixo e eu vou responder como se fosse o PostgreSQL real.

postgres=#

Agora você pode digitar seus comandos SQL após o prompt `postgres=#`. Por exemplo, você pode criar uma tabela, inserir dados ou fazer consultas SQL, e eu vou responder de acordo com a simulação.

Por favor, sintá-se à vontade para começar com um comando SQL específico ou fazer perguntas relacionadas ao PostgreSQL.

Você: `create table usuarios(id bigserial primary key, nome varchar)`

IA:

```
CREATE TABLE  
postgres=#
```

A tabela `usuarios` foi criada com sucesso. Ela possui uma coluna `id` do tipo `bigserial` como chave primária e uma coluna `nome` do tipo `varchar`. Se desejar, você pode continuar executando outros comandos SQL ou fazer qualquer outra pergunta relacionada ao PostgreSQL.

Você: `insert into usuarios(1, 'Fulano da Silva'); insert into usuarios(1, 'Beltrano da Silva'); insert into usuarios(1, 'Cicrano da Silva');`

IA: Parece que há um erro nos comandos `INSERT` que você forneceu. Ao inserir dados em uma tabela no PostgreSQL, você deve especificar os valores entre parênteses e usar o comando `VALUES` OU `INSERT INTO ... VALUES`. Vou corrigir os comandos para você:

```
INSERT INTO usuarios (nome) VALUES ('Fulano da Silva');  
INSERT INTO usuarios (nome) VALUES ('Beltrano da Silva');  
INSERT INTO usuarios (nome) VALUES ('Cicrano da Silva');
```

Aqui estão os comandos corrigidos. Eles inserirão três registros na tabela `usuarios` com os nomes especificados. Vou executar esses comandos para você:

```
INSERT 0 1  
INSERT 0 1  
INSERT 0 1  
postgres=#
```

Os registros foram inseridos com sucesso na tabela `usuarios`. Se você quiser fazer mais consultas ou executar outros comandos, sinta-se à vontade para continuar.

Você: `delete from usuarios`

IA: Você deseja excluir todos os registros da tabela `usuarios`? Por favor, confirme antes de prosseguirmos com a exclusão.

Você: *sim*

IA: Entendido. Vou prosseguir com a exclusão de todos os registros da tabela `usuarios` :

```
DELETE FROM usuarios;
```

Aqui está o comando executado:

```
DELETE 3  
postgres=#
```

Todos os registros da tabela `usuarios` foram excluídos com sucesso. Se você tiver mais alguma ação que deseja realizar ou alguma pergunta relacionada ao PostgreSQL, por favor, me avise.

Você: `INSERT INTO usuarios (nome) VALUES ('Fulano da Silva'); INSERT INTO usuarios (nome) VALUES ('Beltrano da Silva'); INSERT INTO usuarios (nome) VALUES ('Cicrano da Silva');`

IA: Você inseriu com sucesso três novos registros na tabela `usuarios` com os nomes especificados. Aqui está a resposta:

```
INSERT 0 1  
INSERT 0 1  
INSERT 0 1
```

Se precisar fazer mais alguma ação ou tiver alguma outra pergunta relacionada ao PostgreSQL, sinta-se à vontade para prosseguir.

Você: `select * from usuarios`

IA: Aqui estão os registros na tabela `usuarios` após a inserção dos nomes:

```
id | nome
---+-----
 4 | Fulano da Silva
 5 | Beltrano da Silva
 6 | Cicrano da Silva
(3 rows)
```

Isso mostra os três registros que você inseriu anteriormente na tabela `usuarios`, cada um com um ID exclusivo. Se você tiver mais consultas ou qualquer outra coisa que gostaria de fazer, por favor, me informe.

Algumas observações muito interessantes sobre as interações anteriores:

- Repare que o modelo de linguagem fornece um quadro com a resposta juntamente com instruções;
- Perceba que a resposta da terceira interação não apenas identifica erro nos comandos (a ausência do `VALUES`) mas apresenta a solução e a executa corretamente.

Ou seja: o modelo de linguagem entende um comando mesmo quando ele está errado, algo com que nós programadores definitivamente não estamos acostumados.

Sendo um emulador tão avançado de tantos sistemas e serviços, fica fácil presumir que um modelo de linguagem pode ajudar, e muito, na interação com sistemas dos quais temos pouco ou nenhum conhecimento. Veja no exemplo a seguir como perguntar sobre comandos específicos do CentOS de forma simples:

Você: *Como crio um usuário chamado "fulano" com privilégios administrativos no CentOS?*

IA: Para criar um usuário chamado "fulano" com privilégios administrativos (ou seja, torná-lo um superusuário) no CentOS, você pode seguir os seguintes passos:

1. Faça login como superusuário (root) ou um usuário com privilégios administrativos existente;
2. Abra um terminal;
3. Use o comando `adduser` para criar o usuário `fulano`. Substitua `fulano` pelo nome de usuário desejado:

```
sudo adduser fulano
```

Isso criará um novo usuário e um diretório inicial em `/home/fulano`.

4. Defina uma senha para o novo usuário `fulano`:

```
sudo passwd fulano
```

Siga as instruções para definir a senha.

5. Agora, você deve adicionar o usuário `fulano` ao grupo `wheel`, que por padrão no CentOS, concede privilégios administrativos. Use o comando `usermod` para fazer isso:

```
sudo usermod -aG wheel fulano
```

6. Para verificar se o usuário `fulano` foi adicionado com sucesso ao grupo `wheel`, você pode usar o comando `groups`:

```
groups fulano
```

Você verá uma lista de grupos aos quais o usuário pertence, incluindo `wheel`.

7. Para permitir que o usuário `fulano` execute comandos com privilégios de superusuário, edite o arquivo de configuração `sudoers` com o comando `visudo`:

```
sudo visudo
```

Dentro do arquivo `sudoers`, encontre a linha que começa com `%wheel` e descomente-a (removendo o `"#"` no início da linha), se já não estiver descomentada. Deve ficar assim:

```
%wheel ALL=(ALL) ALL
```

Salve as alterações e saia do editor.

Agora, o usuário `fulano` tem privilégios administrativos e pode executar comandos com `sudo`. Certifique-se de que o usuário saiba como usar o comando `sudo` com segurança para evitar erros ou problemas de segurança.

Repare que o modelo de linguagem retorna não apenas o comando, mas também explicações detalhadas de como usá-lo de forma muito eficiente e assertiva.

8.4 Gerando modelos de dados

Vamos retomar as interações do tópico *Especificando requisitos funcionais* do capítulo anterior. Uma vez que temos um conjunto de requisitos definido, podemos usar o modelo de linguagem para implementar um modelo de dados para a aplicação, como mostra o exemplo a seguir:

Você: *Escreva o DDL para o PostgreSQL de um modelo físico de dados para este sistema.*

IA: Para criar um modelo físico de dados para o sistema de controle de estacionamento no PostgreSQL, você precisaria definir as tabelas, relacionamentos e restrições necessárias. Aqui está um exemplo simples de DDL (*Data Definition Language*) que pode servir como ponto de partida. Lembre-se de que esse é apenas um exemplo básico, e você pode precisar ajustar o modelo de acordo com os requisitos específicos e complexidades do seu sistema:

```
-- Tabela de Veículos
CREATE TABLE veiculos (
    id SERIAL PRIMARY KEY,
    placa VARCHAR(20) NOT NULL,
    data_entrada TIMESTAMP NOT NULL,
    vaga_atribuida INT
);

-- Tabela de Reservas
CREATE TABLE reservas (
    id SERIAL PRIMARY KEY,
    id_cliente INT NOT NULL,
    data_reserva TIMESTAMP NOT NULL,
    data_entrada TIMESTAMP NOT NULL,
    data_saida TIMESTAMP NOT NULL,
    vaga_reservada INT,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id),
    CHECK (data_reserva <= data_entrada AND data_entrada <=
data_saida)
);
```

(...) corte aplicado na resposta para uma melhor visualização

Lembre-se de que esse é apenas um esboço básico e pode precisar ser adaptado às necessidades específicas do seu sistema. Além disso, considere adicionar restrições adicionais, índices e outros elementos de acordo com os requisitos de desempenho e segurança do seu sistema. Este é um ponto de

partida para a criação do seu modelo físico de dados no PostgreSQL.

O próprio modelo de linguagem nos lembra de que é apenas um esboço, mas convenhamos que é um bom ponto de partida. E considere que, quanto mais detalhados forem os requisitos funcionais e técnicos, melhor será o resultado. E, ainda que precise de alterações, estas podem ser feitas textualmente, como em uma conversa com um colega. Você poderia, por exemplo, solicitar a alteração das chaves primárias para `BIGSERIAL` e também a implementação de tabelas de papéis e usuários para um controle de acesso mais aprimorado.

8.5 Obtendo instruções detalhadas

Ainda que você não domine uma ou outra tecnologia, você pode pedir por instruções detalhadas de criação e configuração de projetos em uma variedade incrível de plataformas e frameworks, como neste exemplo em Java Spring Boot com React:

Você (em continuação à conversa do exemplo anterior): *Me dê instruções detalhadas de como criar um projeto em Java springboot com React usando autenticação por token para o sistema discutido nesta conversa.*

IA: Para criar um projeto em Java Spring Boot com React usando autenticação por token, você precisará seguir algumas etapas. Vou guiá-lo por meio do processo passo a passo. Lembre-se de que este é um guia geral e os detalhes específicos podem variar com base nas ferramentas e bibliotecas que você escolher.

Etapas 1: Configurar o back-end (Spring Boot)

1. CONFIGURAÇÃO DO PROJETO SPRING BOOT:

- Inicie o Spring Initializr (<https://start.spring.io/>) ou use uma IDE, como o Spring Tool Suite (STS), para criar um projeto Spring Boot com as dependências necessárias, como Spring Web, Spring Security, Spring Data JPA, e PostgreSQL ou outro banco de dados de sua escolha.

2. CONFIGURAR O BANCO DE DADOS:

- Configure as propriedades do banco de dados (URL, nome de usuário, senha) no arquivo `application.properties` OU `application.yml`.

3. CONFIGURAR O SPRING SECURITY:

- Configure o Spring Security para autenticação por token. Você pode criar um filtro personalizado para lidar com a autenticação e autorização. Certifique-se de que todas as URLs relevantes estejam protegidas e exijam autenticação.

4. CRIAR ENTIDADES E REPOSITÓRIOS:

- Crie entidades Java para representar os dados do sistema (por exemplo, Usuário, Veículo, Reserva) e seus respectivos repositórios usando o Spring Data JPA.

5. IMPLEMENTAR AUTENTICAÇÃO E GERAÇÃO DE TOKENS:

- Implemente a lógica para autenticar usuários e gerar tokens JWT (*JSON Web Tokens*). Você pode usar a

biblioteca Spring Security JWT ou criar um filtro personalizado para isso.

6. **APIs RESTful:**

- Crie APIs RESTful usando controladores Spring MVC para realizar operações CRUD (*Create, Read, Update, Delete*) nas entidades do sistema.

7. **TESTAR O BACK-END:**

- Teste o back-end para garantir que a autenticação por token funcione corretamente e que você possa acessar as APIs protegidas com um token válido.

Etapas 2: Configurar o front-end (React)

(...) corte aplicado na resposta para uma melhor visualização

Lembre-se de que esse é um processo geral, e você precisará adaptá-lo às necessidades específicas do seu sistema e às ferramentas e bibliotecas que você escolher. Além disso, considere a segurança em todas as etapas e implemente práticas recomendadas de segurança de aplicativos da web para proteger os dados do seu sistema e a autenticação por token.

Obviamente haverá ajustes que precisam ser identificados e realizados, mas em geral as inscrições são bem eficientes. O importante é ter domínio sobre os fundamentos e conhecer bem a arquitetura proposta para tirar o maior proveito. Neste outro exemplo a seguir, rapidamente conseguimos um passo a passo sobre como desenvolver uma extensão para o Google Chrome:

Você: *Escreva uma extension para o Google Chrome que mostre qual o valor atual em R\$ do dólar americano e do bitcoin.*

IA: Criar uma extensão completa para o Google Chrome requer conhecimento de desenvolvimento web, JavaScript, HTML, CSS e a capacidade de acessar fontes de dados externas, como APIs financeiras, para obter informações atualizadas sobre o dólar americano e o Bitcoin. Abaixo, vou fornecer um exemplo simplificado de como você pode começar a criar uma extensão que exiba o valor atual em reais (R\$) do dólar americano e do Bitcoin usando a API do Mercado Bitcoin e a API de Cotações do Banco Central do Brasil. Lembre-se de que este é um exemplo básico e pode ser necessário fazer ajustes e melhorias para atender às suas necessidades específicas.

Passo 1: Crie a estrutura básica da extensão

1. Crie uma pasta para sua extensão e nomeie-a, por exemplo, `currency-tracker-extension` ;
2. Dentro dessa pasta, crie os seguintes arquivos:
 - `manifest.json` : esse arquivo contém informações sobre a extensão e suas permissões;
 - `popup.html` : esse arquivo será a interface da extensão;
 - `popup.js` : esse arquivo conterá o código JavaScript para buscar e exibir os valores do dólar e do Bitcoin.
 - `popup.css` : esse arquivo conterá estilos CSS para o `popup.html` .

Passo 2: Defina o `manifest.json`

O arquivo `manifest.json` deve conter informações sobre a extensão, incluindo permissões e scripts a serem executados. Aqui está um exemplo simples:

```
{
  "manifest_version": 2,
  "name": "Currency Tracker",
  "version": "1.0",
  "description": "Mostra o valor atual em R$ do dólar americano e do
```

```

Bitcoin.",
  "permissions": ["activeTab"],
  "browser_action": {
    "default_popup": "popup.html",
    "default_icon": {
      "16": "images/icon16.png",
      "48": "images/icon48.png",
      "128": "images/icon128.png"
    }
  },
  "icons": {
    "16": "images/icon16.png",
    "48": "images/icon48.png",
    "128": "images/icon128.png"
  },
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "js": ["popup.js"]
    }
  ],
  "permissions": [
    "https://api.mercadobitcoin.com.br/*",
    "https://olinda.bcb.gov.br/olinda/servico/PTAX/versao/v1/odata/"
  ]
}

```

Passo 3: Crie o `popup.html`

(...) corte aplicado na resposta para uma melhor visualização

Lembre-se de que esse é apenas um exemplo simples e pode ser necessário fazer ajustes e melhorias, como lidar com erros, atualizar os valores periodicamente e melhorar o estilo e a usabilidade da extensão. Além disso, este exemplo usa a API do Mercado Bitcoin, que pode estar sujeita a mudanças. Certifique-se de consultar a documentação da API e adaptar o código conforme necessário.

O modelo de linguagem fornece não apenas os códigos, mas também instruções detalhadas de como fazê-los. Imagine como pode ser rápido enriquecer sua solução com integração a ferramentas que seus usuários adoram! Outro exemplo é a execução de scripts de extensão ou customização.

8.6 Desenvolvendo plugins para múltiplas plataformas

Há uma infinidade de sistemas e ferramentas que permitem customização ou extensão por meio de plugins ou scripts, e o modelo de linguagem conhece quase todas.

O **Wondershare Filmora** é um editor de vídeo que possui um recurso excelente: remover de um vídeo trechos de silêncio, encurtando-o e deixando-o mais dinâmico (<https://filmora.wondershare.com.br/>). O **Davinci Resolve** é um editor bem mais completo, mas sem esse recurso (<https://www.blackmagicdesign.com/br/products/davinciresolve>). Porém, ele permite usar scripts para que o usuário programe seus próprios comandos e efeitos. Será que um modelo de linguagem conhece esse recurso? Sim, se bem treinado. Observe o exemplo a seguir, em que se pede para escrever um plugin para este editor de vídeo que nem é muito popular:

Você: *Escreva um script para editar um vídeo no davinci resolve cortando trechos em silêncio e explique como publicar esse script e usá-lo no software.*

IA: Para criar um script para editar um vídeo no DaVinci Resolve, você pode usar o recurso de edição de script Python disponível no DaVinci Resolve. Vou criar um script simples que corta trechos de silêncio em um vídeo. Antes de começar, certifique-se de que você tem o DaVinci Resolve instalado e está familiarizado com a linguagem de programação Python.

Passo 1: Criar o script Python

Aqui está um exemplo de script Python para cortar trechos de silêncio em um vídeo no DaVinci Resolve:

```
# Importar o módulo Fusion para trabalhar com composições
import DaVinciResolveScript as bmd

# Criar uma instância do aplicativo DaVinci Resolve
resolve = bmd.scriptapp("Resolve")

# Abrir o projeto atual
projectManager = resolve.GetProjectManager()
project = projectManager.GetCurrentProject()

# Acessar a timeline atual
timeline = project.GetCurrentTimeline()

# Acessar a composição atual
composition = timeline.GetCurrentVideoItem()

# Definir um valor de limiar de áudio para determinar o silêncio
# (ajuste conforme necessário)
threshold = -30

# Percorrer cada clipe na composição
for clip in composition.GetClips():
    # Acessar a faixa de áudio do clipe
    audioTrack = clip.GetAudioTrack()

    # Percorrer cada faixa de áudio no clipe
```

```
for audio in audioTrack.getItems():
    # Verificar se o áudio está abaixo do limiar
    if audio.GetLoudness() < threshold:
        # Obter a posição inicial e duração do trecho de silêncio
        startFrame = audio.GetStart()
        duration = audio.GetDuration()

        # Cortar o trecho de silêncio
        clip.SetTrimStart(startFrame + duration)

# Salvar o projeto
projectManager.SaveProject()
```

Passo 2: Executar o script no DaVinci Resolve

Agora que você tem o script, siga estas etapas para executá-lo no DaVinci Resolve:

1. Abra o DaVinci Resolve;
2. Carregue seu projeto de vídeo;
3. Vá para a aba "Fairlight" na parte inferior para acessar o módulo Fairlight;
4. Clique com o botão direito do mouse na faixa de áudio do clipe que deseja editar;
5. Selecione "Script" e, em seguida, clique em "Run Script";
6. Navegue até o local onde você salvou o script Python e selecione-o;
7. O script será executado, cortando os trechos de silêncio na faixa de áudio.

Lembre-se de ajustar o limiar de áudio (`threshold`) de acordo com suas necessidades específicas para determinar o que é considerado "silêncio" no seu vídeo.

Certifique-se de que o DaVinci Resolve tenha a opção de *scripting* habilitada em suas configurações. Isso permitirá que você execute scripts personalizados como o acima.

Para publicar seu script para uso posterior, você pode compartilhar o arquivo `.py` com outras pessoas que também possuem o DaVinci Resolve. Certifique-se de fornecer instruções sobre como executar o script. Você também pode compartilhar o script em fóruns e comunidades relacionadas ao DaVinci Resolve para que outros usuários possam aproveitá-lo.

8.7 Gerando fragmentos de código para construir uma solução completa

Analistas de sistemas podem usar o modelo de linguagem para gerar fragmentos de código com base em suas necessidades específicas e, em seguida, combinar esses fragmentos em uma solução completa e integrada. Isso pode ser feito por meio de prompts cuidadosamente elaborados, que orientam a geração de código de acordo com os requisitos e objetivos do projeto. Por exemplo, se uma analista precisa desenvolver uma funcionalidade específica em uma linguagem de programação, ela pode criar um prompt solicitando que o modelo de linguagem gere um fragmento de código que atenda a determinadas condições. Com base na resposta do modelo de linguagem, a analista pode refinar o prompt e solicitar outro fragmento de código para atender a outra parte do problema.

Por exemplo, podemos usar o modelo de linguagem para implementar, em sequência, as classes de um modelo que foi discutido e refinado em uma mesma conversa — aqui, em continuação à conversa do exemplo do tópico *Especificando requisitos funcionais*:

Você: *Escreva uma classe em Java para disponibilizar uma interface REST de acesso à tabela de veículos.*

IA: Para disponibilizar uma interface REST de acesso à tabela de Veículos em uma aplicação Java Spring Boot, você precisará criar um controlador REST que manipule as operações CRUD (*Create, Read, Update, Delete*) relacionadas aos veículos. Abaixo está um exemplo de classe Java que faz isso:

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/veiculos")
public class VeiculoController {

    private final VeiculoService veiculoService;

    @Autowired
    public VeiculoController(VeiculoService veiculoService) {
        this.veiculoService = veiculoService;
    }

    // Endpoint para listar todos os veículos
    @GetMapping
    public ResponseEntity<List<Veiculo>> listarVeiculos() {
        List<Veiculo> veiculos = veiculoService.listarVeiculos();
        return new ResponseEntity<>(veiculos, HttpStatus.OK);
    }

    (...) corte aplicado na resposta para uma melhor visualização

}
```

Nesse exemplo, a classe `VeiculoController` é um controlador Spring Boot que mapeia as operações CRUD para a entidade `Veiculo`. Certifique-se de que a classe `VeiculoService` esteja

corretamente implementada para realizar as operações no banco de dados. Além disso, você precisará definir a classe `Veiculo` que representa a entidade de veículo, incluindo suas propriedades e anotações JPA, se você estiver usando o Spring Data JPA para acesso ao banco de dados.

Certifique-se também de configurar adequadamente a autenticação e autorização, especialmente se desejar restringir o acesso a esses *endpoints* apenas a usuários autenticados. Além disso, você deve configurar a segurança do aplicativo Spring Boot para proteger seus *endpoints* de acordo com os requisitos do seu sistema.

Você (em continuação à conversa do exemplo anterior): *Agora, escreva a classe `Veiculo` para a `VeiculoController`.*

IA: A classe `Veiculo` representa a entidade de veículo no sistema. Ela deve conter propriedades que correspondam aos campos da tabela de veículos no banco de dados. Além disso, é importante adicionar anotações do Spring Data JPA para configurar o mapeamento entre a classe Java e a tabela no banco de dados. Aqui está um exemplo de como a classe `Veiculo` pode ser definida:

```
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import java.time.LocalDateTime;

@Entity
public class Veiculo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String placa;
    private LocalDateTime dataEntrada;
    private Integer vagaAtribuida;

    // Construtores, getters e setters

    public Veiculo() {
        // Construtor padrão necessário para o JPA
    }

    public Veiculo(String placa, LocalDateTime dataEntrada, Integer
vagaAtribuida) {
        this.placa = placa;
        this.dataEntrada = dataEntrada;
        this.vagaAtribuida = vagaAtribuida;
    }
}
```

```
// Getters e setters
```

```
public Long getId() {  
    return id;  
}  
  
public void setId(Long id) {  
    this.id = id;  
}
```

```
(...) corte aplicado na resposta para uma melhor visualização
```

```
}
```

Nesse exemplo, a classe `Veiculo` é uma entidade JPA com as seguintes propriedades:

- `id` : a chave primária autoincrementada;
- `placa` : a placa do veículo;
- `dataEntrada` : a data e hora de entrada do veículo;
- `vagaAtribuida` : o número da vaga atribuída ao veículo.

Você pode personalizar as propriedades da classe `Veiculo` de acordo com os campos da tabela de veículos no seu banco de dados. Certifique-se também de configurar adequadamente o `@Entity`, `@Id`, `@GeneratedValue`, bem como adicionar construtores, *getters* e *setters* para suas propriedades.

Observe que, em uma mesma conversa, o modelo de linguagem vai contextualizar; portanto, o ideal é trabalhar em sua solução utilizando uma única conversa. Caso seja necessário criar uma nova conversa, você pode resumir os requisitos e submeter os códigos já criados no início da conversa para retomar o contexto.

Dessa forma, ele pode construir uma solução completa, integrando esses fragmentos de código gerados pelo modelo de linguagem. Esse processo pode economizar tempo e aumentar a eficiência do desenvolvimento de software, permitindo que o(a) analista de

sistemas se concentre em outras tarefas importantes do projeto. No entanto, é importante lembrar que o modelo de linguagem é apenas uma ferramenta, e analistas devem sempre verificar e refinar os códigos gerados antes de implementá-los em um sistema em produção.

Uma vez codificado um sistema, será que um modelo de linguagem nos ajudaria a testar esses códigos?

CAPÍTULO 9

Prompts de apoio a testes e revisão de código

"Uma linguagem não faz seu código ser bom, programadores bons fazem seu código ser bom" — Samuel Custódio

Uma pessoa analista de sistemas pode utilizar modelos de linguagem para aprimorar os processos de teste e revisão de código em um novo sistema. O modelo pode ser programado para identificar inconsistências, erros de sintaxe ou falhas de segurança, fornecendo sugestões de correção imediatas.

Isso não só melhora a qualidade do código, mas também libera o analista para focar em testes de casos de uso mais complexos e na validação de requisitos funcionais, assegurando que o sistema atenda às expectativas dos usuários e às normas de qualidade.

9.1 Testando seu código-fonte

Você pode usar a IA para analisar o seu código de maneira surpreendente. Observe o exemplo a seguir, em que submeto um método com erro de lógica, mas sem erros de compilação:

Você: *Verifique se o método `verificarPar()` do código a seguir apresenta algum erro:*

```
function verificarPar() {  
    // Obtenha o valor do input  
    var numero = document.getElementById("numero").value;
```

```
// Verifique se o número é par
if (numero % 5 === 0) {
    document.getElementById("resultado").innerHTML = numero +
" é um número par.";
} else {
    document.getElementById("resultado").innerHTML = numero +
" não é um número par.";
}
}
```

IA: O código que você forneceu parece ter um erro na verificação de paridade. Atualmente, o código verifica se o número é par usando a expressão `numero % 5 === 0`, o que está verificando se o número é divisível por 5 em vez de verificar se é par. Para verificar se um número é par, você deve verificar se o resto da divisão por 2 é igual a 0. Aqui está a correção do código:

```
function verificarPar() {
    // Obtenha o valor do input
    var numero = document.getElementById("numero").value;

    // Verifique se o número é par
    if (numero % 2 === 0) {
        document.getElementById("resultado").innerHTML = numero + " é
um número par.";
    } else {
        document.getElementById("resultado").innerHTML = numero + "
não é um número par.";
    }
}
```

Com essa correção, o código verificará se o número é par corretamente ao verificar se o resto da divisão por 2 é igual a 0.

Eu sequer informei o que é esperado do método. Ele analisou o código para identificar seu propósito, indicou o erro de lógica e ainda

sugeriu uma solução. Podemos ir além e pedir para ele escrever rotinas de testes automatizados em várias linguagens, neste caso em JavaScript:

Você: *Escreva um test case em js para validar este código.*

IA: Para escrever um teste em JavaScript para validar a função `verificarPar()`, você pode usar uma estrutura de teste como o Mocha ou o Jest. Vou fornecer um exemplo de teste usando o Mocha.

Primeiro, certifique-se de que você tenha o Mocha instalado em seu ambiente de desenvolvimento:

```
npm install mocha --save-dev
```

Aqui está um teste simples que verifica se a função `verificarPar()` funciona corretamente:

```
// Importe a biblioteca de assert do Node.js
const assert = require('assert');

// Importe a função que deseja testar
const verificarPar = require('./seu-arquivo-com-o-codigo.js'); //
Substitua pelo caminho correto para o seu arquivo

// Descreva o conjunto de testes
describe('verificarPar', function () {
  // Descreva o caso de teste
  it('Deve retornar "é um número par." para número par', function ()
  {
    // Chame a função verificarPar com um número par
    const resultado = verificarPar(4);

    // Use assert para verificar se o resultado é o esperado
    assert.strictEqual(resultado, '4 é um número par.');
```

```
  });

  // Descreva outro caso de teste
  it('Deve retornar "não é um número par." para número ímpar',
function () {
    // Chame a função verificarPar com um número ímpar
    const resultado = verificarPar(7);

    // Use assert para verificar se o resultado é o esperado
```

```
    assert.strictEqual(resultado, '7 não é um número par.');
```

```
  });
```

```
});
```

Esse exemplo assume que você exportou a função `verificarPar` do seu arquivo de código em um formato compatível com a importação. Você precisará criar um arquivo separado que contenha a função `verificarPar`, ou você pode adaptar o código de acordo com a estrutura do seu projeto. Certifique-se de substituir `./seu-arquivo-com-o-codigo.js` pelo caminho correto para o seu arquivo de código.

Depois de configurar seu ambiente de teste, você pode executar os testes usando o comando `mocha` no terminal. Certifique-se de que todos os testes passam para verificar se a função `verificarPar()` funciona corretamente.

O modelo de linguagem pode ser útil para testes de projeto, ao permitir que analistas gerem casos de teste a partir de um prompt. Por exemplo, uma analista pode fornecer informações sobre um projeto e, em seguida, pedir ao modelo de linguagem que gere casos de teste para esse projeto. O modelo de linguagem pode fornecer uma lista de casos de teste, que a analista pode então revisar e refinar conforme necessário.

Além disso, o modelo de linguagem pode ajudar analistas a criarem scripts de teste automatizados. A analista pode fornecer informações sobre o que deseja testar, e o modelo de linguagem pode gerar um pseudocódigo que pode ser usado como base para escrever um script de teste automatizado.

É importante lembrar que os casos de teste gerados pelo modelo de linguagem podem precisar ser revisados e refinados para garantir que sejam adequados e eficazes para o projeto específico. No entanto, o modelo de linguagem pode ser uma ferramenta facilitadora para economizar tempo e aumentar a produtividade em seus projetos de teste.

9.2 Analisando a segurança do seu código

Na era da Engenharia de Prompt e IA, analistas de sistemas podem aumentar sua produtividade e eficiência ao incorporar essas tecnologias em suas práticas de análise de segurança de código. A integração de IA e modelos de linguagem no processo de verificação de segurança pode ajudar a identificar vulnerabilidades e falhas de forma mais rápida e eficaz. Nesta seção, abordaremos como analistas de sistemas podem utilizar a engenharia de prompt para aprimorar suas práticas de análise de segurança de código.

Análise estática de código

Analistas de sistemas podem utilizar modelos de linguagem e IA para aprimorar a análise estática de código. Ao criar prompts específicos para analisar o código-fonte, a IA pode verificar se há algum problema, como erros de sintaxe, problemas de estilo de codificação ou outras questões que uma ferramenta de análise estática de código poderia identificar. Por exemplo:

Você: *Analise o seguinte código Python em busca de erros de sintaxe, problemas de estilo e outras questões de qualidade de código. Forneça feedback sobre qualquer problema identificado e sugira possíveis melhorias:*

```
def calcular_soma(a, b):  
    resultado = a + b  
    return resultado  
  
numero1 = 10  
numero2 = 20  
soma = calcular_soma(numero1, numero2)  
print("A soma é:", soma)
```

Certifique-se de abordar qualquer erro ou preocupação relacionada à legibilidade, boas práticas de programação ou conformidade com PEP 8, se aplicável.

IA: O código que você forneceu parece estar bem escrito em termos de sintaxe e estilo de código, e está em conformidade com as boas práticas de programação. No entanto, aqui estão algumas observações e sugestões de melhoria:

1. **ESPAÇAMENTO E ESTILO PEP 8:** o código segue em grande parte o estilo PEP 8, que é o guia de estilo de código para Python. No entanto, PEP 8 recomenda duas linhas em branco antes de uma função, então você pode adicionar duas linhas em branco acima da definição da função `calcular_soma` para seguir a convenção PEP 8.

```
def calcular_soma(a, b):  
    resultado = a + b  
    return resultado  
  
numero1 = 10  
numero2 = 20  
soma = calcular_soma(numero1, numero2)  
print("A soma é:", soma)
```

2. **COMENTÁRIOS:** embora o código seja simples e autoexplicativo, é uma boa prática adicionar comentários para explicar partes do código mais complexas ou para documentar a função, especialmente se o código for mais extenso ou se outras pessoas forem colaborar com você.
3. **NOMES DE VARIÁVEIS:** os nomes de variáveis (`a`, `b`, `numero1`, `numero2`, `resultado`, `soma`) são descritivos o suficiente para entender o que cada variável representa, mas se o código real for mais complexo, é útil escolher nomes de variáveis mais descritivos.
4. **MANIPULAÇÃO DE ERROS:** Se a função `calcular_soma` fosse parte de um programa maior, seria aconselhável adicionar manipulação de erros para garantir que os argumentos `a` e `b` sejam números válidos (por exemplo, inteiros ou floats).

Aqui está o código com essas sugestões incorporadas:

```
# Função para calcular a soma de dois números
def calcular_soma(a, b):
    """
    Esta função calcula a soma de dois números.

    Args:
        a (float|int): O primeiro número.
        b (float|int): O segundo número.

    Returns:
        float|int: A soma dos dois números.
    """
    resultado = a + b
    return resultado

numero1 = 10
numero2 = 20

# Calcula a soma
soma = calcular_soma(numero1, numero2)

# Imprime o resultado
print("A soma é:", soma)
```

Lembre-se de que essas são apenas sugestões para melhorar a qualidade do código, e o código original já estava bem escrito em termos de conformidade com PEP 8 e com sintaxe Python correta.

Análise dinâmica de código

A análise dinâmica de código envolve a execução do código para identificar problemas em tempo de execução, como exceções, erros lógicos ou problemas de desempenho. O modelo de linguagem, por padrão, não tem a capacidade de executar código diretamente. No entanto, você pode fornecer um trecho de código que gostaria de

analisar em relação a possíveis problemas de lógica, desempenho ou fluxo de controle. Por exemplo:

Você: *Analise o seguinte trecho de código Python e forneça feedback sobre possíveis problemas de lógica, desempenho ou fluxo de controle:*

```
def calcular_media(valores):  
    total = 0  
    for valor in valores:  
        total += valor  
    media = total / len(valores)  
    return media  
  
notas = [85, 90, 78, 92, 88]  
  
media_notas = calcular_media(notas)  
  
if media_notas >= 90:  
    print("Parabéns! Sua média é excelente.")  
elif media_notas >= 70:  
    print("Sua média está boa.")  
else:  
    print("Sua média está abaixo da média.")
```

Gostaria de saber se há algum problema lógico ou de desempenho nesse código. Além disso, se você tiver sugestões para melhorar a legibilidade ou a eficiência do código, por favor, compartilhe.

IA: O trecho de código que você forneceu parece estar bem escrito em termos de lógica e desempenho, e segue um fluxo de controle apropriado. No entanto, aqui estão algumas observações e sugestões de melhoria em relação à legibilidade e eficiência:

1. **LEGIBILIDADE:** o código é bastante legível, mas você pode adicionar comentários para explicar a função da função `calcular_media` e do trecho que segue. Isso pode ser útil, especialmente se o código for mais complexo.
2. **USO DE FUNÇÃO:** o uso de uma função `calcular_media` é uma boa prática, pois permite reutilizar o código para calcular a média de diferentes conjuntos de valores. No entanto, você pode torná-lo mais geral permitindo que a função aceite uma lista de números, em vez de valores codificados diretamente. Isso tornaria a função mais flexível.

```
def calcular_media(valores):  
    if not valores:  
        return 0 # Lidar com a divisão por zero se a lista  
estiver vazia  
    total = sum(valores)  
    media = total / len(valores)  
    return media
```

Isso permite que você calcule a média de qualquer lista de números, não apenas `notas`.

3. **MANIPULAÇÃO DE ERROS:** o código não lida com a possibilidade de a lista `valores` estar vazia, o que levaria a uma divisão por zero. Adicione um tratamento para esse caso, como mostrado no exemplo anterior.
4. **SAÍDA DE MENSAGEM:** as mensagens impressas na tela são informativas, mas você pode torná-las mais amigáveis e personalizadas, incluindo a média calculada. Por exemplo:


```
if media_notas >= 90:
    print(f"Sua média é {media_notas:.2f}. Parabéns! Você tem um desempenho excelente.")
elif media_notas >= 70:
    print(f"Sua média é {media_notas:.2f}. Sua média está boa.")
else:
    print(f"Sua média é {media_notas:.2f}. Sua média está abaixo da média.")
```

Isso imprimirá a média com duas casas decimais.

Em resumo, o código é funcional e logicamente correto. As sugestões de melhoria são principalmente para tornar o código mais flexível e legível, além de tratar possíveis casos de erro.

Perceba o quanto a engenharia de prompt aplicada aos modelos de linguagem oferece um enorme potencial para aprimorar a análise de segurança do código e aumentar a produtividade dos analistas de sistemas.

Vimos como usar um modelo de linguagem para projetar um sistema, codificar partes e testá-lo. Mas não podemos esquecer a documentação, podemos?

CAPÍTULO 10

Prompts de apoio à documentação

"O que é mais difícil não é escrever muito; é dizer tudo, escrevendo pouco" — Júlio Dantas

Uma analista de sistemas pode utilizar modelos de linguagem como uma ferramenta para melhorar e agilizar a criação de documentações de sistemas. Ao inserir comandos específicos, o modelo é capaz de produzir descrições completas sobre as funcionalidades, procedimentos e estrutura do sistema. Isso inclui o fornecimento de exemplos práticos e clarificação de aspectos técnicos.

A vantagem dessa prática vai além da economia de tempo: assegura a consistência e a compreensibilidade das informações, enriquecendo o conhecimento do analista com aprendizado contínuo e fornecendo orientação virtual. Com isso, o analista não só economiza tempo, mas também se beneficia do enriquecimento profissional através da interação e atualização constantes proporcionadas pelo modelo de linguagem.

10.1 Documentando seu projeto

Um analista de sistemas pode usar a IA para auxiliar na documentação de projetos de várias maneiras. A IA pode ajudar a gerar, revisar e manter a documentação, tornando o processo mais eficiente e preciso. Aqui estão algumas maneiras pelas quais um analista de sistemas pode utilizar a IA para documentar projetos:

- **Geração automática de documentação técnica:** a IA pode analisar o código-fonte e gerar documentação básica com base em padrões de nomenclatura e estrutura do código. Isso pode incluir a criação de *docstrings* (do Python) ou *JavaDoc* (do Java) para funções e classes, bem como a geração de comentários explicativos;
- **Revisão e melhoria da documentação existente:** a IA pode analisar a documentação existente e sugerir melhorias na gramática, estilo, clareza e consistência. Isso pode ajudar a garantir que a documentação seja fácil de entender e esteja atualizada;
- **Manutenção da documentação:** à medida que o projeto evolui, a IA pode ajudar a identificar partes da documentação que estejam desatualizadas ou inconsistentes com o código-fonte atual. Isso ajuda a manter a documentação correta e relevante ao longo do ciclo de vida do projeto. Confira um exemplo a seguir:

Você: *Revise a classe Java a seguir, atualizando o JavaDoc:*

```
/**
 * Classe para gerenciar informações de usuários.
 * Atualmente, ela é usada para calcular impostos.
 */
public class UserManager {

    private double salary;

    /**
     * Construtor que define o salário do usuário.
     * @param salary Salário do usuário.
     */
    public UserManager(double salary) {
        this.salary = salary;
    }
}
```

```
/**
 * Calcula e retorna o desconto do usuário.
 * Este método agora calcula o imposto com base no salário.
 * @return O desconto do usuário.
 */
public double calculateDiscount() {
    return salary * 0.1; // Calcula 10% de imposto sobre o salário
}

// Outros métodos relacionados à gestão de usuários
}
```

IA: Vamos revisar e atualizar o JavaDoc da classe `UserManager` para refletir com precisão sua funcionalidade atual, que está focada no cálculo de impostos baseados no salário do usuário.

```
/**
 * Classe para o cálculo de impostos baseados no salário.
 * Esta classe fornece funcionalidades para calcular um valor fixo de
 imposto
 * sobre o salário de um usuário.
 */
public class UserManager {

    private double salary;

    /**
     * Construtor que inicializa a instância com o salário do usuário.
     * Este salário é utilizado como base para o cálculo de impostos.
     *
     * @param salary Salário do usuário, usado no cálculo do imposto.
     */
    public UserManager(double salary) {
        this.salary = salary;
    }

    /**
     * Calcula e retorna o imposto com base no salário do usuário.
     * Atualmente, o método aplica uma taxa fixa de 10% sobre o
 salário.
     *
     * @return O valor do imposto calculado como 10% do salário.
     */
    public double calculateDiscount() {
        return salary * 0.1; // Calcula 10% de imposto sobre o salário
    }

    // Outros métodos relacionados podem ser adicionados no futuro.
}
```

Nesta versão revisada, os comentários estão alinhados com a funcionalidade atual da classe, que é o cálculo de impostos com base no salário. Os JavaDocs do construtor e do método `calculateDiscount()` foram atualizados para descrever de forma clara e precisa seu propósito e uso.

- **Geração de exemplos de código e uso:** a IA pode gerar exemplos de código e ilustrar como as funções, classes e módulos podem ser usados em diferentes cenários. Isso ajuda a tornar a documentação mais útil e informativa para desenvolvedores que trabalham com o projeto;
- **Conversão da documentação em diferentes formatos:** a IA pode converter a documentação em diferentes formatos, como HTML, PDF, Markdown ou reStructuredText, facilitando a criação e a manutenção de documentos conforme o necessário;
- **Sumarização e organização do conteúdo:** a IA pode ajudar a organizar e resumir o conteúdo da documentação, facilitando a localização de informações relevantes e a compreensão geral do projeto;
- **Integração com ferramentas de gerenciamento de projetos:** a IA pode ser integrada a ferramentas de gerenciamento de projetos, como Jira, Trello ou GitHub, para ajudar a rastrear e gerenciar a documentação ao longo do desenvolvimento do projeto. Por exemplo:

Você: *Escreva um comando em `curl` para atualizar o status de uma issue no jira. A minha url é `mydomain.atlassian.net` , o id da issue é `ABC-42` , o id do status da conclusão é `42` , meu e-mail é `eu@domain.com.br` e meu token é `xxx-yyy-zzz-256` .*

IA: Para atualizar o status de uma issue no Jira usando o comando `curl`, você precisa primeiramente converter a combinação do seu e-mail e do token de API para *Base64*. Depois, use essa string codificada no cabeçalho de autenticação do comando `curl`. Aqui está o comando completo:

1. **CONVERTER E-MAIL E TOKEN PARA BASE64:** Abra o terminal e execute o seguinte comando para codificar suas credenciais em Base64. Substitua `eu@domain.com.br` e `XXX-YYY-ZZZ-256` pelas suas informações reais.

```
echo -n "eu@domain.com.br:XXX-YYY-ZZZ-256" | base64
```

Isso vai retornar uma string codificada, que você usará no comando `curl`.

2. **COMANDO `CURL` PARA ATUALIZAR O STATUS DA ISSUE:** Utilize o resultado da codificação no lugar de `[YourEncodedCredentials]`. Substitua `[YourEncodedCredentials]` pela string codificada obtida no passo anterior.

```
curl --request POST \
  --url 'https://mydomain.atlassian.net/rest/api/3/issue/ABC-42/transitions' \
  --header 'Authorization: Basic [YourEncodedCredentials]' \
  --header 'Content-Type: application/json' \
  --data '{
    "transition": {
      "id": "42"
    }
  }'
```

Esse comando enviará uma solicitação `POST` para a API do Jira, pedindo a mudança de status para a issue `ABC-42` para o status com ID `42`.

Lembre-se de ter as permissões necessárias no Jira para realizar essa operação.

10.2 Automatizando publicações

A Inteligência Artificial (IA) pode ser uma ferramenta valiosa para ajudar analistas de sistemas a otimizar e melhorarem os processos de integração contínua (CI) e entrega contínua (CD). Aqui estão algumas maneiras pelas quais a IA pode ser usada para melhorar os processos de CI/CD:

- **Análise de logs e relatórios:** a IA pode analisar logs e relatórios de erros para identificar padrões e problemas recorrentes. Isso permite que os analistas de sistemas priorizem e resolvam problemas de maneira mais eficaz, melhorando a qualidade geral do código;
- **Manutenção de *pipelines* de CI/CD:** a IA pode ajudar a gerenciar e monitorar *pipelines* de CI/CD, identificando gargalos e áreas de melhoria. Isso pode ajudar a otimizar os processos de CI/CD e garantir que eles sejam executados de maneira eficiente. Podemos, por exemplo, questionar a qualidade de um fluxo de implantação e ainda receber sugestões de melhoria;
- **Monitoramento de tarefas:** a IA pode ser integrada a ferramentas de gerenciamento de projetos, como Jira, Trello ou GitHub, para ajudar a rastrear e gerenciar tarefas relacionadas ao CI/CD. Isso pode melhorar a comunicação entre os membros da equipe e garantir que os processos de CI/CD sejam executados de maneira eficiente. O ChatGPT, por exemplo, possui uma API que pode ser usada para realizar essas integrações de forma muito simples;
- **Planejamento de recursos:** a IA pode ajudar a prever e planejar recursos necessários para os processos de CI/CD, como tempo de desenvolvimento, infraestrutura e mão de obra. Isso pode ajudar a garantir que os recursos adequados estejam disponíveis e sejam utilizados de maneira eficiente.

10.3 Mantendo código-fonte

Na manutenção de código-fonte, a Inteligência Artificial serve como uma ferramenta poderosa para incrementar a eficiência e minimizar erros, contribuindo significativamente em várias frentes. Como visto neste capítulo e nos anteriores, ela agiliza a detecção precoce de falhas e vulnerabilidades, além de analisar e sugerir melhorias na qualidade do código, promovendo práticas de codificação otimizadas e alinhadas com padrões vigentes. Além disso, a IA facilita a refatoração do código, orientando modificações que reforçam sua clareza e funcionalidade e, por conseguinte, ajudam no crescimento e sustentabilidade do projeto.

A identificação de código duplicado e a análise das dependências do sistema também se tornam tarefas menos árduas com a IA, que sinaliza redundâncias para eliminação e alerta para possíveis incompatibilidades em bibliotecas e módulos dependentes. No que tange à garantia de qualidade, a Inteligência Artificial é essencial na automatização de testes, desenvolvendo e aplicando testes que cobrem eficazmente o espectro do código. E não para por aí: a geração e atualização de documentação técnica também são otimizadas pela IA, que traduz o código em descrições úteis para uso corrente e futuro.

Por fim, a capacidade da IA de compreender o histórico de manutenção permite a priorização inteligente de tarefas, assegurando que a atenção dos analistas se centre nos pontos mais críticos. Em suma, a integração da IA nos processos de manutenção não só melhora o estado atual dos sistemas, mas também prepara o terreno para futuras atualizações e inovações.

CAPÍTULO 11

Modelos de Linguagem de Código (Code Language Models)

"...Acredito que vai demorar um pouco mais até que o Copilot ofereça um verdadeiro impulso de produtividade. Mas estou convencido de que isso está chegando." — Colin Eberhardt

Dentre os modelos de linguagem, há aqueles treinados especificamente para interpretar e escrever códigos-fonte: os **Modelos de Linguagem de Código** (*Code Language Models*). Eles representam uma síntese entre a programação e a Inteligência Artificial, fundindo a precisão do código-fonte com a flexibilidade dos modelos de linguagem baseados em IA.

Essencialmente, um Modelo de Linguagem de Código é treinado com ênfase em códigos de linguagens formais e documentações técnicas. Esses modelos são alimentados por uma enorme quantidade de repositórios de código, abrangendo múltiplas linguagens de programação e paradigmas, o que lhes permite oferecer suporte abrangente a desenvolvedores. Desde o autocomplemento de código até a identificação de bugs, passando pela geração de novas linhas de código, eles atuam como ferramentas mais aprimoradas que os modelos de linguagem genéricos e, portanto, podem aumentar a eficiência e a qualidade do desenvolvimento de software.

Neste capítulo, exploraremos os Modelos de Linguagem de Código, desde a sua concepção até suas aplicações práticas, destacando como eles estão moldando o futuro da programação e do desenvolvimento de software.

Origens e desenvolvimento

A evolução dos Modelos de Linguagem de Código acompanha o avanço dos Modelos de Linguagem Geral (LLMs). Contudo, eles se distinguem pela concentração em *datasets* de código-fonte. Exemplos notáveis incluem as variações de modelos como GPT-3 e GPT-4, adaptados para melhor interação com código. Sendo uma especialização dentro do espectro de *modelos de linguagem*, os Modelos de Linguagem de Código são especificamente desenvolvidos para entender e manipular código-fonte. Essa especialização permite uma abordagem mais focada e eficaz no trato com linguagens de programação, padrões de codificação e estruturas de dados.

Esses modelos, embora baseados em arquiteturas Transformer semelhantes aos LLMs gerais, são afinados para captar as nuances e complexidades das linguagens de programação. Eles não só geram código, mas também fornecem insights relevantes para aprimoramento e correção de código, operando em conjunto com IDEs e outras ferramentas de desenvolvimento. Essa especificidade os torna adeptos em padrões de codificação, elevando sua eficácia em aplicações práticas de desenvolvimento de software.

Ferramentas e aplicações

Várias ferramentas se destacam por suas capacidades e aplicações únicas. O **GitHub Copilot**, desenvolvido em colaboração entre GitHub e OpenAI, oferece sugestões de código em tempo real e é compatível com diversos ambientes de desenvolvimento integrado (IDEs). Já o **OpenAI Codex**, também da OpenAI, é conhecido por sua eficácia em Python, apesar de suportar múltiplas linguagens, e foi treinado em bilhões de linhas de código.

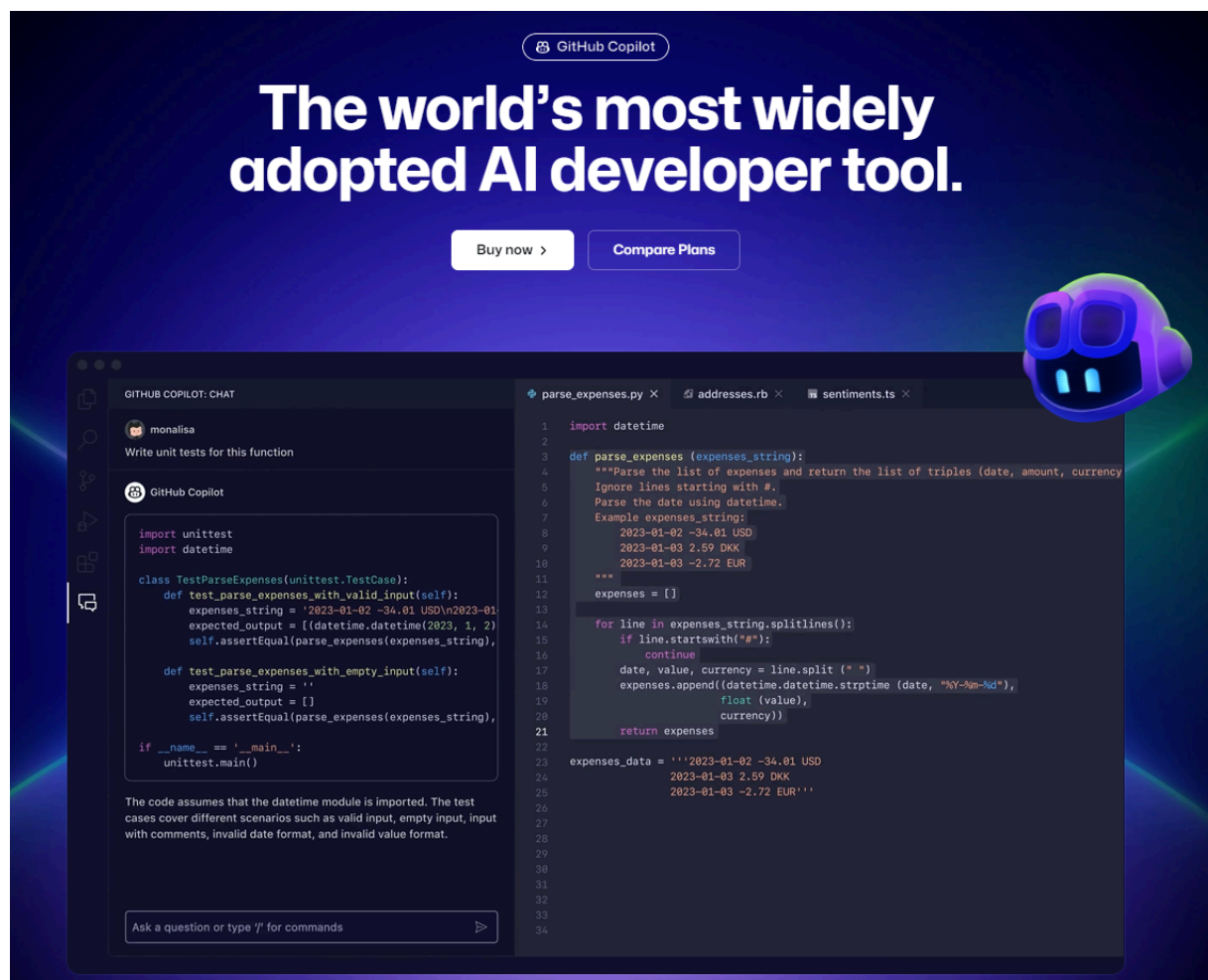


Figura 11.1: GitHub Copilot

Outra solução notável é o **Code Intelligence**, que une testes dinâmicos com IA e autoaprendizagem para encontrar bugs e vulnerabilidades, suportando linguagens como Java, C/C++, Golang e JavaScript. O **ChatGPT** da OpenAI, embora primariamente um chatbot, também oferece funcionalidades de geração de código. O **Visual Studio IntelliCode** da Microsoft, um plugin para o editor Visual Studio Code, destaca-se por suas sugestões de código inteligentes baseadas no contexto.

O **Tabnine**, lançado em 2013, utiliza algoritmos de aprendizado profundo para prever a intenção de codificação do usuário, suportando uma ampla gama de linguagens. O **CodeT5** é uma solução de código aberto que auxilia na geração rápida de código

em várias linguagens, enquanto o **PyCharm** da JetBrains é um IDE focado em Python, conhecido por sua precisão na previsão de código. A lista se estende com ferramentas como **DeepCode** da Snyk, que oferece análise de código na nuvem, e **CodeWP**, um gerador de código para WordPress.

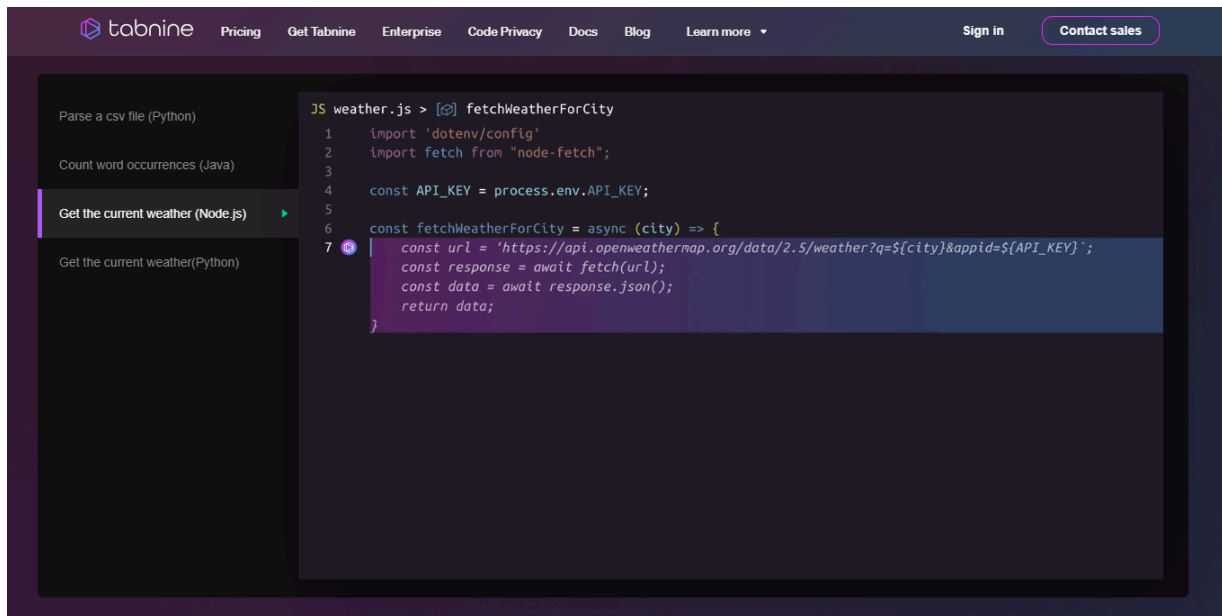


Figura 11.2: Tabnine

Essas ferramentas exemplificam a diversidade e a profundidade dos modelos de linguagem de código no mercado atual, cada uma contribuindo de maneira única para melhorar a eficiência e a qualidade do desenvolvimento de software.

11.1 GitHub Copilot

Dentro do escopo dos Modelos de Linguagem de Código, o GitHub Copilot destaca-se como uma aplicação prática e avançada desses modelos. Integrado às IDEs, o Copilot utiliza o modelo GPT-4 para fornecer assistência inteligente durante o processo de codificação, aumentando a eficiência e a precisão da pessoa desenvolvedora. No

momento em que escrevo este livro, o GitHub Copilot oferece planos diferentes para diversos perfis de uso. O individual é gratuito para estudantes e professores verificados, bem como para mantenedores de projetos populares de código aberto.

Para exemplificar o uso, observe a seguir como desenvolver um aplicativo do tipo "*To-Do List*" com HTML e JavaScript. O primeiro passo é usar o prompt de comando do GitHub Copilot, já instalado no Visual Studio Code, para criar um projeto:

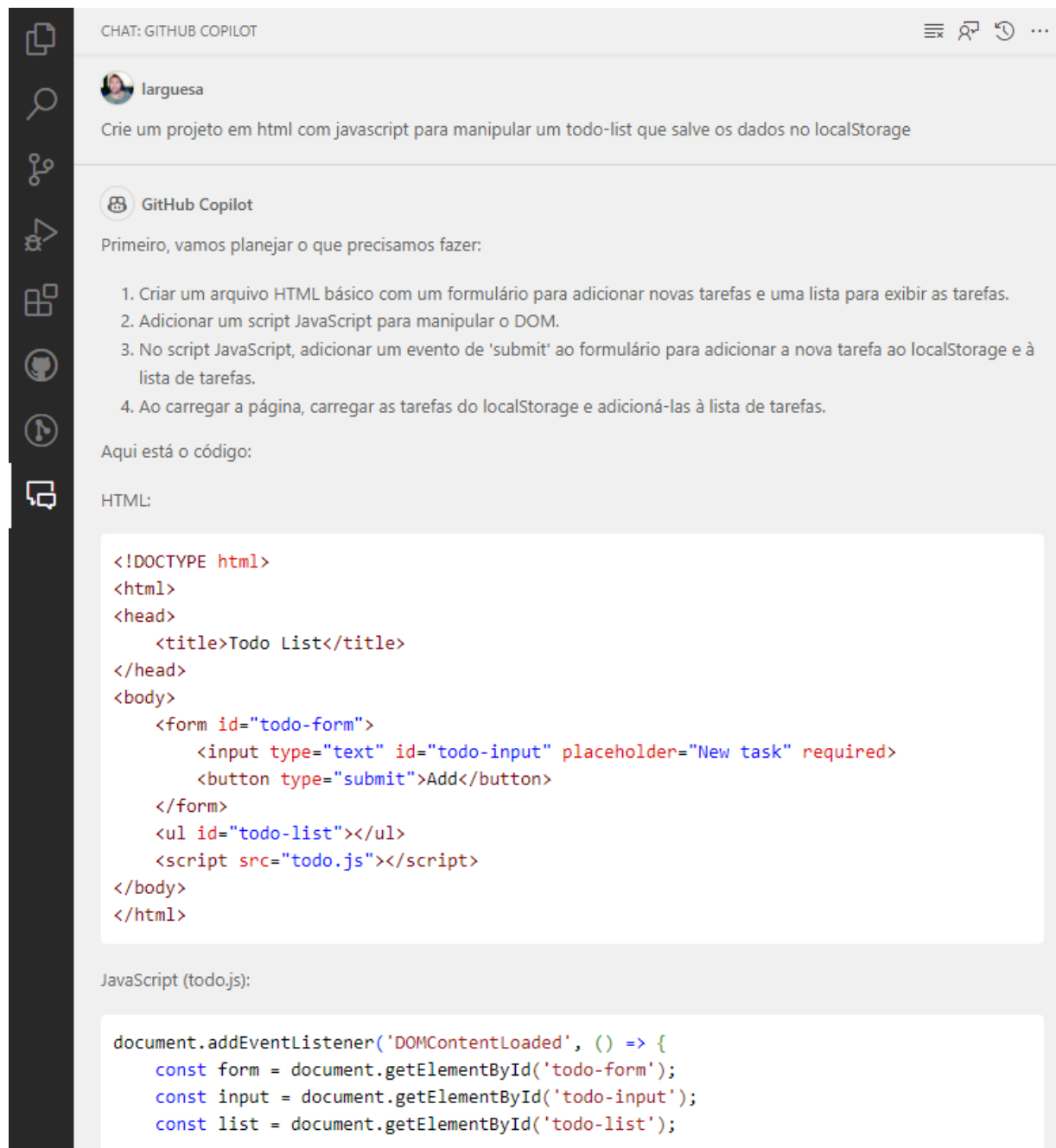


Figura 11.3: Usando o prompt de comando para criar o projeto

Veja que, assim como no ChatGPT e outros modelos de linguagem, ele apresenta quadros com códigos para o HTML e para o Javascript. Podemos usar a própria interface integrada a uma IDE para criar os arquivos:

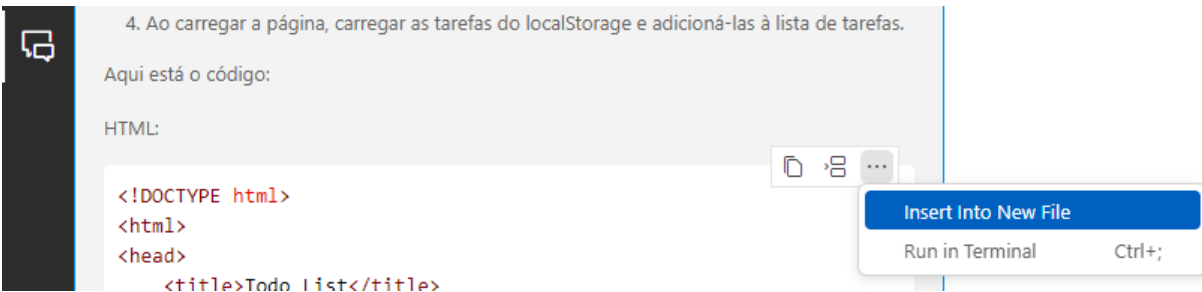


Figura 11.4: Criando os arquivos

A pessoa programadora precisa nomear os arquivos observando os nomes referenciados, algo que o Copilot ainda não faz sozinho:

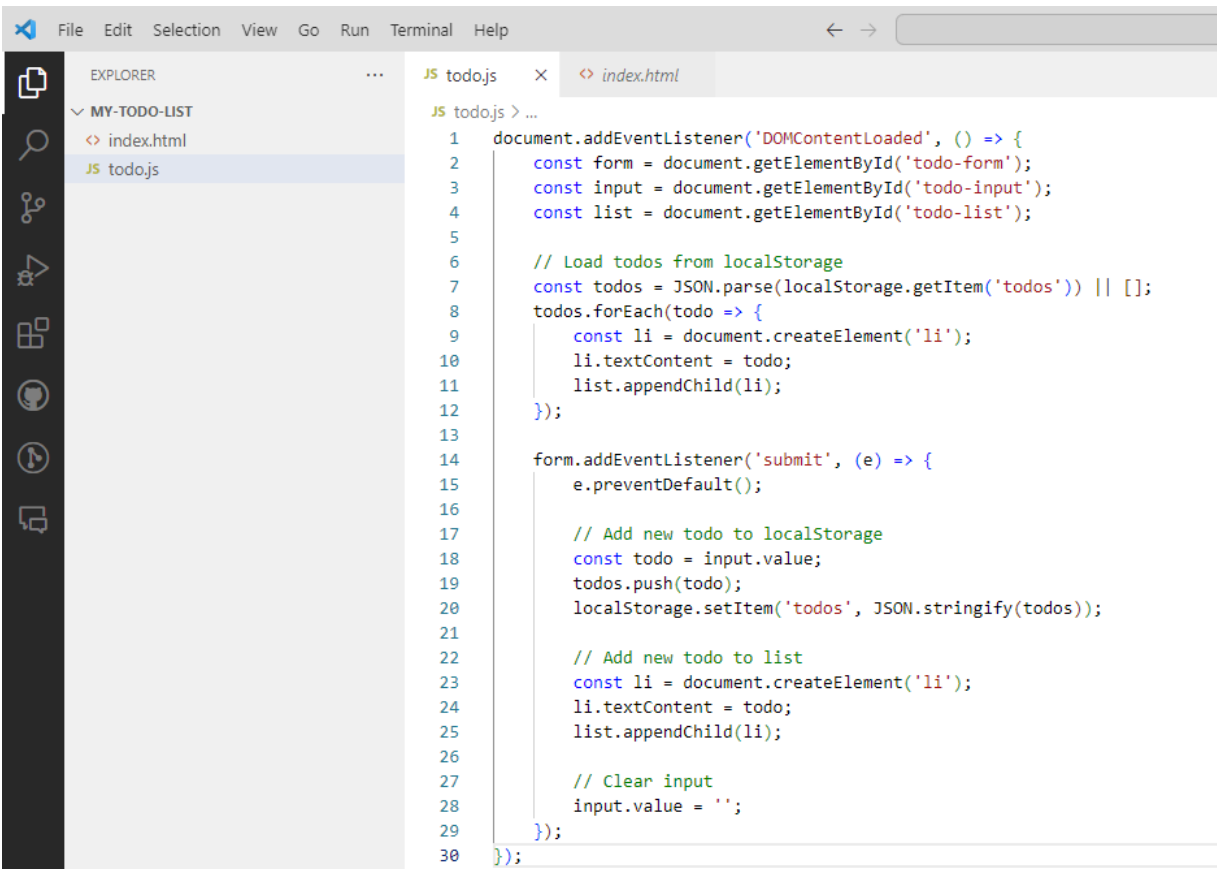


Figura 11.5: Arquivo todo.js

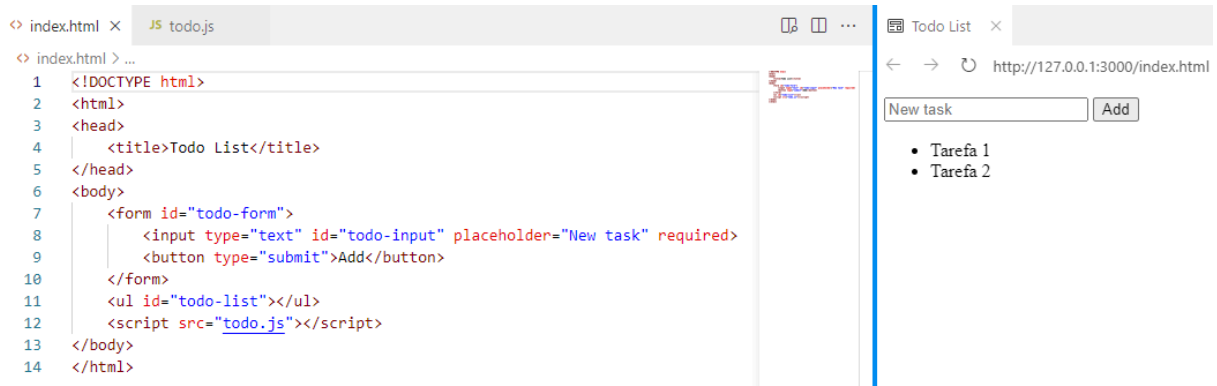
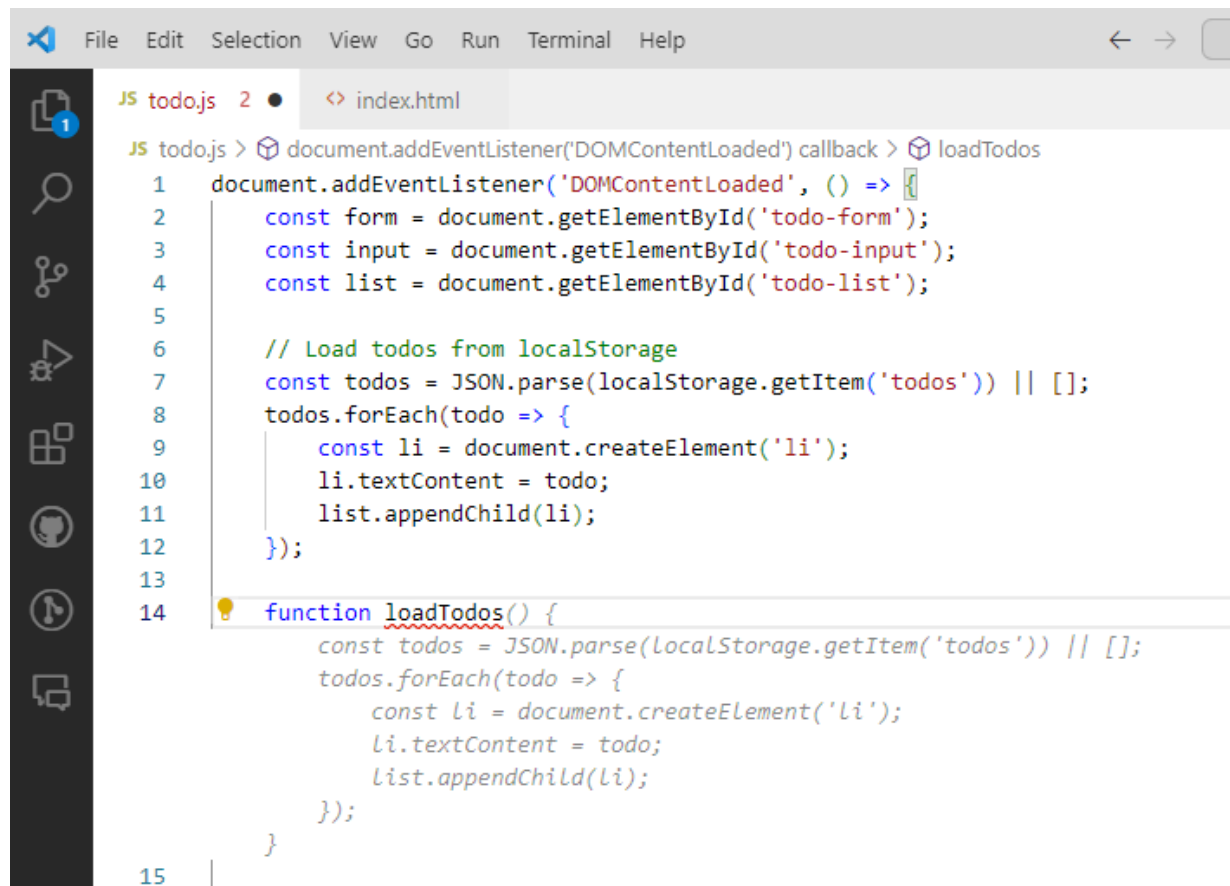


Figura 11.6: Arquivo index.html

Os códigos nunca são gerados da mesma forma. Se você executar a mesma ação no seu computador, observará um resultado diferente. Neste caso, ele gerou um código nada modularizado, o que é ótimo para que eu possa demonstrar a sugestão de código. Observe a seguir que, se a pessoa programadora iniciar a criação de uma função separada para carregar os dados da lista, ele dará como sugestão o código completo da função:



```
JS todo.js 2 ● <> index.html
JS todo.js > document.addEventListener('DOMContentLoaded') callback > loadTodos
1 document.addEventListener('DOMContentLoaded', () => {
2   const form = document.getElementById('todo-form');
3   const input = document.getElementById('todo-input');
4   const list = document.getElementById('todo-list');
5
6   // Load todos from localStorage
7   const todos = JSON.parse(localStorage.getItem('todos')) || [];
8   todos.forEach(todo => {
9     const li = document.createElement('li');
10    li.textContent = todo;
11    list.appendChild(li);
12  });
13
14  function loadTodos() {
15    const todos = JSON.parse(localStorage.getItem('todos')) || [];
16    todos.forEach(todo => {
17      const li = document.createElement('li');
18      li.textContent = todo;
19      list.appendChild(li);
20    });
21  }
```

Figura 11.7: Sugestão do código da função 'loadTodos'

Podemos fazer o mesmo para a função `saveTodos`, como se observa a seguir:

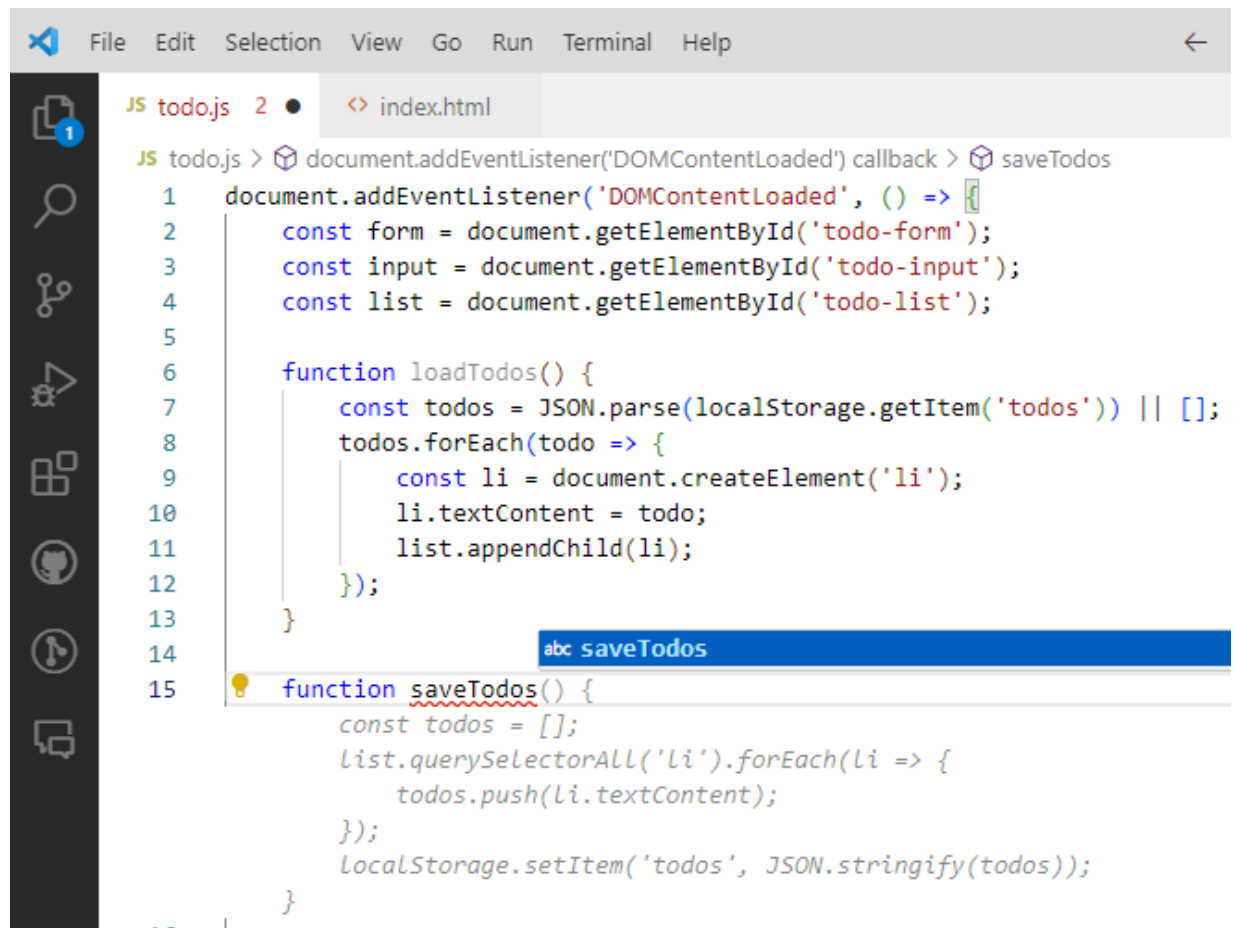
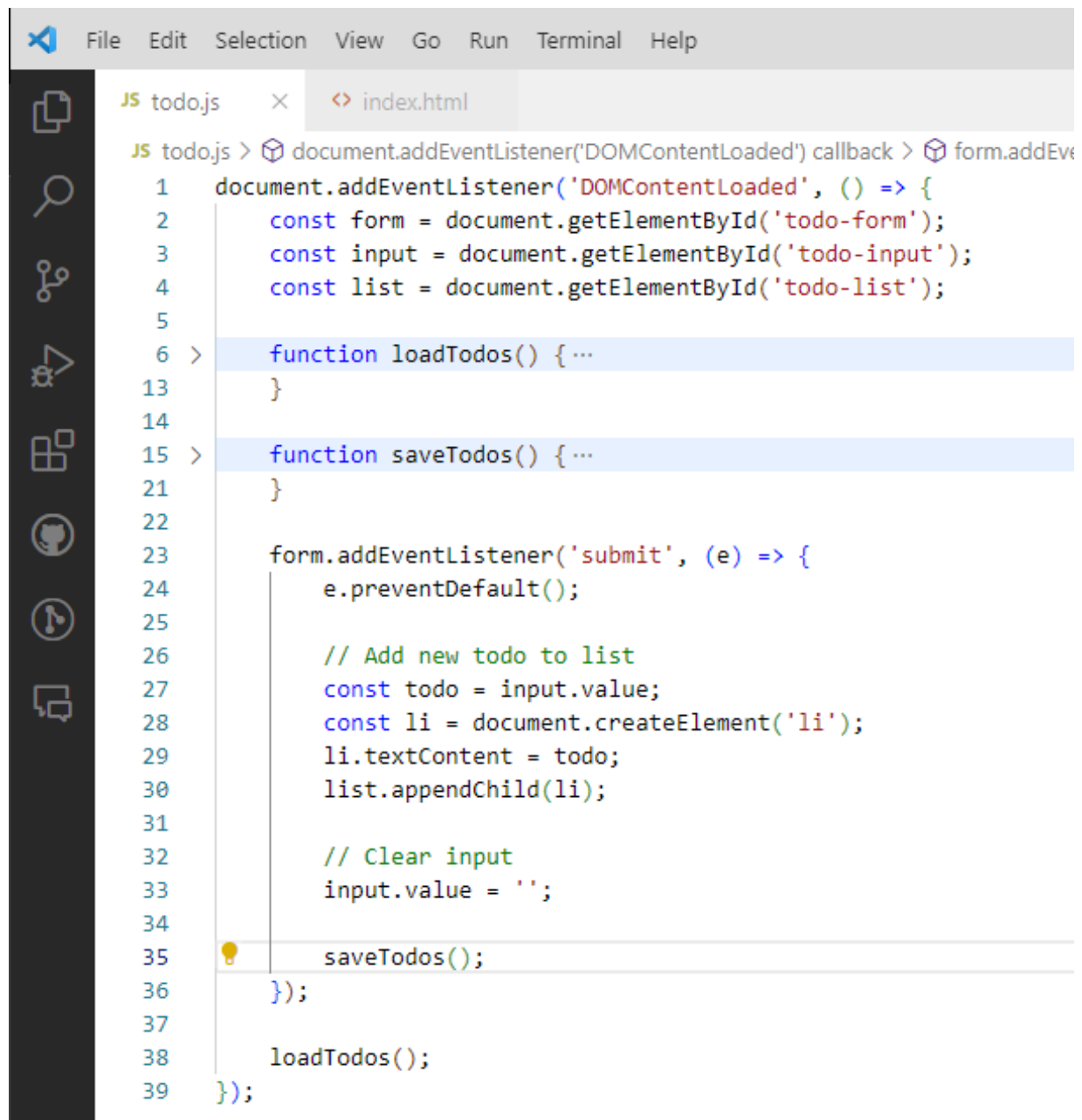


Figura 11.8: Sugestão do código da função 'saveTodos'

O Copilot não vai implementar os comandos que invocam os métodos, pelo menos não quando usamos as sugestões. O programador tem de fazê-lo:



```
File Edit Selection View Go Run Terminal Help

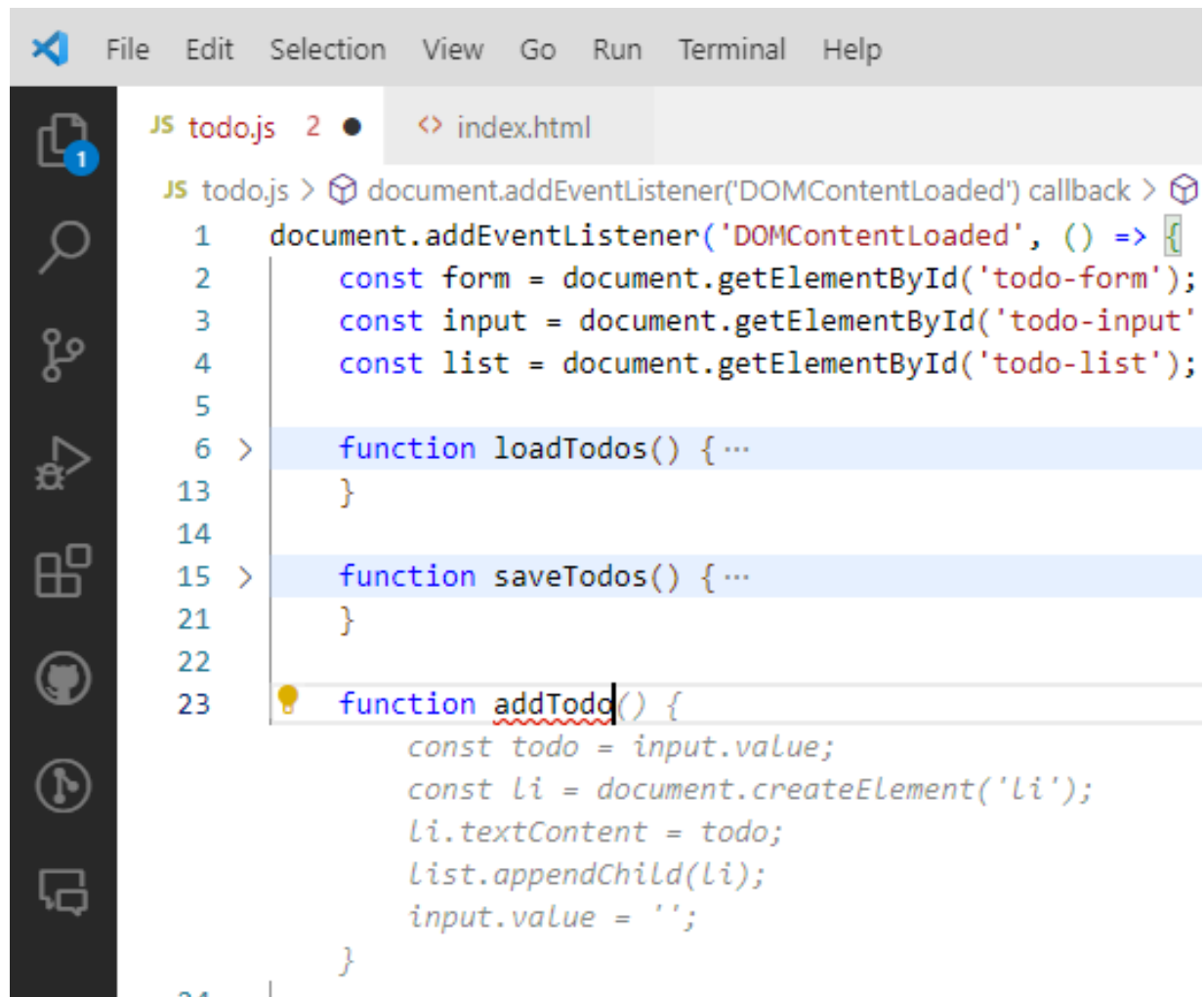
JS todo.js x <> index.html

JS todo.js > document.addEventListener('DOMContentLoaded') callback > form.addEv

1 document.addEventListener('DOMContentLoaded', () => {
2     const form = document.getElementById('todo-form');
3     const input = document.getElementById('todo-input');
4     const list = document.getElementById('todo-list');
5
6     > function loadTodos() { ...
13 }
14
15 > function saveTodos() { ...
21 }
22
23     form.addEventListener('submit', (e) => {
24         e.preventDefault();
25
26         // Add new todo to list
27         const todo = input.value;
28         const li = document.createElement('li');
29         li.textContent = todo;
30         list.appendChild(li);
31
32         // Clear input
33         input.value = '';
34
35         saveTodos();
36     });
37
38     loadTodos();
39 });
```

Figura 11.9: Invocando os métodos 'loadTodos' e 'saveTodos'

Podemos criar a função `addTodo`, e também obtemos a sugestão já com o código implementado:



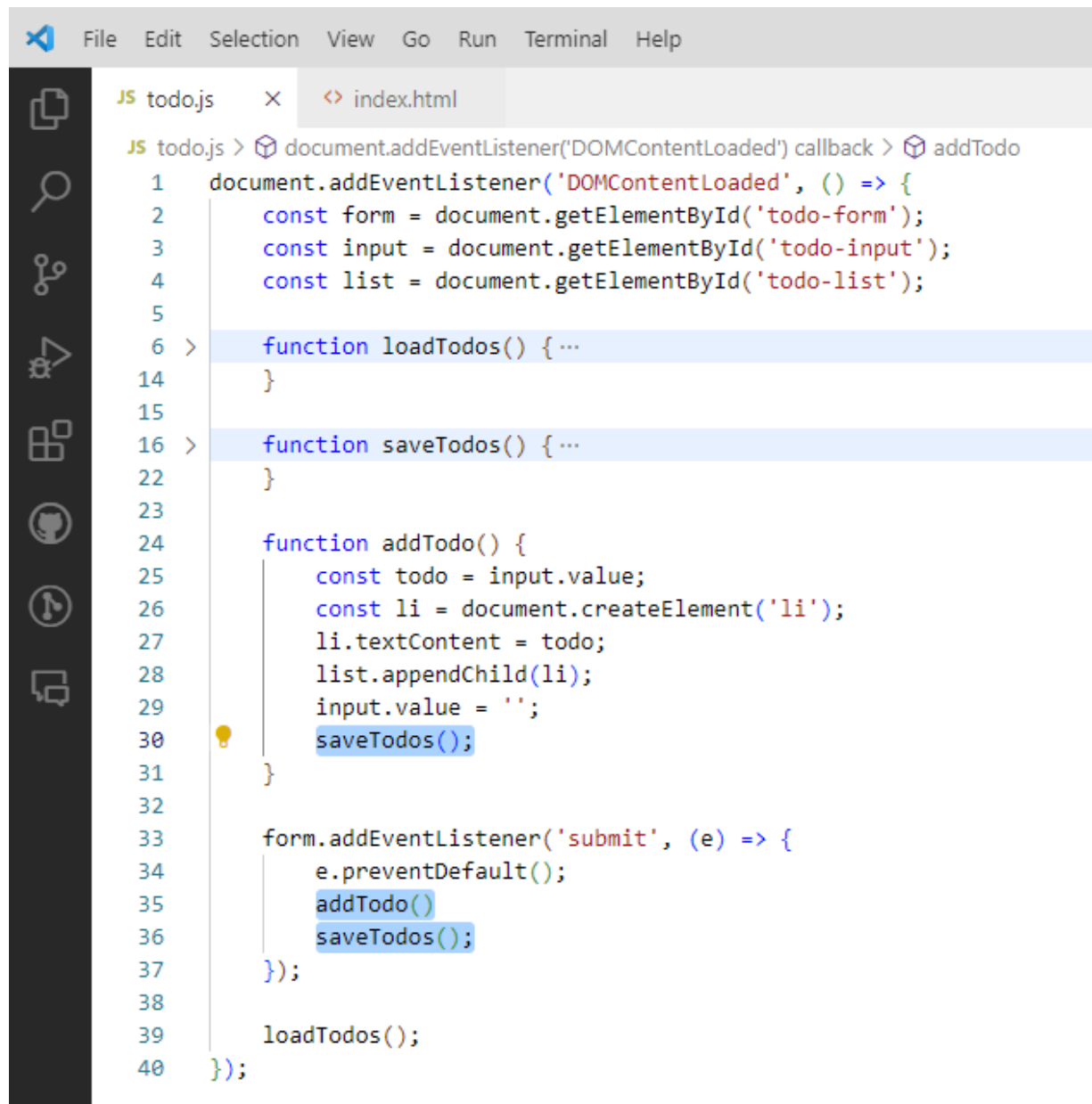
The image shows a screenshot of the Visual Studio Code editor interface. The top menu bar includes 'File', 'Edit', 'Selection', 'View', 'Go', 'Run', 'Terminal', and 'Help'. The editor has two tabs open: 'JS todo.js' and 'index.html'. The 'JS todo.js' tab is active, showing a JavaScript file with the following code:

```
JS todo.js > document.addEventListener('DOMContentLoaded') callback >
1 document.addEventListener('DOMContentLoaded', () => {
2     const form = document.getElementById('todo-form');
3     const input = document.getElementById('todo-input');
4     const list = document.getElementById('todo-list');
5
6     > function loadTodos() { ...
13 }
14
15 > function saveTodos() { ...
21 }
22
23 function addTodo() {
    const todo = input.value;
    const li = document.createElement('li');
    li.textContent = todo;
    list.appendChild(li);
    input.value = '';
}
```

The code is numbered from 1 to 23. The function 'addTodo' is being defined on line 23, and the editor is suggesting the function signature 'function addTodo()' with a lightbulb icon next to it. The function body includes creating a new list item, appending it to the list, and clearing the input field.

Figura 11.10: Sugestão da função 'addTodo'

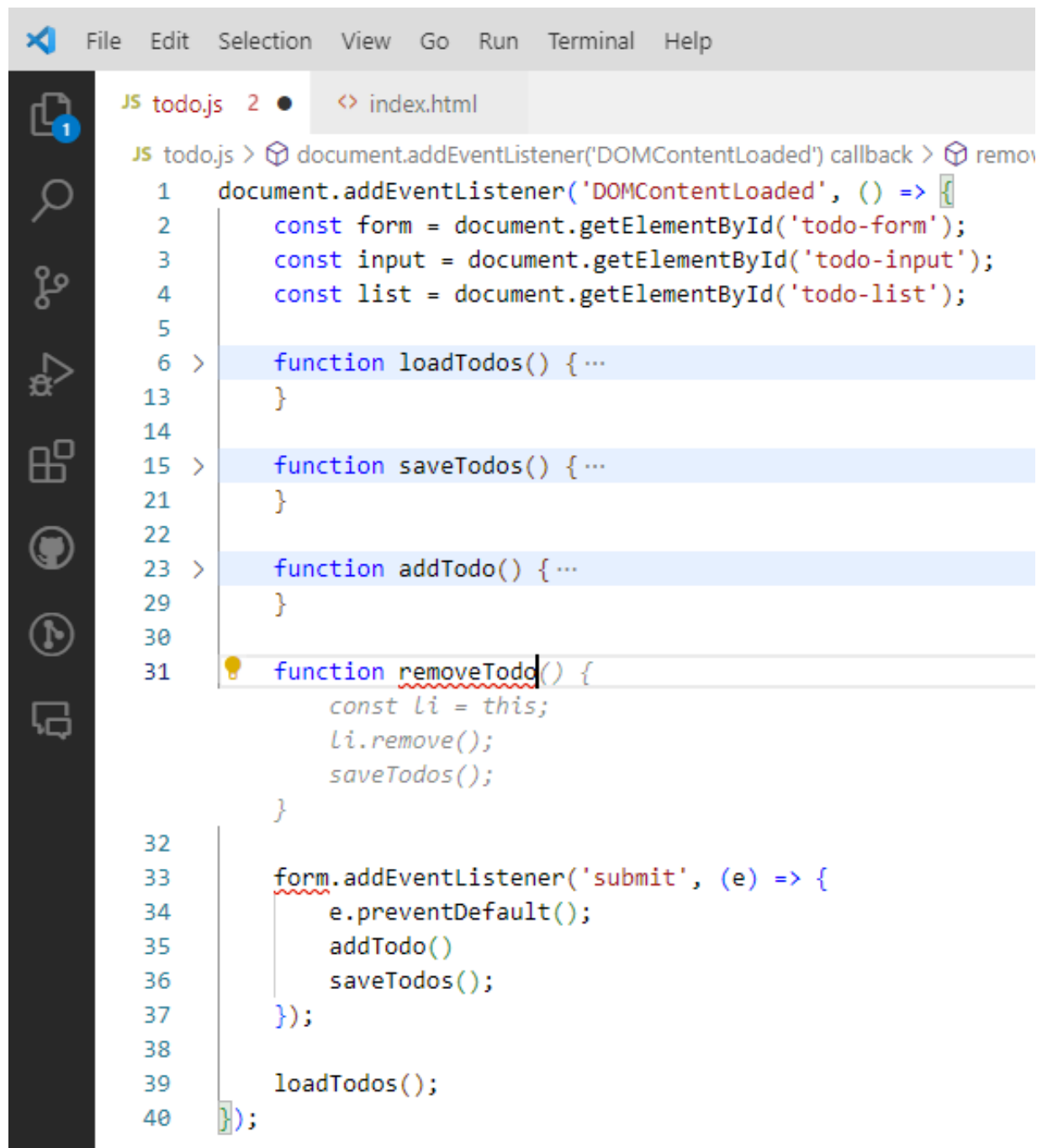
Também é preciso que o programador inclua os comandos que invocam o método:



```
JS todo.js x <> index.html
JS todo.js > document.addEventListener('DOMContentLoaded') callback > addTodo
1 document.addEventListener('DOMContentLoaded', () => {
2   const form = document.getElementById('todo-form');
3   const input = document.getElementById('todo-input');
4   const list = document.getElementById('todo-list');
5
6   > function loadTodos() { ...
14   }
15
16   > function saveTodos() { ...
22   }
23
24   function addTodo() {
25     const todo = input.value;
26     const li = document.createElement('li');
27     li.textContent = todo;
28     list.appendChild(li);
29     input.value = '';
30     saveTodos();
31   }
32
33   form.addEventListener('submit', (e) => {
34     e.preventDefault();
35     addTodo();
36     saveTodos();
37   });
38
39   loadTodos();
40 });
```

Figura 11.11: Invocando a função 'addTodo'

Da mesma forma, podemos criar a função `removeTodo`, mas repare que o código sugerido não está correto:



```
File Edit Selection View Go Run Terminal Help

JS todo.js 2 ● <> index.html

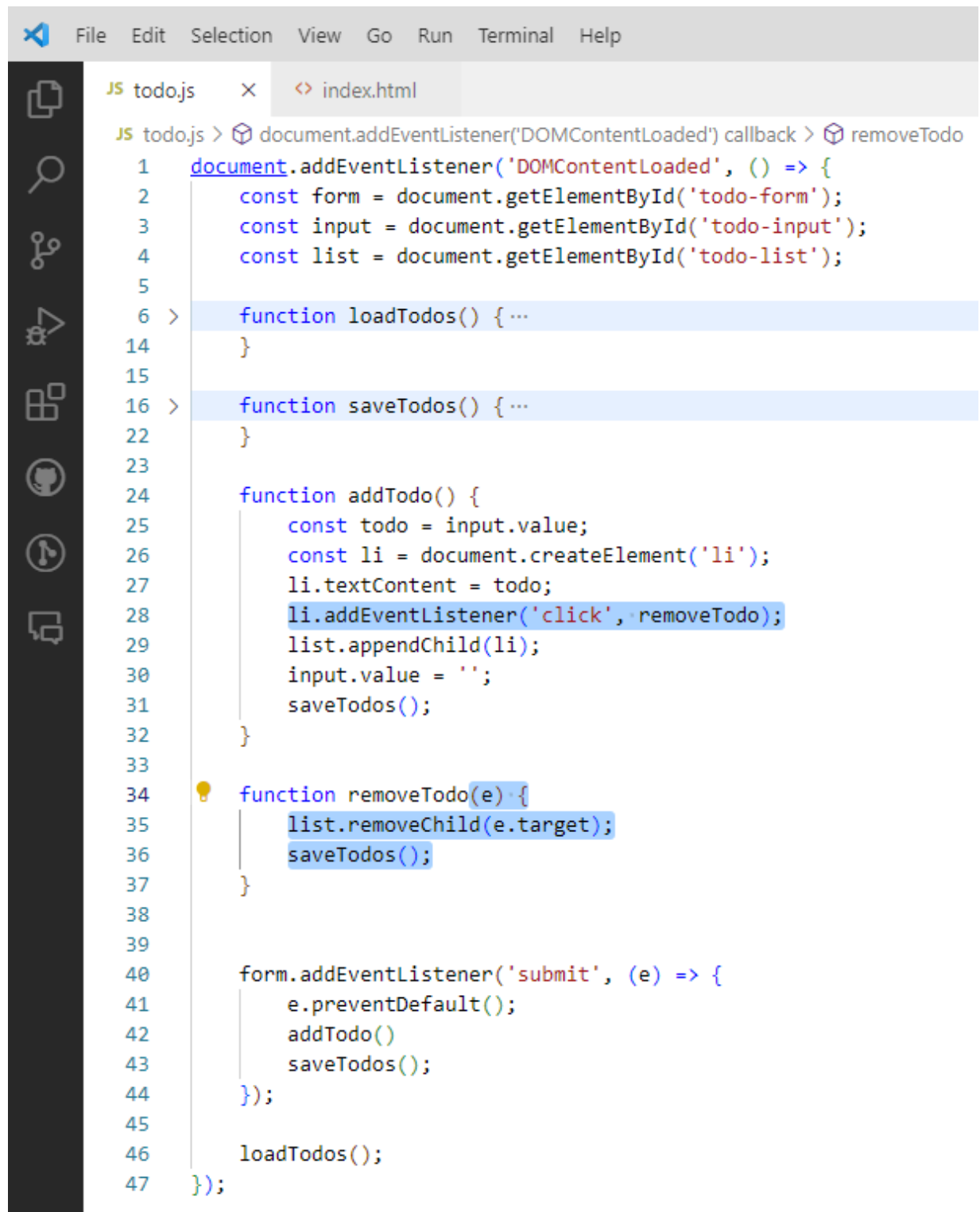
JS todo.js > document.addEventListener('DOMContentLoaded') callback > remove

1 document.addEventListener('DOMContentLoaded', () => {
2   const form = document.getElementById('todo-form');
3   const input = document.getElementById('todo-input');
4   const list = document.getElementById('todo-list');
5
6   > function loadTodos() { ...
13 }
14
15 > function saveTodos() { ...
21 }
22
23 > function addTodo() { ...
29 }
30
31 function removeTodo() {
    const li = this;
    li.remove();
    saveTodos();
  }

32
33 form.addEventListener('submit', (e) => {
34   e.preventDefault();
35   addTodo();
36   saveTodos();
37 });
38
39 loadTodos();
40 };
```

Figura 11.12: Sugestão da função 'removeTodo'

Então é preciso que a pessoa programadora faça os ajustes manualmente, afinal algum trabalho temos que ter. E também é preciso incluir o comando que o invoca:



```
JS todo.js  x  <> index.html

JS todo.js > document.addEventListener("DOMContentLoaded") callback > removeTodo

1  document.addEventListener('DOMContentLoaded', () => {
2      const form = document.getElementById('todo-form');
3      const input = document.getElementById('todo-input');
4      const list = document.getElementById('todo-list');
5
6  >  function loadTodos() { ...
14  }
15
16 >  function saveTodos() { ...
22  }
23
24  function addTodo() {
25      const todo = input.value;
26      const li = document.createElement('li');
27      li.textContent = todo;
28      li.addEventListener('click', removeTodo);
29      list.appendChild(li);
30      input.value = '';
31      saveTodos();
32  }
33
34  ⚠ function removeTodo(e) {
35      list.removeChild(e.target);
36      saveTodos();
37  }
38
39
40  form.addEventListener('submit', (e) => {
41      e.preventDefault();
42      addTodo();
43      saveTodos();
44  });
45
46  loadTodos();
47  });
```

Figura 11.13: Ajustando o método 'removeTodo' e invocando-o

Em poucos minutos, já temos um aplicativo funcional, porém com um estilo muito simples, com uma *list* e a exclusão no clique do item, nada intuitivo:

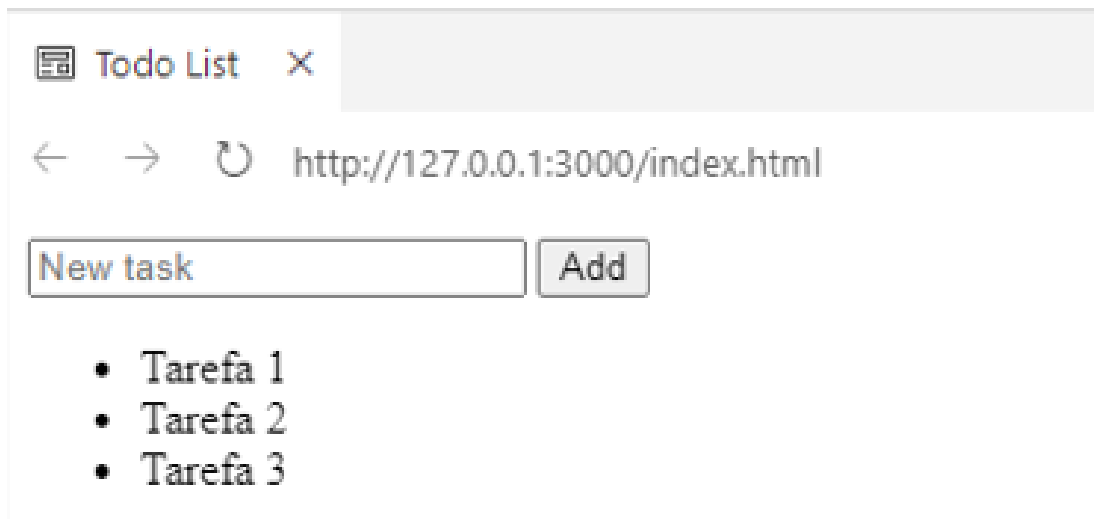


Figura 11.14: Leiaute simples

Mas aí, podemos recorrer ao prompt. Como toda a pasta do *workspace* serve como um contexto implícito do prompt, a vida da pessoa programadora fica muito mais fácil. Podemos pedir a estilização de forma muito direta, como se observa a seguir, e o código já existente é alterado integralmente:

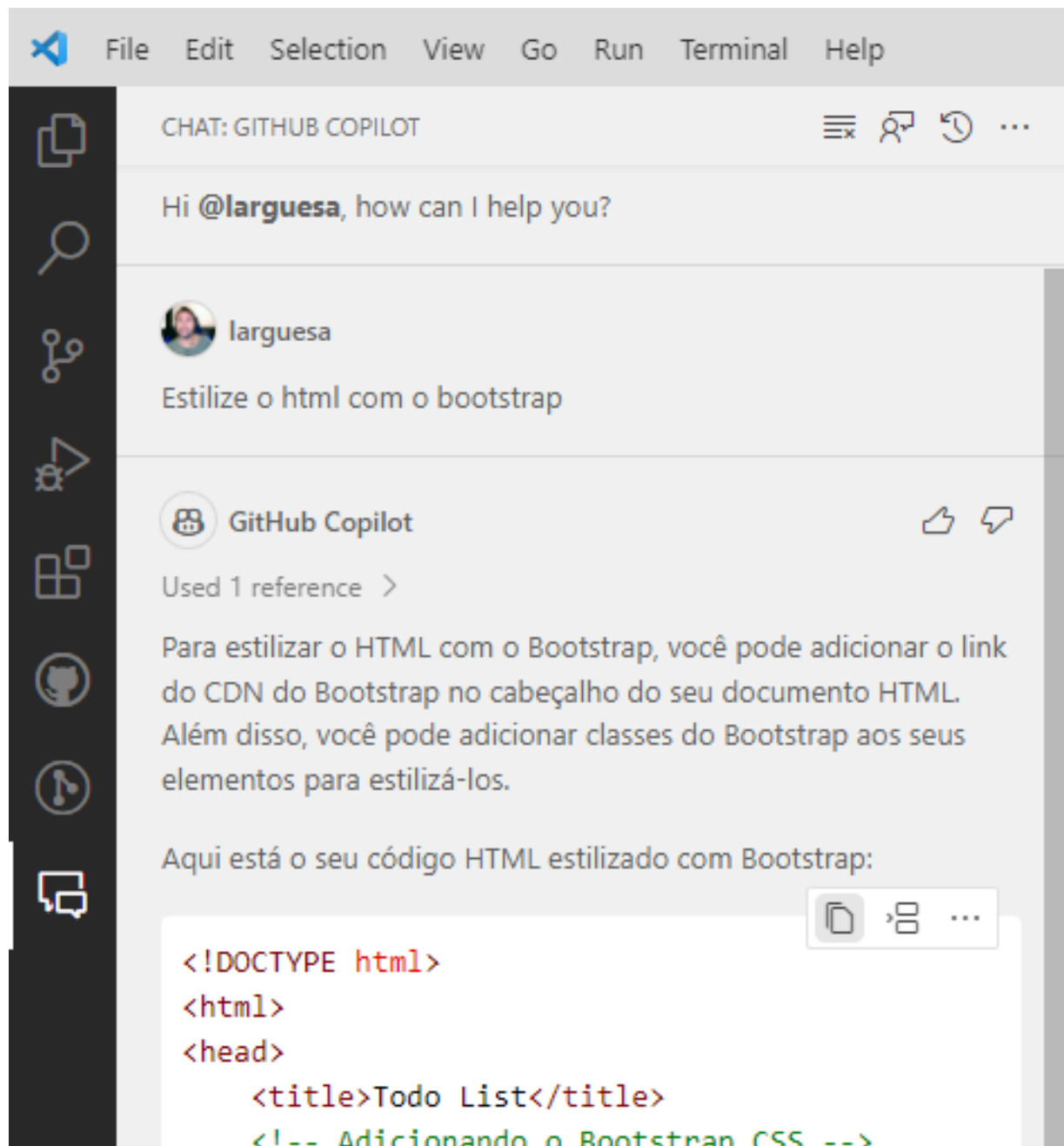


Figura 11.15: Estilizando com o bootstrap usando o prompt

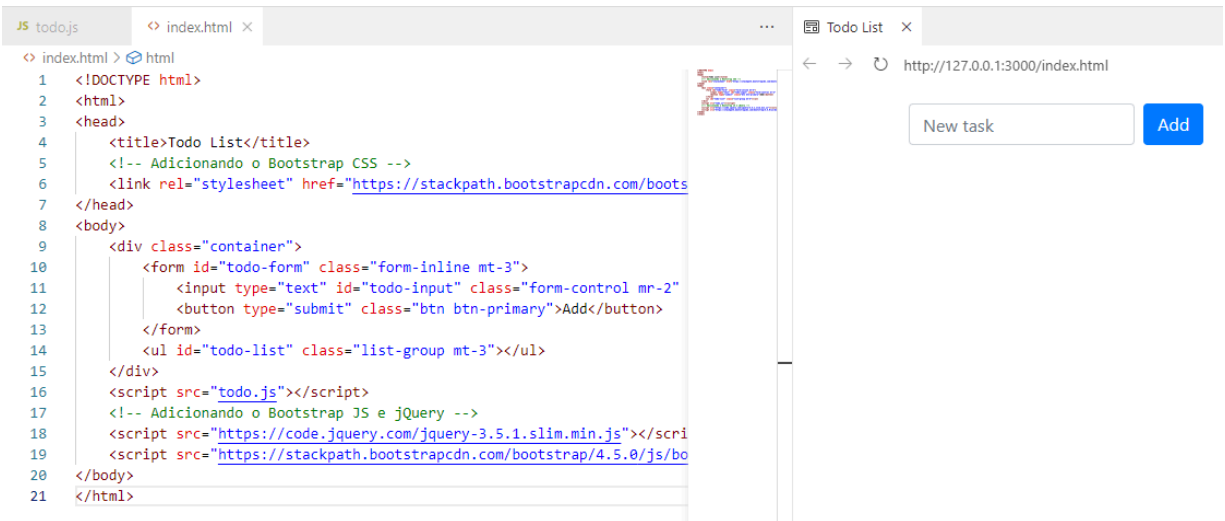


Figura 11.16: Código e leiaute estilizados

E, por fim, podemos ainda usar o prompt em sequência para pedir que substitua a lista por uma tabela e use um botão para a remoção do item:

 File Edit Selection View Go Run Terminal Help



CHAT: GITHUB COPILOT



 larguesa

Substitua o list por uma tabela e inclua uma coluna na tabela para a remoção do item ao invés do clique

 GitHub Copilot

Used 1 reference >

Primeiro, vamos planejar as alterações que precisamos fazer:

1. Substituir a tag `<ul>` por uma tabela `<table>`.
2. Adicionar cabeçalhos de coluna para a tarefa e a ação de remoção.
3. Atualizar o JavaScript para adicionar itens como linhas de tabela em vez de itens de lista.

Aqui está o código HTML atualizado:

```
<!DOCTYPE html>
<html>
<head>
  <title>Todo List</title>
  <!-- Adicionando o Bootstrap CSS -->
  <link rel="stylesheet" href="https://stackpath
</head>
<body>
  <div class="container">
    <form id="todo-form" class="form-inline r
      <input type="text" id="todo-input" cl
      <button type="submit" class="btn btn-
```

Figura 11.17: Substituindo a lista por uma tabela com o prompt

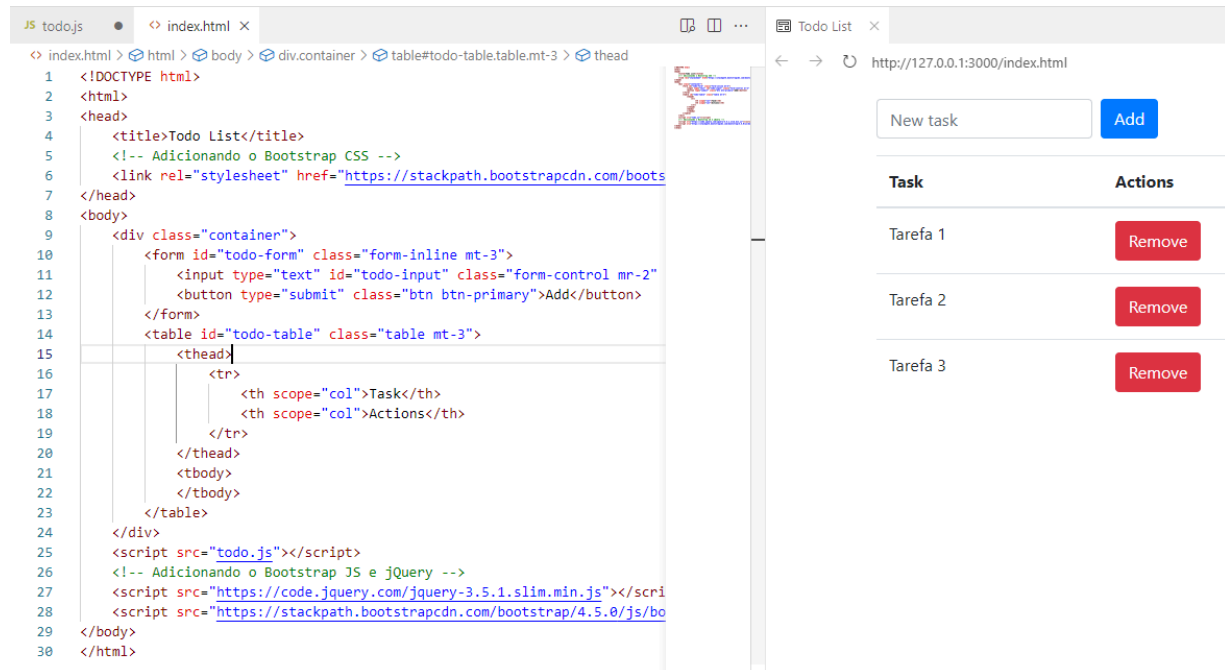


Figura 11.18: Obtendo o leiaute final

A implementação do Copilot X amplia a interação da IA em todas as etapas do desenvolvimento de software. Essa ferramenta não se limita à geração de código: ela também oferece explicações de código, ajuda na detecção e correção de erros e vulnerabilidades de segurança, e permite a interação direta no código através de comandos e ações inteligentes. A versão Enterprise do Copilot, com lançamento em 2024, oferece personalização para organizações, permitindo sugestões baseadas em códigos internos e privados, além de melhorar a segurança, privacidade e conformidade. Essa versão empresarial visa integrar totalmente o Copilot ao ciclo de vida do software, desde a concepção até a implementação.

O Copilot também se destaca pela sua capacidade de prevenção de vulnerabilidades através de um sistema baseado em LLM, bloqueando padrões de codificação inseguros em tempo real e fornecendo correções proativas. Além disso, o GitHub Copilot Partner Program busca expandir o ecossistema do Copilot, integrando-o com ferramentas e serviços de terceiros. Esse programa representa um

passo significativo na ampliação das capacidades do Copilot, permitindo uma aplicação mais vasta e diversificada da IA no desenvolvimento de software.

Esses desenvolvimentos no GitHub Copilot ilustram a evolução contínua e o impacto significativo dos Modelos de Linguagem de Código no ambiente de desenvolvimento de software, proporcionando a desenvolvedores ferramentas cada vez mais sofisticadas e integradas.

CAPÍTULO 12

Desafios e tendências futuras na Engenharia de Prompt

"A Inteligência Artificial provavelmente vai causar o fim do mundo, mas enquanto isso não acontece, existirão grandes empresas." — Sam Altman

12.1 Lidando com vieses e controvérsias nos prompts

Os vieses e as controvérsias nos prompts representam um desafio significativo no campo da Engenharia de Prompt. À medida que os modelos de linguagem são treinados em grandes conjuntos de dados, eles podem adquirir e perpetuar vieses e opiniões controversas presentes nos dados de treinamento. Neste tópico, abordaremos estratégias e práticas recomendadas para minimizar e lidar com vieses e controvérsias ao criar prompts eficientes:

- **Conscientização dos vieses:** antes de tudo, é importante estar ciente dos possíveis vieses presentes nos modelos de linguagem e no conteúdo gerado por eles. Essa conscientização ajudará desenvolvedores e analistas a identificar rapidamente quando um prompt ou uma resposta gerada pode ser tendencioso ou controverso;
- **Cuidado na formulação de prompts:** ao criar prompts, é fundamental ser cuidadoso na formulação para evitar a introdução de vieses ou a solicitação de informações

- controversas. Busque utilizar linguagem neutra e imparcial, e evite fazer suposições sobre o público-alvo;
- **Teste e revisão contínua:** é preciso testar e revisar regularmente os prompts e as respostas geradas pelos modelos de linguagem para identificar e corrigir vieses ou controvérsias. Isso pode ser feito através da análise manual ou com o auxílio de ferramentas e algoritmos desenvolvidos especificamente para detectar vieses e controvérsias;
 - **Transparência e responsabilidade:** por fim, é importante promover a transparência e a responsabilidade no uso de modelos de linguagem e engenharia de prompt. Isso inclui comunicar claramente os possíveis vieses e limitações aos usuários finais, além de estabelecer mecanismos de feedback e relatórios para lidar com preocupações e melhorar continuamente os sistemas.

Atualmente, não existe uma entidade específica para receber denúncias sobre comportamentos inapropriados de ferramentas de IA. E a aprendizagem contínua, tecnicamente, ainda é um grande desafio. No entanto, há esforços contínuos para abordar os desafios éticos e regulatórios relacionados ao uso da IA. Diretrizes estão sendo desenvolvidas para garantir a utilização ética, justa e legal da IA, focando em aspectos como avaliação constante, privacidade, segurança e transparência. Além disso, iniciativas como a Common Sense e a Mozilla Foundation estão trabalhando em sistemas independentes de avaliação de IA, enquanto autoridades nos EUA e na UE propõem legislações para regulamentar a tecnologia, visando prevenir problemas como manipulação e desinformação.

Levar em conta os desafios associados aos vieses e às controvérsias na Engenharia de Prompt é uma forma de garantir a eficácia e a responsabilidade dos sistemas baseados em IA e PLN. Adotar essas estratégias e práticas recomendadas ajudará a mitigar os riscos e a melhorar a qualidade das interações e dos resultados gerados pelos modelos de linguagem.

12.2 Avanços em modelos de linguagem e suas implicações na Engenharia de Prompt

Os avanços contínuos em modelos de linguagem estão mudando a paisagem da Engenharia de Prompt e apresentando novas oportunidades e desafios. Dentre as principais tendências e desenvolvimentos em modelos de linguagem e suas implicações na criação de prompts eficientes, destacam-se:

- **Modelos de linguagem de maior escala:** à medida que os modelos de linguagem se tornam maiores e mais complexos, eles são capazes de lidar com tarefas cada vez mais sofisticadas e abrangentes. Isso oferece novas possibilidades para a Engenharia de Prompt, mas também levanta questões sobre a eficiência e a acessibilidade dos modelos, bem como os desafios éticos e de privacidade associados à utilização de grandes volumes de dados para treinamento. Para exemplificar, os avanços do GPT-4 perante o GPT-3.5 são notáveis, e a expectativa é ainda maior em relação ao GPT-5;
- **Modelos especializados e de nicho:** o desenvolvimento de modelos de linguagem especializados e adaptados a domínios específicos está se tornando cada vez mais comum. Esses modelos podem fornecer melhor desempenho e precisão em tarefas e contextos específicos, mas também exigem uma abordagem mais cuidadosa e personalizada na criação de prompts para garantir que as respostas geradas sejam relevantes e apropriadas. Recentemente, o ChatGPT passou a oferecer na versão paga a possibilidade de o usuário criar um GPT derivado com uma espécie de contextualização predefinida, o que já é uma ação neste sentido;
- **Melhoria na compreensão do contexto e na geração de respostas:** os avanços nos modelos de linguagem estão permitindo uma melhor compreensão do contexto e uma geração de respostas mais coerentes e relevantes. Isso tem

implicações diretas na Engenharia de Prompt, pois as pessoas desenvolvedoras precisarão adaptar suas abordagens para capitalizar essas melhorias e garantir que os prompts sejam projetados para maximizar a eficácia dos modelos;

- **Abordagens híbridas e interação com outras tecnologias:** à medida que os modelos de linguagem evoluem, estão sendo desenvolvidas abordagens híbridas que combinam modelos de linguagem com outras tecnologias, como visão computacional e aprendizado por reforço. Isso apresenta novas oportunidades para a Engenharia de Prompt, mas também exige que desenvolvedores estejam familiarizados com várias tecnologias e possam projetar prompts que integrem efetivamente essas abordagens. Como exemplos, o Bing está integrado às buscas, o ChatGPT já permite a geração de imagem integrada, e assim caminha a humanidade;
- **Aprendizado contínuo e adaptação:** os modelos de linguagem ainda não aprendem em tempo real, embora o feedback e a interação com os usuários possam ser usados em treinamentos futuros. Porém, há inúmeras iniciativas de aprendizagem contínua, também conhecidas como *aprendizado ao longo da vida* ou *aprendizado incremental*. São uma área de pesquisa ativa na Inteligência Artificial e no aprendizado de máquina. Isso tem implicações importantes para a Engenharia de Prompt, pois exige a criação de mecanismos de feedback e a revisão e atualização contínua dos prompts para garantir que eles se mantenham eficazes e relevantes ao longo do tempo;
- **Explicabilidade e interpretabilidade dos modelos:** à medida que os modelos de linguagem se tornam mais sofisticados, torna-se cada vez mais importante entender como e por que eles geram certas respostas. A Engenharia de Prompt deve se concentrar na construção de prompts que facilitam a explicabilidade e a interpretabilidade dos modelos, permitindo aos usuários compreender melhor o raciocínio por trás das respostas geradas;

- **Questões éticas e regulatórias:** o avanço contínuo dos modelos de linguagem levanta questões éticas e regulatórias importantes, como o uso justo de dados, a prevenção de abusos e a responsabilidade por decisões baseadas em IA. Alguns países, como a Itália, inclusive já os proibiram. A Engenharia de Prompt deve abordar essas questões, garantindo que os prompts sejam projetados de acordo com as diretrizes e regulamentações éticas e legais aplicáveis e que promovam a transparência e a responsabilidade na tomada de decisões baseadas em IA;
- **Colaboração humano-IA:** à medida que os modelos de linguagem avançam, a colaboração entre humanos e IA se tornará cada vez mais importante. A Engenharia de Prompt deve buscar criar prompts que facilitem essa colaboração, permitindo que os sistemas baseados em IA complementem e aprimorem o trabalho humano, em vez de substituí-lo;
- **Personalização e adaptação ao usuário:** os avanços nos modelos de linguagem estão permitindo um nível crescente de personalização e adaptação às necessidades e preferências individuais dos usuários. A Engenharia de Prompt deve se concentrar em desenvolver prompts que permitam a personalização e a adaptação, garantindo que os sistemas baseados em IA sejam capazes de atender às necessidades e expectativas específicas de cada usuário.

12.3 O que será dos direitos autorais

A questão dos direitos autorais de obras criadas por Inteligência Artificial (IA) ainda é um assunto complexo e em evolução, sem um consenso definitivo. A legislação atual de direitos autorais, como a Lei 9.610/1998 no Brasil, foi criada em um contexto anterior à presença marcante da IA e tende a centralizar os direitos autorais

em pessoas físicas, criando desafios regulatórios para obras criadas por sistemas de IA. Há discussões sobre se as obras de IA devem ser protegidas juridicamente ou colocadas em domínio público, e questões sobre originalidade e criatividade de tais obras são amplamente debatidas.

Diferentes abordagens são sugeridas, como atribuir a titularidade ao programador, proprietário, usuário final, ou até mesmo considerar a criação de uma personalidade eletrônica para a própria IA. No cenário internacional, diversos modelos são propostos, e esses entendimentos variam consideravelmente, o que exige um exame crítico para verificar a viabilidade desses modelos no ordenamento jurídico brasileiro.

No cenário internacional, o debate apresenta uma série de perspectivas e considerações distintas. Nos Estados Unidos, a discussão se concentra em como os princípios tradicionais de direitos autorais, como autoria, infração e uso justo, são aplicáveis a conteúdos criados ou utilizados por IA. Existe um interesse em determinar se as obras geradas por IA podem ser consideradas de autoria humana ou se elas estão fora do atual marco legal de proteção dos direitos autorais. Além disso, há um esforço para distinguir as obras produzidas exclusivamente por humanos daquelas produzidas por ferramentas de IA.

Por outro lado, a União Europeia tem sido pioneira na consideração de novas regulamentações para lidar com os desafios impostos pela IA em relação aos direitos autorais. Algumas análises sugerem que os resultados da criação artificial por IA não deveriam ser protegidos por direitos autorais, dado que os seres humanos perderam o controle sobre o processo criativo. Isso levou à proposta de introduzir direitos conexos especiais para proteger a produção de IA contra apropriação por terceiros. Além disso, a UE propôs regras para garantir que as empresas que utilizam ferramentas de IA generativa divulguem qualquer material protegido por direitos autorais usado no desenvolvimento de seus sistemas, enfatizando a

transparência no uso de obras protegidas para treinar modelos de IA.

Globalmente, a paisagem legal para direitos autorais de IA é variada e está em evolução. Há um debate sobre se as obras geradas por IA devem ser reconhecidas como passíveis de direitos autorais e quais direitos os proprietários deveriam ter sobre tais obras. As abordagens diferem entre regiões, com algumas considerando o reconhecimento da IA como detentora de direitos *sui generis*, enquanto outras debatem sobre a necessidade de envolvimento humano para que uma obra se qualifique para proteção de direitos autorais.

Essas discussões refletem a necessidade de adaptar as leis de direitos autorais existentes às características únicas do conteúdo gerado por IA, abordando questões como autoria, originalidade e o papel da criatividade humana no processo de criação da IA. À medida que a tecnologia de IA continua a evoluir, também evoluirão os marcos legais que a regulam, exigindo adaptações e discussões contínuas por legisladores e acadêmicos.

12.4 O futuro da interação humano-IA por meio de prompts

A interação humano-IA está em constante evolução. Mas enquanto não existirem implantes cibernéticos integrando os modelos ao nosso cérebro, os prompts desempenham um papel importante nessa interação, atuando como a interface entre humanos e IA. Neste tópico, exploraremos algumas tendências e desenvolvimentos futuros na interação humano-IA por meio de prompts.

- **Conversas mais naturais e fluidas:** à medida que os modelos de linguagem avançam, podemos esperar que as conversas entre humanos e IA se tornem mais naturais e

fluidas. A Engenharia de Prompt se concentrará em criar prompts que permitam uma comunicação mais eficiente e compreensível entre humanos e IA, levando em consideração aspectos como contexto, emoção e nuances linguísticas;

- **Aprendizado contínuo e adaptação:** como vimos, no futuro os sistemas baseados em IA poderão aprender e se adaptar continuamente com base nas interações com os usuários. A Engenharia de Prompt deverá considerar essa capacidade de aprendizado, desenvolvendo prompts que permitam à IA entender e incorporar o feedback do usuário em tempo real, melhorando assim a qualidade e a relevância das respostas geradas;
- **Colaboração em tempo real:** a interação humano-IA por meio de prompts também pode evoluir para incluir colaborações em tempo real entre humanos e sistemas de IA. Isso poderia envolver a criação de prompts que facilitam o trabalho conjunto em tarefas complexas, permitindo que humanos e IA compartilhem conhecimento e recursos para resolver problemas e tomar decisões de maneira mais eficiente;
- **Assistência personalizada e proativa:** os sistemas baseados em IA do futuro podem ser capazes de fornecer assistência personalizada e proativa aos usuários, prevendo suas necessidades e oferecendo suporte antes mesmo de ser solicitado. A Engenharia de Prompt terá que desenvolver prompts que permitam à IA identificar e agir sobre essas necessidades antecipadamente, melhorando a experiência do usuário e aumentando a eficácia da interação humano-IA;
- **Ética e responsabilidade na interação humano-IA:** à medida que a IA se torna mais integrada às nossas vidas, questões éticas e de responsabilidade se tornarão cada vez mais relevantes. A Engenharia de Prompt deverá considerar essas preocupações ao criar prompts, garantindo que os sistemas de IA sejam transparentes, justos e responsáveis em suas interações com os usuários;

- **Oportunidade no campo da acessibilidade:** modelos de linguagem apresentam oportunidades no campo da acessibilidade, aperfeiçoando os avanços recentes que a tecnologia permitiu para melhorar a comunicação, educação e independência de pessoas com deficiências diversas. Eles podem facilitar a comunicação para aqueles com deficiências auditivas ou de fala através da conversão de texto em fala ou vice-versa, auxiliar indivíduos com dificuldades de aprendizagem na leitura e escrita, e proporcionar tradução em tempo real para a linguagem de sinais. Esses modelos também melhoram a navegação na web para pessoas com deficiências visuais, oferecem suporte educacional personalizado, criam interfaces mais intuitivas para tecnologias assistivas, fornecem assistência em saúde mental, e facilitam o acesso a informações. Além disso, contribuem para a autonomia e aprimoram a interação social, especialmente para pessoas com transtornos do espectro do autismo, tornando-se ferramentas valiosas para aumentar a inclusão e a acessibilidade em diversas áreas da vida cotidiana.

Em suma, o futuro da interação humano-IA por meio de prompts é promissor e oferece uma série de oportunidades e desafios. Quem sabe como essa interface pode evoluir e que elementos podem surgir para tornar esta interação mais fluida e ágil. Há um campo aberto para ser explorado.

12.5 Conclusão

Neste livro, exploramos diversos aspectos da Engenharia de Prompt e seu papel no uso eficiente de sistemas baseados em Inteligência Artificial e Processamento de Linguagem Natural aplicados ao desenvolvimento de sistemas. Abordamos os componentes fundamentais dos prompts, estratégias para criar prompts eficientes e métodos de teste e otimização do desempenho. Também analisamos a relação entre a engenharia de prompt e os modelos de

linguagem, bem como aplicações práticas em diversos setores e tendências futuras na interação humano-IA.

Ao longo deste livro, você pode estar se perguntando se utilizei o ChatGPT para auxiliar na elaboração do conteúdo, e a resposta é: claro! Quando dispomos de ferramentas que aumentam a produtividade, devemos aproveitá-las sem hesitação. Contudo, é importante usá-las de maneira inteligente. O ChatGPT não foi responsável por escrever o conteúdo, e mesmo durante as consultas houve muitas falhas. Durante o processo de discussão e elaboração do texto final, o ChatGPT tendia a se concentrar no desenvolvimento e treinamento de modelos de linguagem, fazendo com que eu tivesse de reformular constantemente o conteúdo para manter o foco na engenharia de prompt para apoio à análise e desenvolvimento de sistemas. Além disso, o ChatGPT forneceu exemplos desconexos ou incorretos em quase todas as ocasiões, exigindo diversas interações para obter exemplos reais que fossem práticos e aplicáveis. E é exatamente essa abordagem que devemos adotar em nossos projetos, utilizando a IA de maneira eficiente e adaptada às nossas necessidades. Neste processo, a IA é um copiloto, não um protagonista.

A engenharia de prompt é uma habilidade essencial para analistas e desenvolvedores de sistemas que buscam aproveitar ao máximo as capacidades da IA e do PLN. Dominar a arte de criar prompts bem formulados e eficazes pode resultar em uma produtividade sem precedentes na história da humanidade, levando a soluções mais eficientes, personalizadas e adaptadas às necessidades dos usuários.

Neste contexto, a rápida adaptação dos profissionais é necessária para se manterem relevantes no mercado. A superespecialização atual de cargos será provavelmente substituída por uma generalização apoiada pela IA. O profissional mais valioso não será aquele ultraespecializado em uma linguagem específica, mas sim aquele que domina os fundamentos para extrair e combinar códigos gerados pela IA em soluções integradas e eficientes. Paradoxalmente, a IA pode ajudar a resolver um problema latente

global: a escassez de analistas de sistemas. Com a IA, as equipes de TI poderão realizar muito mais com menos recursos.

A boa notícia é que, embora se projete a perda de centenas de milhões de empregos, também se estima um aumento do PIB global. Um relatório do Goldman Sachs no início de 2023 apontou um aumento de aproximadamente 7%, o que representa uma oportunidade para o aumento de produtividade e surgimento de postos de trabalho com maior valor agregado.

À medida que a IA e o PLN continuam a evoluir e se expandir para novos domínios e setores, a importância da Engenharia de Prompt só aumentará. Acompanhando as tendências emergentes e enfrentando os desafios e oportunidades apresentados, os profissionais envolvidos no desenvolvimento e aplicação de sistemas baseados em IA estarão melhor preparados para lidar com um futuro em constante mudança, no qual a interação humano-IA terá um papel cada vez mais importante em nossas vidas cotidianas.

Esperamos que este livro tenha contribuído para a sua preparação rumo a esse admirável mundo novo, com o perdão da inevitável referência à obra visionária de Aldous Huxley.

CAPÍTULO 13

Referências

BOWMAN, Samuel; ANGELI, Gabor; POTTS, Christopher; MANNING, Christopher. A large annotated corpus for learning natural language inference. In: *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, p. 632-642, Lisboa, Portugal, 2015. Disponível em: <https://arxiv.org/abs/1508.05326>.

BROWN, Tom B.; MANN, Benjamin; RYDER, Nick; SUBBIAH, Melanie; KAPLAN, Jared; DHARIWAL, Prafulla; NEELAKANTAN, Arvind; SHYAM, Pranav; SASTRY, Girish; ASKELL, Amanda; AGARWAL, Sandhini; HERBERT-VOSS, Ariel; KRUEGER, Gretchen; HENIGHAN, Tom; CHILD, Rewon; RAMESH, Aditya; ZIEGLER, Daniel M.; WU, Jeffrey; WINTER, Clemens; HESSE, Christopher; CHEN, Mark; SIGLER, Eric; LITWIN, Mateusz; GRAY, Scott; CHESS, Benjamin; CLARK, Jack; BERNER, Christopher; MCCANDLISH, Sam; RADFORD, Alec;; SUTSKEVER, Ilya; AMODEI, Dario. Language Models are Few-Shot Learners. *Cornell University arXiv*, 2020. Disponível em: <https://arxiv.org/abs/2005.14165>.

DEVLIN, Jacob; CHANG, Ming-Wei; LEE, Kenton;

TOUTANOVA, Kristina. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. *Cornell University arXiv*, 2018. Disponível em: <https://arxiv.org/abs/1810.04805>.

ELOUNDOU, Tyna; MANNING, Sam; MISHKIN, Pamela; ROCK, Daniel. GPTs are GPTs: An Early Look at the Labor Market Impact Potential of Large Language Models. *Cornell University arXiv*, 2023. Disponível em: <https://arxiv.org/abs/2303.10130>.

GENERATIVE pre-trained transformer. In: *Wikipédia: a enciclopédia livre*. Disponível em: https://en.wikipedia.org/wiki/Generative_pre-trained_transformer.

GITHUB/DAIR-AI. *Prompt Engineering Guide*. Disponível em: <https://github.com/dair-ai/Prompt-Engineering-Guide>.

GOLDMAN SACHS. *Generative AI could raise global GDP by 7%*. 5 abr. 2023. Disponível em: <https://www.goldmansachs.com/intelligence/pages/generative-ai-could-raise-global-gdp-by-7-percent.html>.

HUGENHOLTZ, P. Bernt; QUINTAIS, João Pedro.

Copyright and Artificial Creation: Does EU Copyright Law Protect AI-Assisted Output? *IIC - International Review of Intellectual Property and Competition Law*, v. 52, p. 1190-1216, 2021. Disponível em: <https://link.springer.com/article/10.1007/s40319-021-01115-0>.

HUGGING FACE. Hugging Face — The AI community building the future. Disponível em: <https://huggingface.co>.

LIM, Daryl. Generative AI and copyright: principles, priorities and practicalities. *Journal of Intellectual Property Law & Practice*, v. 18, n. 12, 2023. Disponível em: <https://academic.oup.com/jiplp/advance-article/doi/10.1093/jiplp/jpad081/7331468>.

MECÂNICA quântica. In: *Wikipédia: a enciclopédia livre*. Disponível em: https://pt.wikipedia.org/wiki/Mec%C3%A2nica_qu%C3%A2ntica.

MIKOLOV, Tomas; SUTSKEVER, Ilya; CHEN, Kai; CORRADO, Greg; DEAN, Jeffrey. Distributed Representations of Words and Phrases and their Compositionality. In: *Proceedings of the 26th*

International Conference on Neural Information Processing Systems (NIPS'13), v. 2, p. 3111-3119, 2013. Disponível em:
<https://arxiv.org/abs/1310.4546>.

OPENAI. GPT-4 Technical Report. *Cornell University arXiv*, 2023. Disponível em:
<https://arxiv.org/abs/2303.08774>.

PENNINGTON, Jeffrey; SOCHER, Richard; MANNING, Christopher. GloVe: Global Vectors for Word Representation. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, p. 1532-1543, Doha, Qatar, 2014. Disponível em:
<https://aclanthology.org/D14-1162.pdf>.

RADFORD, Alec; NARASIMHAN, Karthik; SALIMANS, Tim; SUTSKEVER, Ilya. Improving language understanding by generative pre-training. *OpenAI*, 2018. Disponível em:
https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

RUDER, Sebastian. *Neural Transfer Learning for Natural Language Processing*. Orientadores: John G. Breslin; Parsa Ghaffari. 2019. 311 p. Tese (Ph.D.) –

Engineering and Informatics, National University of Ireland, Galway, 2019. Disponível em: https://aran.library.nuigalway.ie/bitstream/handle/10379/15463/neural_transfer_learning_for_nlp.pdf.

WHITE, Jules; FU, Quchen; HAYS, Sam; SANDBORN, Michael; OLEA, Carlos; GILBERT, Henry; ELNASHAR, Ashraf; SPENCER-SMITH, Jesse; SCHMIDT, Douglas C. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. *Cornell University arXiv*, 2023. Disponível em: <https://arxiv.org/abs/2302.11382>.

ZHUK, Alesia. Navigating the legal landscape of AI copyright: a comparative analysis of EU, US, and Chinese approaches. *AI Ethics*, 2023. Disponível em: <https://link.springer.com/article/10.1007/s43681-023-00299-0>.